Security & Compliance

Polycarpic — a virtual Single Family Office

Owner: `seceng` agent **Status:** Approved **Version:** 1.9.19 **Last updated:** 2026-11-13

CONTENTS

1. Compliance Posture
2. Trust Boundaries
3. Threat Model (STRIDE)
4. Authentication & Authorization
5. Data Classification & Handling
6. Secret Management
7. Controls Catalog
8. CI/CD Security
9. Logging, Monitoring, Detection
10. Incident Response
11. Open Risks (Accepted)
12. Rulings on ARCH Open Questions

1. Compliance Posture

Source of truth is PRD §6 (Non-Functional Requirements) and the Principal's audience decision.

Polycarpic is software a person installs and runs for their own household. The Principal will never operate it as a service for others. That is a standing non-goal (PRD §3), not a v1 deferral. Nobody is anybody's "customer", so no formal compliance regime attaches.

That does not lower the bar. The Principal made the PRD §6 security items hard requirements, and this document treats them that way. Every control here answers a threat in §3, not an auditor. The yardstick is **OWASP ASVS 4.0 Level 2**, used as a checklist, not as a certification target.

Regime	In scope?	Why	Target evidence
GLBA Safeguards Rule	No	Applies to financial institutions that serve customers, and a self-hosted personal tool has none. It would likely attach if the hosting non-goal were reversed (R-6).	—
PCI-DSS	No	We never store, process or transmit card numbers. Aggregators expose transactions, not PANs. Adapters must not persist a PAN even if a provider payload carries one (C-34).	C-34 adapter fixture test
GDPR / CCPA	No (voluntary alignment)	Processing by a natural person in a purely personal or household activity falls under the GDPR Art. 2(2)(c) household exemption. CCPA thresholds are never met. We adopt the data-rights practices anyway (PRD §6): export, account deletion and entity purge, all scoped to the immutable ledger.	C-50, C-51, C-52
SOC 2	No	No service organization and no customers.	—
HIPAA	No	No health data by design. Payee strings can still reveal medical providers, which is one reason memo and payee text is Tier 3 (§5).	—
Aggregator terms (Plaid, SimpleFIN)	Yes, per operator	Each self-hoster obtains their own provider keys and accepts that provider's terms. Plaid production access comes with a security questionnaire. The controls here are written so an operator can answer it truthfully.	§6, C-30 to C-35

Regime	In scope?	Why	Target evidence
Tax record retention (IRS guidance, 3–7 years)	Advisory	Not a regime we enforce. The purge flow must warn the owner about it (PRD §5).	C-50 warning copy

2. Trust Boundaries

ARCH §7 cites TB-1 to TB-10, so those numbers stay stable. The OQ-1 revision (§6.3) redefines TB-4. TB-11 and TB-12 are new. Each boundary becomes a threat entry in §3.

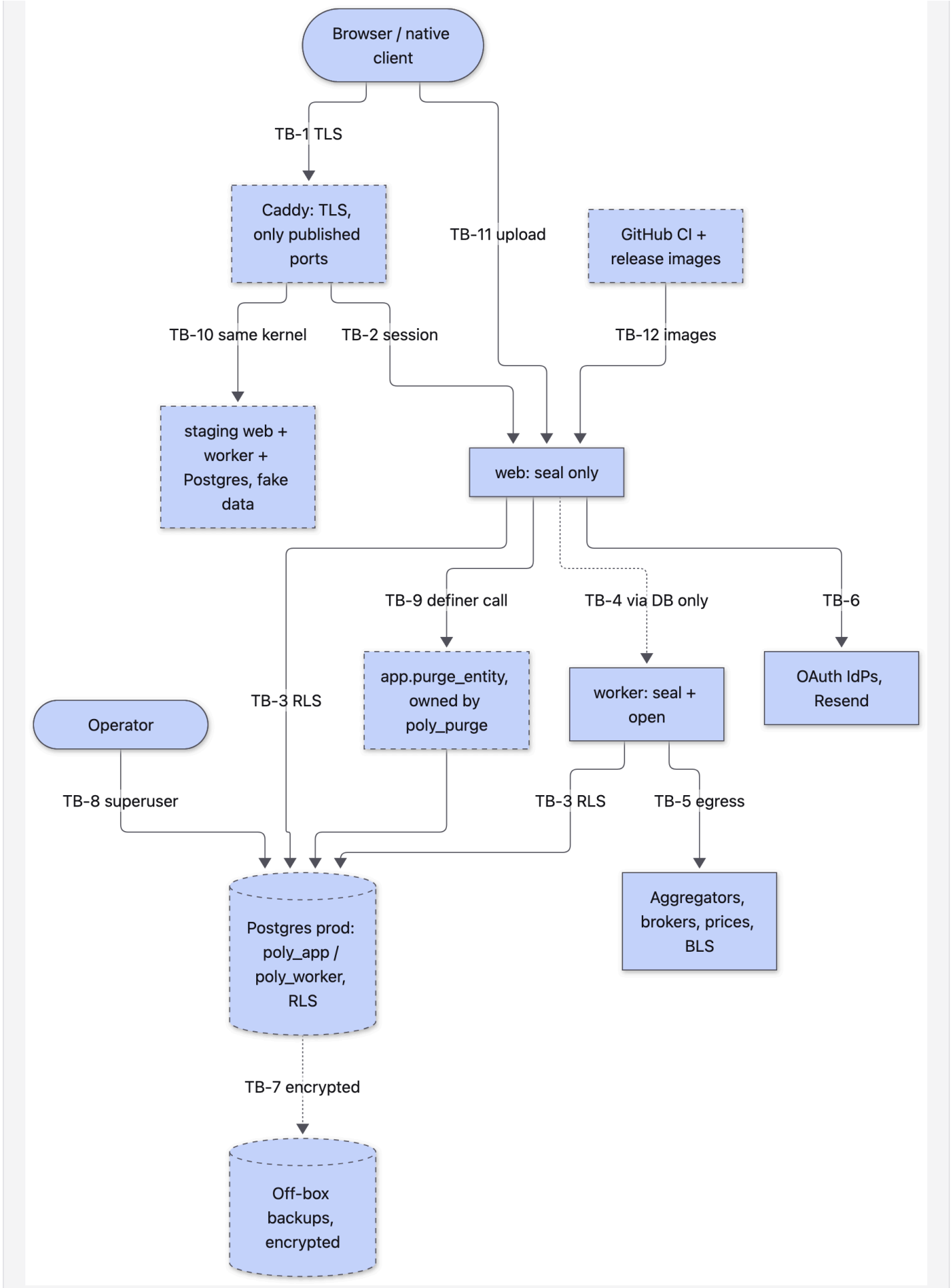
#	From	To	Boundary type	Notes
TB-1	Internet	Caddy	Network, TLS	TLS 1.2+ with HSTS. Caddy is the only container that publishes ports (80 and 443). *Local-network box (ARCH §7.8):* the "From" side is the LAN only. Nothing on the internet reaches Caddy, and the posture is C-76.
TB-2	Browser, native and MCP clients	SvelteKit `web`	Authentication	A session cookie or bearer token goes through `getSession()` and becomes a branded `SessionUserId`. Anything else is anonymous. CSRF and origin checks sit here.
TB-3	`web` / `worker`	Postgres	DB role + RLS	The primary authorization boundary. Apps connect as `poly_app` or `poly_worker`, never as the owner. Tenant context is transaction-local (§4.3). *Transport:* a connection is described by the `DATABASE_*` settings and never by a URL. The password comes from a file, and `DATABASE_URL` and `DATABASE_PASSWORD` are refused. `DATABASE_SSLMODE` must be set; nothing infers it, and a missing or unknown value refuses to connect in every process, not just in `migrate check-config`. On the single box, Postgres sits on the internal compose network and never leaves the host, so `disable` or `require` is acceptable there. `require` encrypts but verifies nothing, so it adds nothing against an attacker inside the host. Any connection that leaves

#	From	To	Boundary type	Notes
				the host, such as Neon or Vercel (POLY-121), must use `verify-full`: the certificate chain and the host name are both verified. `verify-full` currently trusts only the system CA store. A private or pinned CA needs a CA-file setting, which lands with `packages/config` (POLY-85).
TB-4	`web`	`worker` (the credential-opening capability)	Process + key separation	`web` can seal credentials but cannot open them; only `worker` holds the private key (§6.3). The two share only the database.
TB-5	`worker`	Aggregators, brokers, price and CPI providers	Egress, third-party trust	Outbound only. Credentials are opened just in time. Every response is untrusted input: payee and memo text can be written by anyone who sends the user money.
TB-6	`web`	OAuth IdPs, Resend	Third-party identity	The IdP asserts identity. Resend sees verification and magic-link URLs in transit.

#	From	To	Boundary type	Notes
			and email	
TB-7	Box	Off-box backup store	Custody	Backups are encrypted before egress, with a key separate from the credential key (§6.6).
TB-8	Operator (SSH, `docker compose exec`)	Box, Postgres superuser	Administrative	Bypasses RLS and triggers by definition (R-1). C-24 detects what it does; it cannot prevent it.
TB-9	Ordinary write path	Purge path	Privilege (DB role)	The only path that deletes posted rows. It runs inside a function owned by the NOLOGIN `poly_purge` role (§4.5).
TB-10	Staging	Prod compose	Container	Staging auto-deploys every merge to `main` and holds no real books. Its data is the fake source plus Plaid Sandbox and SimpleFIN demo data. It has separate networks, secrets and

#	From	To	Boundary type	Notes
	compose project	project	isolation, shared kernel	database. Staging has its own keyring, which seals sandbox tokens only, and it never holds prod's keyring or the production Plaid secret (C-77, R-4).
TB-14	Parent member (`poly_app`)	Aggregates of entities beneath the parent	Privilege (definer role)	The only path by which structure grants anything. Allowlisted definers owned by `poly_rollup` return fixed aggregate shapes, never rows. Their reads are confined by role-targeted policies to the caller's descendants. The cap-table and allocated-income definers (`poly_stake`) and the graph triggers (`poly_graph`) sit on the same boundary (§3.8, C-25, C-27).
TB-11	User upload (CSV / OFX)	`web` parser	Untrusted file	The credential-free import path, and a parser attack surface (C-40).
TB-12	GitHub (CI, Actions, Ren	Release imag es, prod	Supply chain	Whoever can publish an image that prod runs owns the box (§8).

#	From	To	Boundary type	Notes
	ovate, registry)	deploy		
TB-13	Operator (SSH tunnel)	Admin view listener, bound to loopback and never routed by Caddy	Administrative, out-of-band	*Placeholder; defined under POLY-81.* The design options are ARCH OQ-15: (a) a loopback route group inside `web`, or (b) a separate admin service with a read-only status DB role. The threats and controls are written when POLY-81 is ruled.


Browser / native client
TB-1 TLS
Caddy: TLS, only published ports
TB-11 upload
GitHub CI + release images
TB-10 same kernel
TB-2 session
TB-12 images
staging web + worker + Postgres, fake data
web: seal only
TB-9 definer call
TB-4 via DB only
TB-6
TB-3 RLS
app.purge_entity, owned by poly_purge
worker: seal + open
OAuth IdPs, Resend
Operator
TB-8 superuser
TB-3 RLS
TB-5 egress
Postgres prod: poly_app / poly_worker, RLS
Aggregators, brokers, prices, BLS
TB-7 encrypted
Off-box backups, encrypted

3. Threat Model (STRIDE)

One table per component group. Only threats with non-trivial likelihood × impact are listed. L = likelihood, I = impact (H/M/L). Control IDs refer to §7.

Attacker model.

- (a) An internet attacker who can reach Caddy.
- (b) A curious or malicious household member with a legitimate viewer or manager grant.
- (c) A counterparty who controls text that lands in our data. Anyone who sends the user money writes a payee or memo string.
- (d) A compromised dependency or CI path.
- (e) A thief with a backup or a disk.
- (f) An attacker with code execution in `web`. This one is in scope, and the design limits what they get.

The operator's own superuser is not modelled as an attacker (R-1).

3.1 TB-1 / TB-2 — Edge & web app (Caddy, SvelteKit, `packages/auth`)

ID	Threat	STRIDE	L	I	Control(s)
T-01	A stranger signs up on a public instance and gets an account on someone's household box. This includes an empty instance, in the window between the first deploy and the operator's first sign-up.	S / E	H	M	C-01 (no first-sign-up path; invite-based bootstrap)
T-02	Credential stuffing, spraying or brute force against password login.	S	M	H	C-03, C-04
T-03	OAuth account takeover: an IdP identity with an unverified email auto-links to an existing user.	S	L	H	C-05
T-04	Session cookie theft through XSS, subdomain cookie injection or the	S	M	H	C-02, C-07, C-41

ID	Threat	STRIDE	L	I	Control(s)
	network.				
T-05	CSRF on state-changing form actions or `/api/v1`.	T/E	M	H	C-06
T-06	Stored XSS through provider payee or memo strings, or through entity or report names set by a co-member.	T/E	M	H	C-41, C-07
T-07	Account enumeration through login, reset or signup responses.	I	M	L	C-04
T-08	A stolen session is used for an irreversible or exfiltration-shaped action: purge, export, provider link, membership grant.	S/E	L	H	C-03 (step-up), C-50, C-52. A purge runs with no delay (R-16), so the controls are step-up, the typed name, a fresh export and an immediate email to every member.
T-09	Floods against expensive endpoints: reports, export, upload.	D	L	M	C-04, C-40, C-42
T-82	A password thief signs in to an account with no second step, on their own device, and at the OQ-34 gate sets up a factor they hold. The system has no earlier factor to compare against, so it would accept the first one offered. The thief then holds a "verified" second step and passes every later step-up: export, a bank link, sharing, a role raise.	S/E	M	H	C-03 (the first factor is enrolled only after an emailed code is confirmed, bound to the session). Residual: a thief who holds the password *and* the email account. Email is already the recovery root, since magic-link sign-in, invitations (C-01) and the new-device notice (§9) all go to it, so this adds no new root.

ID	Threat	STRIDE	L	I	Control(s)
T-83	The email anchor is bypassed: BetterAuth's `/two-factor/enable`, `/two-factor/verify-totp`, `/passkey/generate-register-options` or `/passkey/verify-registration` called directly; its `otp` method (email as the factor itself) enabled; the code guessed, replayed or carried into another session; or the code sent to a new address in the middle of an email change.	S / E	M	H	C-03 (endpoint wrapper, the database guard of A-29, single-use hashed code, attempt cap, session binding, the code goes only to the current verified address), C-04
T-86	A stolen session cookie, of any age, calls BetterAuth endpoints that our step-up does not wrap. It reads every sibling session's raw token (`/list-sessions`), removes a passkey, or registers its own passkey on an enrolled account, which outlives a password reset. For an OAuth-only user (`allowPasswordless`) it can also read the TOTP secret, regenerate backup codes or turn TOTP off with no password (code-read, 1.7.18).	S / E / I	M	H	C-03 (the before-hook allowlist over `/api/auth/*`, A-34), C-02

3.2 TB-3 — Database & tenancy (`packages/db`, `packages/tenancy`, RLS)

ID	Threat	STRIDE	L	I	Control(s)
T-10	IDOR: a route accepts an `entityId` the caller has no membership on.	I / T	H	H	C-13, C-14, C-17

ID	Threat	STRIDE	L	I	Control(s)
T-11	A pooled connection carries tenant A's context into tenant B's request.	I	M	H	C-11, C-12
T-12	A new table ships without RLS: staging, draft, lot or some future table.	I	H	H	C-13 (catalog-derived battery)
T-13	RLS is bypassed through ownership: a view runs as its owner, a materialized view has no RLS, or a SECURITY DEFINER function is not on the allowlist.	I / E	M	H	C-15, C-16
T-14	A write policy without `WITH CHECK`, or an updatable `entity_id`, lets a row be inserted into or moved to another entity.	T / E	M	H	C-14
T-15	A manager grants roles, or the last owner is removed.	E	M	H	C-20
T-16	A user attaches their entity under someone else's Household, injecting rows into the victim's roll-up reports.	T / S	L	M	C-19
T-17	Existence oracle: FK checks bypass RLS, so sequential IDs let an attacker probe for other tenants' rows.	I	M	L	C-18
T-18	SQL injection in `poly_app` sets `app.user_id` to anyone. The GUC is asserted by the app, not authenticated by the database.	S / I / E	L	H	C-44 prevents it; RLS does not (R-2)
T-19	Cross-tenant metadata read through user-defined entity types. This is by design (kickoff §2.4).	I	H	L	R-9; the C-14 view exposes the label only
T-26	On a multi-owner entity (co-trustees, partners), one owner acts alone. They admit a	E /	M	H	C-28 (every other owner approves,

ID	Threat	STRIDE	L	I	Control(s)
	member or raise a role, remove a co-owner, or rewrite the cap table, possibly as a 0-unit bookkeeper owner or from a hijacked owner session.	T			enforced in the database; direct invitations revoked at the 1 → 2 transition), C-03 step-up. The founder's force path is narrow: co-owner removal or demotion only, after a decline or expiry, with notice. R-19.
T-81	A member raised to manager or owner gets the new role's power before enrolling a second step. A phished password then holds owner, or a pending owner counts toward the approver set or the 1 → 2 transition.	E	M	H	C-03 (the raise held in `pending_role` with `role` unchanged until enrolment, fail-closed), C-28 (direct-path held raises cancelled at 1 → 2)
T-20	Membership-table policies recurse through the authz functions, or FORCE RLS blinds those functions, so policies fail open or error.	E/D	M	H	C-15 (functions owned by `poly_authz`)
T-90	The sign-up path is the one place `app.user_id` is set with no session behind it (`db/auth`, ARCH §3.1). If any code can reach that step, or can reach it with an id it chose, it acts as any user: as an existing user, or as an unprovisioned one before their Person exists. The same holds for the invitation-accept step, which writes a membership that `poly_app` cannot write directly: reached for an existing user, it would seat them without their consent.	S/E	L	H	C-84

ID	Threat	STRIDE	L	I	Control(s)
	A context left on a pooled backend afterwards is T-11 again.				
T-95	An invitation token leaks: forwarded, in the browser history, in a referrer, in a cache or an access log. Its holder signs up as the invitee with a password of their choosing.	S	L	M	C-01, R-22
T-96	An IdP identity carrying an unverified email (an unverified Google address; a GitHub profile email the account does not own) signs up with no token and claims every live invitation for that address.	S / E	M	H	C-05, C-01
T-97	Invitations become an oracle: a signed-in transaction asks whether, and how many, live invitations exist for an email in another tenant, or holds their row locks.	I / D	L	M	C-01, C-15
T-98	The operator's issuing path is reached from somewhere other than the operator's command. A worker job runs as the same `poly_worker` role, with `app.user_id` set to a job principal. That job, or other code holding the worker credential, issues invitations, sets the audit actor to something else, or issues an invitation that shares an entity or carries a higher role.	S / E	L	M	C-01, C-15, C-84
T-99	The operator's printed sign-up link persists beyond the operator's terminal: in a container log (when the command runs as a container's PID 1), in the structured log, in telemetry, or in the database's statement log. Its holder signs up as the invitee (T-95).	I	L	M	C-01

ID	Threat	STRIDE	L	I	Control(s)
T-100	The bootstrap is abused. A bootstrap invitation is claimed after the instance already has users, two bootstrap runs leave two live first-user links, or a session holding the issuer's advisory lock stalls the operator.	S/D	L	M	C-01, C-15
T-91	The migration chain is tampered with or diverges. A migration with an older timestamp than the last applied one is skipped without a word (drizzle-orm 0.45.4, code-read), so a FORCE or a policy never ships. An applied file is edited after the fact. Two runs race. The `SET ROLE` the migrate client needs lands on a pooled or request-path connection.	T/E	M	H	C-85, C-13

3.3 Ledger integrity — tampering & repudiation (`ledger`, `securities`, `posting`)

ID	Threat	STRIDE	L	I	Control(s)
T-21	A posted entry is edited or deleted through the app role, an ad-hoc SQL session, or `TRUNCATE`. `TRUNCATE` fires no row triggers and ignores RLS.	T/R	M	H	C-21, C-10 (no TRUNCATE grant), C-24 (a change outside the triggers breaks the chain)
T-22	An unbalanced entry is committed.	T	L	H	C-22
T-23	Forged attribution: `actor_id` is set to another user.	R/	M	M	C-23

ID	Threat	STRIDE	L	I	Control(s)
		S			
T-24	The operator superuser disables triggers (`ALTER TABLE … DISABLE TRIGGER` , `session_replication_role = replica`) and edits history undetectably. The same applies to editing a restored backup.	T / R	L	H	C-24 (detects it, POLY-M2), R-1
T-80	The app role forges the journal sequence or the chain without a superuser. Paths: writing or pre-seeding an `entity_journal_counter` row; supplying its own `journal_number` ; firing the deferred chain trigger early with `SET CONSTRAINTS … IMMEDIATE` so lines added afterwards are never hashed; or attaching a `poly_journal` trigger function to another table.	T / R	M	M	C-24 (`poly_journal` grants, counter created by trigger, line refused once chained), C-10, C-15
T-25	A manager's `auto_post` rule posts attacker-shaped entries without review.	T	L	M	C-17 (manager+), C-23 (`actor = rule:<id>`), C-53

3.4 TB-4 / TB-5 / TB-11 — Import, adapters & credentials

ID	Threat	STRIDE	L	I	Control(s)
T-30	A DB dump or backup leaks aggregator tokens, which are standing bearer access to real bank data.	I	M	H	C-30, C-72, C-73

ID	Threat	STRIDE	L	I	Control(s)
T-31	Code execution or SSRF in the internet-facing `web` process opens every stored credential.	I / E	L	H	C-30 (`web` holds the public key only), C-31
T-32	Sealed-blob swap: someone with DB write copies the victim's sealed token into their own connection row, and the victim's bank data flows into the attacker's entity.	T / I	L	H	C-30 (AAD binds the blob to connection, entity and provider)
T-33	A token or URL leaks into logs, errors, API responses or the stored payload.	I	H	H	C-31, C-33, C-34, C-75
T-34	Unlink deletes the local row but leaves the token live at the provider.	I	M	M	C-32
T-35	A malicious CSV or OFX: XXE, billion-laughs, oversize input, spreadsheet-formula payloads.	T / D / I	M	M	C-40, C-41
T-36	Full account numbers stored in raw payloads widen the blast radius of any read leak.	I	H	M	C-34; payload readable by manager+ only (§4.2.1)
T-37	A malicious attachment (a PDF/HTML polyglot, a forged content type, a header-injecting file name) runs in our origin, or attachments fill the disk.	T / E / D	M	M	C-46

3.5 Worker & jobs (`apps/worker` , `packages/refdata`)

ID	Threat	STRIDE	L	I	Control(s)
T-40	A job with blanket cross-tenant write: one bug corrupts every tenant.	E / T	M	H	C-60 (every tenant write has a user principal)

ID	Threat	STRIDE	L	I	Control(s)
T-41	A job keeps importing after the linking user loses their grant or deletes their account.	E / I	M	M	C-60 (`principal_invalid`), C-52
T-42	An unauthenticated caller triggers the Vercel Cron internal route.	E / D	M	M	C-61
T-43	Tampered or garbage reference data (prices, CPI) skews valuations.	T	L	M	TLS, source priority, C-45

3.6 TB-9 — Export, account deletion, purge & audit

ID	Threat	STRIDE	L	I	Control(s)
T-50	Purge abuse: an owner who is not the sole owner, `poly_app` setting the purge GUC directly, a stale export, or writes made after the export.	T / E	L	H	C-50
T-51	The purge path deletes nothing and reports success. This happens under FORCE RLS with no DELETE policy.	T	H	H	C-50 (`poly_purge` role-targeted DELETE policies; a zero-rows-remain assertion)
T-52	The purge erases its own evidence.	R	L	M	C-53
T-53	Export files linger on disk or in temp storage.	I	M	H	C-51
T-54	PRD §5's "rewrite the actor attribution" on account deletion would mean an UPDATE on append-only rows.	T	H	M	C-52 (tombstone the user row, never the entries)
T-55	An exported CSV runs a spreadsheet formula planted in a payee string.	T / E	M	M	C-41

ID	Threat	STRIDE	L	I	Control(s)
T-84	A password thief deletes an account that has no second step, purging its own Person's books, or is forced to enrol a factor first and so lands in T-82.	S / T	M	H	C-52 (password plus the emailed code, no factor set-up; §4.6)

3.7 TB-7 / TB-8 / TB-10 / TB-12 — Host, backups & supply chain

ID	Threat	STRIDE	L	I	Control(s)
T-60	Postgres or `web` is exposed publicly, because Docker-published ports bypass `ufw`.	I / E	M	H	C-70, C-71
T-61	A stolen disk, snapshot or backup exposes everything.	I	L	H	C-72 (on Hetzner, a detached volume only), C-73
T-62	A box compromise also deletes or encrypts the off-box backups.	D / T	L	H	C-73 (Object Lock in compliance mode; the box key is put-only; the read key is off the box)
T-63	A compromised npm release (worm or typosquat) runs install scripts in CI or in the image.	E / I	M	H	C-80, C-82
T-64	A GitHub or Actions compromise gets prod to run its image: by publishing an image or a release, by moving a tag, or by changing the signer list or the deploy script that the box fetches.	E	L	H	C-81, C-83 (attestation pinned to the repository and workflow; the tag signed by a key in the operator's own `allowed_signers`; the tag's commit bound to the attested image; GitHub holds

ID	Threat	STRIDE	L	I	Control(s)
					no credential that reaches a box)
T-92	Someone pushes a branch (any branch: a feature branch, or an agent's) whose `build.yml` has been edited to build that branch. It runs with `packages: write` and `id-token: write`. Its attestation still names `.github/workflows/build.yml` in this repository, so a check on the signer workflow alone accepts an image that never went through `main`'s review.	T/E	M	H	C-83 (`--source-ref refs/heads/main` on every verify; `build.yml` runs only for `main`), C-81
T-93	Shell injection through pull-request metadata. A branch name, title or label containing `$(…)` or backticks is placed by a `${{ }}` expression inside a `run:` script, and runs as shell in the job. Git allows both in branch names.	T/E	M	M	C-81 (event data reaches scripts only through `env:`; the job holds a read-only token and no secrets)
T-94	Cache poisoning. A cache entry that a PR or an earlier run wrote (buildx layers, pnpm store, `.turbo`, Trivy DB) is restored into the build that gets attested. The attested image then carries content that no commit holds.	T	L	H	C-81 (the attested build restores no cache; the PR cache is keyed by the lockfile hash only, and frozen-lockfile re-checks integrity)
T-65	Code merged but not yet released runs on staging on the prod kernel, then escapes its container.	E	L	H	C-70, C-74, R-4
T-66	A secret is committed to the repo or baked into an image.	I	M	H	C-82, C-74

ID	Threat	STRIDE	L	I	Control(s)
T-67	The local box, meant to be LAN-only, becomes reachable from the internet. Causes: a router UPnP auto-forward, a home router that allows inbound IPv6 to the box's global address, or a Docker-published port bypassing the host firewall.	I / E	M	H	C-76
T-68	The DNS-API token on the local box, used for the DNS-01 certificate, is stolen. The thief can issue certificates for the zone, or redirect the name for clients that resolve it publicly.	S / I	L	M	C-76 (token scoped to one zone, a file secret mounted into Caddy only); R-18
T-79	Private-CA mode on the local box. The operator's root certificate is installed on every device, so whoever holds its signing key can impersonate *any* website to those devices, banks included.	S / I	L	H	C-76: a name-constrained root, the root key offline, only an intermediate on the box. R-20.
T-69	The whole local box is stolen from the home.	I	L	H	C-72: the TPM unlock is sealed to Secure Boot. The disk alone is unreadable elsewhere, and the whole box boots only to an authenticated login. C-73: backups are encrypted separately. R-21. See T-85 for a thief who opens the case.
T-85	Offline tamper on the home box. Someone with the box for about an hour takes out the SSD, writes to the unencrypted root partition (an SSH key, a	T / E / I	L	H	C-72: the root partition is encrypted under the same TPM unlock (one container with the data; method proven

ID	Threat	STRIDE	L	I	Control(s)
	service) or rebuilds the unsigned initramfs, and puts it back. The Secure Boot state is unchanged, so PCR 7 unseals the data partition and their code runs with the disk unlocked. The Secure-Boot-off test does not detect it.				at OQ-INFRA-10), so the file edit does not work. Residual, accepted: rebuilding the unsigned initramfs on the unencrypted boot files, unless they can be measured (OQ-INFRA-2). R-21.
T-76	Staging, which holds a keyring and sandbox tokens, is configured with prod's keyring, the production Plaid secret or a real SimpleFIN token. Real bank access then lands on the box that runs unreleased code.	I / E	L	H	C-77, C-74
T-88	An operator console account, or a provisioning token left on the laptop, is taken over. Hetzner gives the rescue console and the volume. Cloudflare gives the name, and through it a certificate. Backblaze gives the backup store (not deletion within 35 days). GitHub gives the release path, short of the signing key.	S / E	L	H	C-78, C-83, C-73
T-89	The backup login `poly_backup` (REPLICATION, BYPASSRLS, read-all) is stolen or misused. It reads every tenant's rows over SQL, including BetterAuth's password hashes and session tokens (in the clear, R-12) and the sealed credential blobs (inert without the keyring). Through the replication protocol it copies the whole cluster. A permanent	I / D	L	H	C-79 (backup-only network with no internet, `pg_hba` by that network, read-only default, no write grant, upload split from the credential, slot and WAL caps, the slot alert), C-10

ID	Threat	STRIDE	L	I	Control(s)
	replication slot it creates holds WAL until the disk fills and Postgres stops.				

3.8 TB-14 — Nesting, roll-up & cap table (`poly_rollup`, `poly_stake`, `poly_graph`)

ARCH §3.1a and §3.3a–c. Access stays per entity. These are the only paths by which structure or equity yields any figure about an entity the caller holds no membership on.

ID	Threat	STRIDE	L	I	Control(s)
T-70	A parent member with no membership on a child reads the child's rows (entries, lines, payees, memos, lots, positions) through a roll-up definer, the holdings popover, or a bug in a definer body.	I	M	H	C-25
T-71	Structure enumeration: a non-member reads `entity_rollup` rows, or ancestor names, beyond the ruled grant (members of the parent; the child's owner sees parent names only).	I	M	L	C-19, C-25
T-72	Time-slicing: calling the roll-up, economic or allocated-income definer for one-day periods, or comparing balances on consecutive days, rebuilds near-transaction-level detail of an	I	M	M	Consent disclosure (C-25): the inclusion step and the upward-consent notice state that any period is visible. No period guard. R-17.

ID	Threat	STRIDE	L	I	Control(s)
	entity the caller is not a member of.				
T-73	The audience of an entity's totals grows without its owner knowing. It grows when the entity's parent is placed under another parent (consent travels upward), or when a member is added to an ancestor.	I	M	M	C-25: a placement notice and an ancestor-invite notice, both written by `poly_graph` triggers and both carrying the consent-disclosure sentence and the pull-out link. Also `app.parents_of` and pull-out. Notify, never block.
T-74	The cap-table candidate list, or the holder notification, leaks names of Persons or entities to someone with no relation to them.	I	L	M	C-27
T-75	A holder's owners, who have no membership on the held entity, read other holders' rows or the held entity's books.	I	M	M	C-27
T-77	A cycle check or holder-graph check runs under a role that cannot see the whole graph, so a cycle through entities invisible to that role is accepted. Roll-ups then double-count, and allocations loop.	T	M	M	C-25 (graph triggers owned by `poly_graph`, with a graph-wide read)
T-78	A cycle or depth refusal reveals that a hidden link exists.	I	L	L	R-15. The inserter must own both ends, and the error names no entity.

4. Authentication & Authorization

§4.3–§4.5 are the rulings on ARCH OQ-2, OQ-3 and OQ-4. OQ-1 is ruled in §6.3. §12.1 indexes them.

4.1 Authentication

- **Provider.** BetterAuth in our own Postgres, on its built-in Kysely path over our routing pool (A-31), not the Drizzle adapter (below). Its tables are declared in the Drizzle schema and share our migration chain. Sign-in methods:
  - Email + password, hashed with scrypt (the BetterAuth default).
  - OAuth 2.0 with PKCE and a verified `state`, via Google or GitHub. Sign in with Apple arrives with the native iOS milestone (POLY-122, unscheduled).
  - Passkeys.
  - Optional magic links sent through Resend.
- **Invite-only registration (C-01).** Every account starts from an invitation, the first one included.
  - **No first-sign-up path.** On an empty instance, sign-up and OAuth sign-in without a valid invitation are refused, exactly as on any other instance. Why: "the first user on an empty instance becomes the bootstrap user" is a race on a public VPS between the first deploy and the operator's first sign-up, and a stranger who wins it holds the instance.
  - **The bootstrap invitation.** The operator runs `polycarpic user invite <email> --bootstrap` on the box. It issues the same single-use, email-bound, expiring invitation as a plain `user invite` (below) and writes an `audit_event`. It refuses if any user exists, so the runbook step cannot be replayed on a live instance by mistake. The bootstrap user gets only their own Person. There is no in-app administrator.
  - INFRA also keeps prod behind a maintenance page until the operator opens it (INFRA §7.1 steps 14 to 17). That narrows the window, but it is not the control: the refusal above is.
  - Self-signup is off. New users arrive only through a membership invitation from an entity owner.
  - OAuth sign-in for an unknown email with no pending invite is rejected.
  - **No switch reopens public sign-up**. No configuration flag or environment variable enables self-signup, so there is nothing to flip by mistake or through a configuration compromise.
  - **The invitation token.** 32 bytes from the CSPRNG, carried as 43 characters of canonical base64url. Anything that does not decode to exactly 32 bytes and re-encode to the same string is not a token. Only `sha256` of the decoded bytes is stored (`invitation.token_hash`, exactly 32 bytes by a check constraint), and `poly_app` cannot select it. The plaintext is never stored or logged.

        - A *live* invitation is unclaimed, unrevoked and unexpired. Claiming binds it to one user, so it works once. A claimed invitation is no longer live, and a revoked one is refused. `revoked_at`, `claimed_user_id`, `claimed_at` and `accepted_at` are each set once and never change afterwards.
        - The link is `/sign-up?invitation=<token>`. That page answers with `Referrer-Policy: no-referrer` and `Cache-Control: no-store`, and it removes the token from the address bar once it has read it. The token reaches the server only in the sign-up request body. Caddy's access log drops the query string (INFRA).
        - Every refusal is the same 403 `invitation_required`: no token, a malformed, unknown, expired, revoked or claimed token, a token for another email, or a lock timeout. It carries no database text. The check runs before BetterAuth looks up the email, so a refusal never says whether an account exists. On the OAuth and magic-link routes, every refusal redirects to the same error target.
    - **What proves the email.** On `/sign-up/email`, the token is the proof: only the invitation it names is claimed, and the new account's email stays unverified. On `/callback/:id` and `/magic-link/verify`, there is no token, and an email the IdP verified, or the magic link, is the proof. A token-less sign-up claims every live invitation for that address, and the database refuses it unless `user.email_verified` is true. The route decides which mode applies, never a body field.
    - **What sign-up seats.** A viewer invitation on an entity seats a viewer membership. An invitation with no entity gives only the user's own Person. A manager or owner invitation is claimed but not seated: no membership is written until the invitee has enrolled a second step and the enrolment write seats it (C-03, C-28). The user lands on the newest entity they were seated on, or else their own Person. A claimant reads their own memberships, never an invitation row.
    - **An invitation that shares no entity:** `polycarpic user invite <email>`.
        - Runs on the box only, by the operator, and writes an `audit_event`.
        - Creates a single-use invitation with the same expiry as an entity invitation.
        - The invitee gets only their own Person entity.
        - No entity is involved, so neither the co-owner approval (§4.2.2) nor the ancestor-invite notice applies.
- **MFA (C-03).** TOTP and passkeys, with single-use backup codes. BetterAuth stores the codes encrypted (XChaCha20-Poly1305 under the auth secret), not hashed. It checks a code by decrypting the list, so hashing them is not an option.
    - A second step (TOTP or a passkey) is required for any user holding owner or manager on any entity, with the one exemption below. It is optional for viewers (R-7). OAuth-only users

must enrol a passkey or TOTP to hold those roles, because we cannot verify the IdP's MFA.

- **The own Person is exempt until the first real action (OQ-34).** Every user owns their own Person from sign-up, so without an exemption the rule above would bind everyone, viewers included. The owner row on a user's own Person (`entity.self_user_id`) is exempt while that Person is unshared: it has no other member, and it is not rolled up into a parent entity (A-28). Enrolment is forced, and the action is not performed, before any of these:
  - creating a Trust, Business, Household or Holding (the creator's owner row is refused);
  - sharing the own Person: the first invitation on it, or linking it under a parent, whose members would then see it (C-25);
  - accepting an owner or manager invitation, or having a raise to manager or owner take effect (held, below);
  - any step-up action on the own Person (export, purge, granting access, linking a bank), because step-up needs a second factor.

  Viewing shared entities never triggers it. Unenrolled, the user can read and keep their own unshared books by hand, and nothing more. The refusals are in the database (BEFORE INSERT triggers on `membership` and `invitation`, ARCH §3.1c), not only in the UI.
- **The first factor is anchored on the account email (T-82).** A gate that asks for the password and then accepts the first factor offered stops only someone at an unlocked laptop. A password thief can do the same steps on their own device. So a user with no factor enrols their first one only after confirming a code sent to the account's email address, as well as the password. This applies at every OQ-34 gate, at the sign-in gate after an operator `reset-mfa`, and to an OAuth-only user, whose fresh IdP round trip stands in for the password.
  - **Order.** Password (or the fresh round trip), then the emailed code, then the factor. The code is asked for before any TOTP secret is shown or any passkey ceremony begins, so a thief learns nothing and stores nothing.
  - **The code.** Six digits from a CSPRNG, also offered as a link. It lasts 10 minutes and is used once. It is stored hashed as a T1 verification token and never logged. Five wrong entries void it. Sends are limited to 3 per 15 minutes per user (C-04). The response is the same whether or not a send was suppressed.
  - **Bound to the session.** The code, and the enrolment grant it opens, are keyed to the session that started the set-up. A code entered in another session, or after a new sign-in, is refused. The grant lasts 10 minutes and is consumed by the first factor write.
  - **Only the current, verified address.** Never a pending new address: changing email is itself a step-up action, so an unenrolled user enrols first, against the old address.

- **Not completed until confirmed.** No TOTP row is marked verified, and no passkey row is written, without a live grant for that user and session. `second_step_at` stays null, and `app.promote_held_roles()` does not run. The gate is in the app, which wraps BetterAuth's enrolment endpoints, and in the database (A-29), so a forgotten route fails closed.
- **Email is not a factor.** BetterAuth's `otp` two-factor method is not configured. An emailed code anchors the first enrolment, and never stands in for the second step.
- **Once a factor exists, nothing changes.** Adding a second method, and every step-up, uses the existing factor. The email anchor is for the first factor only.
- **Residual.** A thief with both the password and the email account can enrol. Email is already the recovery root (magic-link sign-in, invitations, the new-device notice in §9), so this adds no root. The user is emailed when a factor is enrolled (§9).
- **BetterAuth, measured (better-auth 1.7.18 and @better-auth/passkey 1.7.18, POLY-62 spike).** Where the docs and the code differ, the code was taken as correct. Paths are under `node_modules/better-auth/dist/`.
  - **No native email-confirmed enrolment.** The two-factor and passkey plugins have no such step. `emailOTP` offers only `email-verification`, `sign-in`, `forget-password` and `change-email`. Its codes are keyed on the email address, not the session, and `/email-otp/check-verification-otp` needs no session. It cannot be chained as a precondition, so the app's `email_challenge` and grant are the only path.
  - **Enabling two-factor** (`plugins/two-factor/index.mjs`). `/two-factor/enable` refuses a missing password (400) and a wrong one (`INVALID_PASSWORD`). It does not ask for a fresh session. With the `otp` method unconfigured, `method: "otp"` is refused (`OTP_NOT_CONFIGURED`). Enabling writes a `twoFactor` row with `verified = false`, an encrypted secret and encrypted backup codes, and leaves `user.twoFactorEnabled` false. It returns the TOTP URI and the backup codes at once, so the gate must sit on `/enable`. `skipVerificationOnEnable` defaults to false; the row is written with `verified = false`.
  - **Backup codes before TOTP is verified.** A signed-in session can redeem one at `/two-factor/verify-backup-code` (measured: 200), and nothing flips. BetterAuth's sign-in does not ask for a second step while `twoFactorEnabled` is false. So a backup code is never a step-up proof unless `second_step_at` is set (C-03).
  - **The first TOTP verify replaces the session.** `/two-factor/verify-totp` (`totp/index.mjs`) runs these steps in order: it sets `user.twoFactorEnabled`, creates a new session, deletes the requesting one, and then sets

`twoFactor.verified`. When the last UPDATE runs, the requesting session no longer exists, and a grant that cascades from it is already deleted (measured). On BetterAuth's own pool these are four separate autocommits.

- **Passkeys** (`@better-auth/passkey`). There is no `/passkey/add-passkey`. Registration is `/passkey/generate-register-options` and `/passkey/verify-registration`. Both require a session no older than `session.freshAge` (default 24 hours; a 25-hour-old session got `SESSION_NOT_FRESH`), and no password. "Fresh" means the session's age, not a re-authentication. A session-only call wrote a passkey row (measured, with a software authenticator). `registration.afterVerification` runs before the row is written and can refuse.
- **Hooks.** `databaseHooks` run only for the core models: `user`, `session`, `account` and `verification`. The plugins write `twoFactor` and `passkey` through the raw adapter, and no hook ran for either (measured). So a database hook cannot be the enrolment gate. A global `hooks.before` sees the path and the session id, and when it throws, nothing is written (measured on `/two-factor/enable` and on passkey options). It runs in the process, not in a database transaction.
- **Connection.** On its own pool, BetterAuth's `twoFactor` INSERT ran on a different backend and transaction from our open `withTenant`, and the trigger read `app.session_id` as null (measured). BetterAuth accepts any object with `connect()` as a Postgres pool. A 30-line routing pool hands it our open transaction's client, and turns its `BEGIN`/`COMMIT` into savepoints. With that pool, the `twoFactor` and `passkey` triggers saw the GUC on our backend and in our transaction. Rolling ours back removed BetterAuth's row. A trigger that refuses without the GUC blocked a write outside `withTenant` and allowed one inside it. The first-verify sequence ran inside one transaction, and passkey registration with `createSession` ran through savepoints (all measured). This is A-31.
- **Passwordless users.** By default `/two-factor/enable` always asks for a password (`utils/password.mjs` `shouldRequirePassword`; code-read). An OAuth-only user could not enrol TOTP. `allowPasswordless: true` skips the password only for users with no credential password. The before-hook must then require the fresh IdP round trip (A-33).

- **A raised role waits for enrolment (mechanism ARCH §3.1c).** When a member without a second step is raised to manager or owner, `membership.role` stays the role in force (viewer), and the raise is parked in `membership.pending_role`, with who raised it and, on the approval path, the applying `change_request`. Nothing that authorizes reads `pending_role`: `has_role`, the RLS policies, `app.owner_count`, the §4.2.2 approver set,

the 1 → 2 transition and founder eligibility all see viewer, so a check that forgets the pending state fails closed (C-03, T-81). A-25 is met:

- `user.second_step_at` is written only by a `poly_govern` definer trigger that recomputes it from BetterAuth's factor tables. `poly_app` cannot set it.
- The only write that makes the role effective is `app.promote_held_roles()` (`poly_govern`), in the enrolment's own transaction. It re-checks for a factor row, changes only the role columns, and writes a `role.promoted` `audit_event`. An approved co-owner raise is not re-approved.
- An INSERT of manager or owner for an unenrolled user is refused, so an owner or manager invitee enrols before the row exists and is never a held viewer.
- Any owner may cancel a held raise as a lowering, with no approval. A held raise does not expire.

The edges:

- **A new owner never inherits a raise they had no vote on.** At the 1 → 2 transition, a held raise the sole owner made directly is cancelled along with the direct invitations, in the same transaction and under the same entity-row lock (C-28), and its raiser is notified. A held raise applied through `app.apply_change` stays, as an invitation applied that way does, because the owners at the time approved it.
- **The last method cannot be removed** while the user holds manager or owner outside the own-Person exemption. The operator's audited `reset-mfa` is the only way past. It revokes every session, and the sign-in gate then reaches no route but Set up a second step until they enrol.
- A held owner is not eligible for a founder transfer, and the account-deletion transfer offers only enrolled members (§4.6).

- Lockout with no support desk has a cheap answer: an operator-side CLI (`polycarpic user reset-mfa <email>`). It runs only on the box and writes an `audit_event`. The operator is already the trust root (R-1).

- **BetterAuth configuration: rulings on the architect's three resolutions (on ARCH §3.1c).**
  - **(1) `enrolment_grant.session_id` is `ON DELETE SET NULL`: confirmed.** With `CASCADE`, the first `/two-factor/verify-totp` deletes the requesting session, and with it the consumed grant, before the `twoFactor.verified` guard runs (measured order). `SET NULL` keeps the consumed row for that guard. It opens nothing, because *live* requires `session_id` to be non-null and equal to `app.session_id`. A grant whose session ends before it is used is dead, and no later session can pick it up. The verified guard matches only `consumed_txid = txid_current()`, which is the top-level transaction id even inside BetterAuth's savepoints. Conditions:

    - the before-hook and every guard test liveness against the request's session, never "the user has a grant";
    - the kept rows hold no secret; account deletion removes them with the user's sessions.
- **(2) BetterAuth runs on the Kysely pool path only: confirmed.** The property the guards depend on, that BetterAuth's writes happen in our transaction, was measured only on that path. The Drizzle adapter takes a fixed `db` and is unmeasured. Conditions:
    - `packages/auth` passes the routing pool as `database` and does not import `drizzleAdapter` (a lint rule);
    - the A-31 integration test builds BetterAuth from the same exported configuration the app uses, never a test-only one, so swapping the adapter fails CI;
    - moving to the adapter later is a new measurement and a SecEng review.
- **(3) `session.freshAge` is 30 days: confirmed, with an amendment.** Code-read on 1.7.18: BetterAuth's freshness is the session's age, not a re-authentication. It guards only `/list-sessions`, `/unlink-account`, the two passkey registration endpoints and the disabled `/delete-user`. At 24 hours it blocks passkey enrolment on an older session and does nothing about a younger stolen one. So it is not a control, and setting it to the session maximum is acceptable, but only if our own gate covers what it touched. The enrolment grant covers a user with no factor. Nothing yet covers an enrolled user, and several session-only endpoints reach further (T-86):
    - `/list-sessions` returns every live session of the user with its token, and tokens are stored in the clear (R-12). One stolen session yields all the others.
    - `/passkey/generate-register-options` and `/passkey/verify-registration`, on an enrolled account, let a stolen session add its own passkey. A password change revokes sessions but not passkeys, so this access survives the reset.
    - `/passkey/delete-passkey` needs only a session. Removing a method is a step-up action, and the database's last-method trigger is a floor, not the step-up.
    - With `allowPasswordless` (A-33), `/two-factor/get-totp-uri`, `/two-factor/generate-backup-codes` and `/two-factor/disable` need no password from an OAuth-only user. A session alone then reads the TOTP secret, which is a permanent second factor for the thief.

  Amendment (A-34): the global `hooks.before` is an allowlist over `/api/auth/*`. Every BetterAuth path is in exactly one class, and a path in no class answers 404:

    - *Open:* sign-in, sign-up by invitation, the OAuth and magic-link callbacks, sign-out, sign-in verification of the second step (`/two-factor/verify-totp` and `/two-factor/verify-backup-code` during sign-in, passkey authentication), change-password (it already needs the current password), update-user (name and image only).

- *Enrolment:* `/two-factor/enable`, the first `/two-factor/verify-totp`, and the two passkey registration endpoints. With no `second_step_at`, they need the live grant (above). With it, they need `requireRecentAuth()`: a step-up in this session within 5 minutes, which takes the existing factor.
- *Step-up* (`requireRecentAuth()`): `/passkey/delete-passkey`, `/two-factor/disable`, `/two-factor/generate-backup-codes`, `/unlink-account`, `/link-social`, `/change-email`; `/revoke-other-sessions` (A-38).
- *Blocked:* `/get-session` over HTTP (A-38: its body carries the session token that the httpOnly cookie hides from script; the server calls it in-process); `/two-factor/get-totp-uri` (the secret is shown once, at enable); `/list-sessions` and `/revoke-session`, which carry raw tokens (the app's own session list returns rows with no token and revokes by id); `/two-factor/send-otp` and `/two-factor/verify-otp`, since `otp` is off; `/delete-user`.

A test lists the endpoints of the configured BetterAuth instance and fails on any path missing from the table, so a version bump that adds one fails CI until it is classified. **On ARCH's path table (A-38), with these amendments:**

- `/get-session` blocked over HTTP, with two conditions. (i) The blocked class is also enforced in the SvelteKit `handle`, which answers 404 before calling `auth.handler`, so the block does not rest on the before-hook's in-process discriminator (unmeasured). (ii) No load function returns the session token: the root layout returns a projection (user id, name, image, expiry), and the in-process call's `Set-Cookie`, when it refreshes the session, is forwarded on the response.
- `/revoke-other-sessions` moves to **step-up**. Open, it lets a stolen cookie sign the owner out everywhere and keep the thief's own session. `/revoke-sessions` stays open, because it also ends the caller's session.
- `/magic-link/verify` and `/callback/:id` stay open, but the second-step requirement makes no exception for them. A session they create for a user with `second_step_at` set must not pass `withTenant` until the second step is verified. Whether BetterAuth's `twoFactor` plugin covers these two sign-in paths, and not only `/sign-in/email`, is (unmeasured) on 1.7.18. The route-policy test measures it. If it does not, ARCH adds the hold.
- `/update-user`: `image` is null or a path on our own origin. An external URL is refused, because every co-member's browser would fetch a URL the user controls.
- `/request-password-reset`, `/reset-password` and `/send-verification-email` answer identically for known and unknown emails (C-04). A completed reset revokes every session of the user (`revokeSessionsOnPasswordReset: true`).

    - `/sign-in/social` and `/callback/:id` answer 404 in private-CA mode (C-76). The route policy carries that gate, and the test runs in both modes.
    - The other 13 named routes keep the class ARCH gives them, including the six new blocked paths.
- **Step-up.** These actions require a re-authentication within the last 5 minutes (password + MFA, or a passkey):
    - purge
    - entity export
    - granting a membership, changing a role, or cancelling a held raise
    - requesting, approving, declining or forcing a co-owner request (§4.2.2)
    - editing the cap table
    - transferring the founding-owner role, or stepping down from it
    - linking, relinking or unlinking a provider
    - removing a second-step method, or disabling MFA
    - changing email
    - deleting the account

  Step-up always takes the second factor. A user with none who starts one of these actions, on their own Person included, is sent to Set up a second step first, and the action is not performed (OQ-34). The one exception is deleting an account that has no factor (§4.6).
    - **The window is per session.** `requireRecentAuth()` reads a `recent_auth_at` stamp on the server-side session record, set by a successful step-up. It does not read the session's creation time.
    - **A completed first enrolment counts as a step-up.** It has checked the password (or a fresh IdP round trip), the emailed code and the new factor's own proof, in that session. That is more than Confirm it's you asks. So it sets `recent_auth_at`, and the gated action that sent the user to set-up continues without asking again, within the same 5 minutes. A failed or abandoned set-up sets nothing.
    - The unenrolled deletion stamp (§4.6) is narrower. It authorises only that deletion run, and no other step-up action.
- **Web sessions (C-02).** Server-side BetterAuth records.
    - The cookie is `__Host-`-prefixed, `HttpOnly`, `Secure` and `SameSite=Lax`.
    - Sessions expire after 7 days idle or 30 days absolute.
    - All sessions are revoked on password change, MFA reset or account deletion.
    - Membership changes need no session revocation, because visibility is recomputed from `membership` in every transaction.

- **Native and MCP tokens (later).** Named, listable, individually revocable bearer tokens, stored hashed in our own table. BetterAuth's `bearer` plugin is not this: its token is the session token, stored in the clear (R-12). Without `requireSignature: true` it accepts the unsigned token too (`plugins/bearer/index.mjs` ; code-read). If the plugin is ever enabled, it is enabled with `requireSignature: true` .
  - Stored hashed as Tier 1, and kept in the device Keychain.
  - Scoped to a *user*, never to an entity. A token grants what its user can do at the time of each request.
- **No in-app super-admin.** No application role reads across tenants. The operator's superuser is the only such principal, and it sits outside the app (R-1).
- **Service authentication.**
  - Each service uses its own Postgres role and password, delivered as files (C-74): `web` uses `poly_app` , `worker` uses `poly_worker` , migrate uses `poly_migrator` .
  - `web` and `worker` never talk over HTTP; they coordinate through pg-boss tables.
  - On Vercel, Cron calls an internal route authenticated by `Authorization: Bearer <CRON_SECRET>` , compared with `timingSafeEqual` . The route is compiled out of the `adapter-node` build (C-61).

4.2 Authorization model

RBAC over the membership join (`user` , `entity` , `role`), enforced by Postgres RLS. A user's rights come only from `membership` rows. They never come from a `user_id` column on a data table, and never from entity structure. Access is a grant; the hierarchy grants nothing.

`can(role, action)` in `packages/tenancy` exists to render a correct UI and to return a clean 403 early. It is *not* an enforcement point.

The design test: delete every `can()` call, and the system must still be safe. Any control that fails this test is app-layer-only authorization, and the security gate rejects it.

There are two exceptions, and both are enforced in database functions rather than `can()` : step-up recency (checked by the route, and required as an argument by the purge function) and sole-owner status.

4.2.1 Permission matrix (owner / manager / viewer)

Roles are the same for every entity type. The type drives tax routing and reporting, never authorization. The matrix is stored as data in `packages/tenancy` . Every row gets a pgTAP assertion that bypasses the app layer, plus an endpoint-level test (C-17).

Action	viewer	manager	owner	Enforced by
Read dashboard, balances, posted entries, lots, reports, drafts	✓	✓	✓	SELECT policy: `entity_id = any ((select app.visible_entities()))`
Read linked-account name and last-4	✓	✓	✓	Full numbers are never stored (C-34), so there is nothing to mask per role.
Read the raw provider payload (`raw_transaction.payload`)	✕	✓	✓	Policy or column grant through a manager+ view. The payload is the least-curated, most PII-dense field we hold.
Read the entity's membership list	own row	✓	✓	SELECT policy on `membership`
Create, edit or delete a draft; confirm a draft (post); reverse an entry	✕	✓	✓	Per-command policies with `WITH CHECK app.has_role(entity_id,'manager')`

Action	viewer	manager	owner	Enforced by
Edit the chart of accounts; create or edit rules, including `auto_post`	×	✓	✓	As above. `auto_post` creation writes an `audit_event`.
Upload CSV / OFX	×	✓	✓	Insert policy on `import_batch` and `raw_transaction`
Link a provider connection	×	✓ step-up	✓ step-up	Insert policy on `provider_connection`. The linking user becomes `linked_by_user_id`.
Unlink a connection	×	own links only	any, step-up	Policy compares `linked_by_user_id`. Revocation runs as a worker job (C-32).
Sidebar-IA actions (ARCH §2.4 @ 884748c). G6 in ARCH §3.2a relies on these rows.				
Refresh a provider token (relink)	×	✓ step-up, own links	✓ step-up	A relink stores a new sealed credential, so it is treated like a link (C-30). The policy compares `linked_by_user_id` for managers. Writes an `audit_event`.

Action	viewer	manager	owner	Enforced by
Map a linked account to a custodial GL account	×	✓	✓	UPDATE policy on `linked_account`: manager+. The target account must belong to the same entity (composite FK). A mapping change writes an `audit_event`, because it redirects where bank data posts.
Sync now	×	✓	✓	Enqueues a worker job and never opens the credential in `web` (TB-4). Rate-limited per connection (minimum interval set in INFRA) against provider quota and cost.
Post, Split, Group, + New entry, Correct entry	×	✓	✓	G6: `WITH CHECK app.has_role(entity_id,'manager')` on `journal_entry` / `journal_line`, with the actor forced by trigger (C-23). Correct entry is a reversal plus a new entry and never an edit (C-21). G1, G2, G4 and G5 are re-checked by the database at commit.
Rule auto-post opt-in (opting in counts as confirmation)	×	✓	✓	The opting-in user is recorded on the rule (`auto_post_by_user_id`) and is the principal for every auto-post, as in OQ-3. If that user drops below manager or is deleted, `auto_post` flips off and an `audit_event` is written, mirroring `principal_invalid`. Auto-posts pass G1–G6 like any post. Opt-in writes an `audit_event` and emails the owners.
Per-entity import schedule	×	✓	✓	UPDATE policy on the entity's import settings: manager+. The interval is bounded below by the same INFRA minimum as Sync now.
Set entity metadata (name, tax	×	×	✓	UPDATE policy on `entity`: owner

Action	viewer	manager	owner	Enforced by
classification)				
Set a parent link	×	×	✓ on child *and* parent	C-19. The `poly_graph` trigger refuses cycles and links past depth 8, and notifies every owner beneath (C-25).
Remove a parent link	×	×	✓ on child *or* parent	DELETE policy on `entity_rollup`. Writes an `audit_event`.
Read parent links; see the parents above my entity	links where a member of the parent	as viewer	as viewer, plus `app.parents_of` on an owned child	SELECT policy on `entity_rollup`: any membership on the parent, or owner on the child. `app.parents_of` returns ancestor ids and names only (C-19).
Read a parent's consolidated totals (POLY-1.1)	✓	✓	✓	`app.rollup_totals` / `app.rollup_economic`, guarded by viewer+ on the parent. Aggregates only, never rows (C-25).
Read the cap table	✓	✓	✓	SELECT policy on `equity_total` / `equity_holding` by `entity_id`. A holder's owners who are not members use `app.my_stakes()` (C-27).
Edit the cap table; read holder candidates	×	×	✓ step-up; co-owner approval if more than one owner	INSERT policy on `equity_*`: sole owner only. With more than one owner, the edit is held and applied only through `app.apply_change` (§4.2.2, C-28). `app.cap_table_candidates` is owner-only (C-27).

Action	viewer	manager	owner	Enforced by
Invite a member, raise a role, add an owner	×	×	✓ step-up; co-owner approval if more than one owner	INSERT policy on `invitation` and UPDATE policy on `membership`, owner, while the entity has one owner (the C-28 gate trigger). `poly_app` holds no INSERT on `membership`: an invitee is seated only by the accept definer, with their consent and, for manager or owner, after the second step. Otherwise the change goes through §4.2.2. The invite email is sent only once the change applies (C-20, C-28).
Remove or demote a co-owner	×	×	✓ step-up; every other owner approves, the target included. After a decline or expiry, the founding owner may force it.	Through §4.2.2 only (`app.apply_change`, or `app.force_change` for the founder). A last-owner trigger also applies (C-20). The founder cannot be removed or demoted this way.
Transfer the founding-owner role, or step down from it	×	×	founder only, step-up	An UPDATE of `entity.founding_owner_user_id` through a `poly_govern` definer. The new founder must be a current owner. Every owner is notified, and an `audit_event` is written (§4.2.2).

Action	viewer	manager	owner	Enforced by
Remove a viewer or manager; lower or give up one's own role	×	×	✓ step-up (own role: self, any role)	A single-owner action. DELETE and UPDATE policies on `membership`, plus the last-owner trigger (C-20).
Export the entity's complete books	×	×	✓ step-up	PRD §5 grants this to the owner. It is exfiltration-shaped, so it is audited (C-51).
Purge the entity's books	×	×	sole owner, step-up	C-50 (§4.5)
Read the entity's `audit_event` rows	×	×	✓	SELECT policy: owner, or `actor_user_id = app.current_user_id()`. Rows are written only by definer triggers, which set the actor from the session context, so the actor clause shows a user only what they did.
Clone or edit own report definitions; export own personal data; delete own account	self	self	self	Owner-user policies. Account deletion resolves every solely-owned entity in the flow, by transfer or purge, or the user aborts. The user is the only invoker (§4.6, C-52).

4.2.2 Co-owner approval on multi-owner entities

Co-trustees and partners govern as a group. Without this rule, any one owner, including a 0-unit bookkeeper owner, could admit anyone, raise a role or rewrite ownership alone (T-26). On an

entity with more than one owner, these changes need the approval of **every other current owner**:

- (a) inviting a member, or raising a role (viewer → manager → owner);
- (b) removing or demoting a co-owner;
- (c) any cap-table edit (`equity_total` , `equity_holding`).

Removing a viewer or manager, and lowering or giving up one's own role, stay single-owner actions. Sole-owner entities are unaffected.

- **A pending state, enforced in the database.**
  - **The request.** A `change_request` row holds entity, kind, payload, requester, created time, expiry and status (`pending | applied | declined | expired | withdrawn`). One `change_approval` row is written per approving owner, and both tables are append-only apart from the status.
  - **Direct writes.** `poly_app` 's direct writes for the actions above (an `invitation` INSERT, a `membership` raise, any UPDATE or DELETE of another owner's `membership` row, and `equity_*` edits) pass only while `app.owner_count(entity_id) = 1` . A BEFORE trigger, `app.gate_owner_changes()` , enforces this. It locks the entity row first, and refuses (`42501`) when the count is not 1, null included, or when the lock finds no row. Removing or demoting a viewer or manager, lowering one's own role, and revoking an invitation stay direct. Until `app.apply_change` exists, a multi-owner entity cannot change its owners at all.
  - **Multi-owner path.** Otherwise the only write path is `app.apply_change(change_id)` . It is an allowlisted definer owned by a new NOLOGIN, NOBYPASSRLS `poly_govern` role, holding role-targeted INSERT and UPDATE on exactly those tables. A `can()` -only gate fails the §4.2 design test.
- **Who approves.** The approvers are every current owner other than the requester, recomputed when the change is applied, never fixed when it is requested.
  - For a co-owner's removal or demotion, the target is one of the approvers, so no owner can be removed by the others alone. A deadlock is broken by the founding owner (below).
  - An approval counts only if its approver is still an owner at apply time.
  - One decline ends the request.
  - The requester may withdraw it.
- **Applying.** When the last approval lands, `apply_change` runs in that approver's transaction and re-validates the payload against current state:
  - the role still exists;
  - the cap-table Σ invariant holds;

    - the entity version the request was made against is unchanged, or there is no conflicting applied change since.

  If validation fails, the request is declined as stale and nothing applies. Only on apply does the invitation become live and its email go out.
- **Step-up.** Requesting, approving and declining each need a re-authentication within 5 minutes, with the second factor required for owners (C-03).
- **Audit and notice.** Each step writes an `audit_event`: `change.requested`, `.approved`, `.declined`, `.withdrawn`, `.expired` and `.applied`. Every owner is notified on request and on outcome. Pending requests, including an invitee's email address, are visible to owners only.
- **The founding owner (R-19 ruling).** Each entity records one founding owner, `entity.founding_owner_user_id`, set to its creator. For a Person entity that is its own user.
    - **Always an owner.** A trigger refuses any state where the founder does not hold owner.
    - **Breaking a removal deadlock.** Once a request to remove or demote a co-owner is `declined` or `expired`, the founder may force it through `app.force_change(change_id)`, a `poly_govern` definer.
        - It requires that the caller is the founder, that the kind is a co-owner removal or demotion, and that the status is declined or expired.
        - It requires a step-up within 5 minutes, with the second factor.
        - It writes a `change.forced` `audit_event` and notifies every owner, the target included.
        - The founder may raise the request themselves. The force still needs the decline or the 14-day expiry first, so the target always has warning.
    - **No other extra power.** The founder has none over invites, role raises or cap-table edits, which still need every other owner. The founder is never a target of `force_change`.
    - **Leaving the role.** The founder can only step down or transfer it in-app, as a founder-only action, with step-up and a notice to every owner.
        - Stepping down names a successor among the current owners.
        - A founder who would give up their own owner role must transfer the founder role first, if another owner exists.
    - **Account deletion.**
        - In the in-flow deletion (§4.6), a founder of an entity with other owners must name a successor among them. The tombstone transaction refuses while any entity still names them as founder.
        - On a sole-owned entity, the transfer target becomes founder. On a purge, the question does not arise.

    - **A founder who vanishes without deleting.** The backstop is the operator CLI (`polycarpic entity set-founder`, `polycarpic membership …`), which is audited and notifies every owner.
- **Edges** (SecEng rulings):
    - **An owner who never responds.**
        - A request expires 14 days after it is made, and expired counts as declined. Silence never approves.
        - Owners are reminded at day 7.
        - The requester may ask again. One open request per kind and target, and an expired request can be re-raised at most once per 7 days.
    - **An approver in the middle of account deletion.**
        - While their deletion flow runs they are still an owner and may approve or decline.
        - At their tombstone they stop being an owner, so their approvals stop counting and they stop being required.
        - Each pending request on their entities is re-evaluated. If it is now fully approved, it is *not* auto-applied: the requester is notified to confirm, with step-up.
        - Leaving an entity is giving up one's own role, so it needs no approval.
    - **The owner set changes while a request is pending.**
        - A new owner becomes a required approver.
        - A departed owner drops out, and the requester-confirm rule above applies.
    - **Bootstrap, when the second owner is added.**
        - A sole owner adds the first co-owner, by invite or by raising a member's role, as a single-owner action. There is nobody to approve.
        - The invitee's acceptance is their own consent, and they do not approve their own admission.
        - From the moment two owners exist, every further change in (a) to (c) needs approval.
        - **Invitations sent while sole owner (OQ-31, confirmed).** When the effective owner count goes from 1 to 2, every outstanding invitation the sole owner issued on the direct path is revoked, at every role, and its inviter is notified to re-raise it as a request. Otherwise an owner invitation issued alone could land after the approval rule applies.
            - An invitation records how it was issued (direct, or the `change_request` that applied it), and that column is not updatable. Approved invitations are not revoked.
            - The revocation runs in the same transaction as the membership write that makes the second owner, not in a later job. That write locks the entity row before counting

owners, so two direct owner invitations accepted at once serialize: the first makes two owners and revokes the second.
        - A revoked token is refused at acceptance with the same response as an expired one.
        - The rule fires on every 1 → 2 transition, including after an entity drops back to one owner. A raised owner still waiting for enrolment (§4.1) does not count until the role is effective. The transition, and the revocation, happen at enrolment.
    - The entity-creation default cap table, and transfer in account deletion, are sole-owner cases.
- **Pending changes and the purge.** A purge needs a sole owner, so §4.2.2 never meets it. Pending requests on a purged entity are deleted with it, and their audit events survive.

pgTAP for the sidebar-IA rows (qa-engineer, C-17). Each row gets an allow test and a deny test, run as `poly_app` with the app layer bypassed:

- A viewer cannot run any of the actions: every INSERT or UPDATE is rejected by policy.
- A manager cannot relink or unlink another user's connection.
- A mapping to another entity's account fails the composite FK.
- An auto-post whose opting-in user has been demoted to viewer writes 0 rows, and its rule ends with `auto_post = false`.
- A "Correct entry" never UPDATEs the original; it leaves one reversal plus one new entry.
- Sync and schedule below the minimum interval are rejected.

Revocation semantics. Revoking a membership takes effect on the next transaction; there is no cached authorization. The user's sessions stay valid for their other entities. Data a client has already fetched cannot be recalled (R-11). The native milestone must purge its cache on any 401, 403 or empty scope.

4.3 RLS mechanism — ruling on ARCH §3.3 (OQ-2)

These stand as designed: transaction-local GUC, STABLE SECURITY DEFINER visibility functions, ENABLE + FORCE on every tenant table, and default-deny.

Additions (a)–(d) are in ARCH:

- (a) The coverage probe is derived from the catalog.
- (b) No CREATE on `public`.
- (c) Per-command policies with `WITH CHECK` on every write; `FOR ALL` is banned.
- (d) The inversion canary stays.

Review found defects that (a)–(d) do not cover. Amendments (e)–(l) are new:

e. **FORCE RLS vs. the definer functions (latent defect, T-20).**
   - *The problem.* Under `FORCE`, the table *owner* is also subject to policies. ARCH has `visible_entities()` and `has_role()` owned by `poly_owner`, and they read `membership`, which is itself FORCEd.
   - *Why it breaks.* Once `membership`'s own SELECT policy calls `has_role()` (so that managers can list members), the chain function → `membership` policy → function makes Postgres raise "infinite recursion detected in policy". If the policy does not call it, the functions silently see only the rows the policy allows.
   - *Fix.* A `poly_authz` role (NOLOGIN, NOBYPASSRLS) with SELECT on `membership` and `entity_type` only, plus a role-targeted policy `CREATE POLICY authz_read ON membership FOR SELECT TO poly_authz USING (true)`. Both functions are owned by `poly_authz`. FORCE stays uniform and nothing gets BYPASSRLS. `poly_app` cannot use the policy, because it is not a member of `poly_authz`.

f. **The empty-string GUC (defect).**
   - *The problem.* `current_setting('app.user_id', true)` returns NULL when unset only until the GUC is first touched in a session. After any transaction-local set, the placeholder reads back as `''`, and `''::uuid` raises. That is fail-closed, but noisy and fragile.
   - *Fix.* Every policy reads `app.current_user_id()`, which is `nullif(current_setting('app.user_id', true), '')::uuid`. The pool-reuse test runs its no-context probe on a connection that has already carried a context.

g. **`TRUNCATE` (defect).** It ignores RLS and fires no row-level DELETE triggers. *Fix:* a statement-level `BEFORE TRUNCATE` trigger that raises on every append-only table, and no login role holds TRUNCATE.

h. **Pinned search_path.**
   - Definer functions use `SET search_path = ''`, and every name is schema-qualified.
   - EXECUTE is revoked from PUBLIC and granted explicitly.
   - Policies call these functions as `(select app.fn())`, so each is evaluated once per statement as an initplan.
   - A catalog allowlist names every SECURITY DEFINER function, and any function not on it fails the battery.

i. **Immutable tenancy columns.** `entity_id`, `actor_kind` and `actor_id` are left out of `poly_app`'s column-level UPDATE grants, so no row can change tenant or author. UPDATE policies carry both `USING` and `WITH CHECK`.

j. **Views.** Every view over tenant tables is created `WITH (security_invoker = true)`. Materialized views over tenant data are banned, because RLS cannot apply to them. The battery asserts both rules.

k. **Role hygiene.** `poly_app` and `poly_worker` are NOSUPERUSER, NOBYPASSRLS and NOCREATEROLE. They have no TRUNCATE, REFERENCES or TRIGGER grants and are members of no other role. The battery checks this with `pg_has_role` and `has_table_privilege`. No role but the superuser and `poly_backup` holds BYPASSRLS or REPLICATION; `poly_backup` is bounded by C-79.

l. **Non-enumerable identifiers.** Primary keys for tenant-visible rows are UUIDs (v7 is acceptable). FK checks bypass RLS, so sequential IDs would make an existence oracle. ARCH §4.1's `/entities/42` becomes a UUID.

Pool safety. `set_config(…, true)` is safe under the in-process node-postgres pool and under transaction-mode pgbouncer, including Neon's pooler. The rules that keep it safe:

- Statement-mode pooling is banned, because it breaks transactions.
- The deploy checklist asserts the pooler mode on both targets (DevOps).
- `withTenant()` refuses to nest (AsyncLocalStorage guard).
- `withTenant()` accepts only a branded `SessionUserId`. The type is exported from `packages/db`; its constructor is on the `db/session` subpath, which only `packages/auth` may import (ARCH §2.1, `layers.json`). A brand is a compile-time check only, so lint also refuses a cast to `SessionUserId` outside that module and test files (A-39).
- The raw pool is not exported. Two things keep it out of reach: the `exports` map, so a deep import fails to resolve, and the layer map, under which only `packages/db` may import the Postgres driver (ARCH §2.1). A path import would skip the `exports` map, so lint refuses every relative or absolute import that leaves a package (Import boundary, §6.3).
- Session-level `SET` is banned by lint. There is one exemption, by file path: the migrate module's `SET ROLE poly_owner`. It runs on its own client, which is never a pool, and is closed after the run (C-85, A-40).
- There is one context-free connection path, `db/auth`, which only `packages/auth` may import. It is BetterAuth's plain `poly_app` pool plus `withContextFreeTx`, and it is the only place outside `withTenant` that sets `app.user_id`. It does so once, local to the transaction, and only for the user row inserted in that transaction (C-84). It is not a third way into tenant data: it starts with no context, so RLS returns nothing until that step runs.

What RLS buys, and what it does not.

- *It buys:* authorization bugs become empty results instead of leaks. That covers a missing `WHERE`, an IDOR, a forgotten table.
- *It does not buy:* any defence against SQL injection or code execution inside `poly_app`. The GUC is app-asserted, so anyone who can run SQL as `poly_app` can set it to anyone (R-2).
- *For that case:* prevention is C-44. The blast radius is capped by C-30 (`web` cannot open credentials) and C-10 (no DDL, no TRUNCATE).

4.4 System jobs — ruling on OQ-3

A job's tenant writes run under the linking user's membership. No tenant policy has a system bypass clause. The requirements already on POLY-64 stand:

- `provider_connection.linked_by_user_id` is NOT NULL.
- `withSystem(job, connection, fn)` sets `app.user_id` to that user and `app.actor = system:<job>`. Authority and attribution stay separate.
- If that user's membership drops below manager, or their account is deleted, the connection flips to `principal_invalid`. That state surfaces on the dashboard, writes an `audit_event` and notifies the owners. It never fails silently.
- A job never falls back to running context-free. If no principal resolves, it errors.

Additions:

- **Entity-bound jobs with no connection** (the tax-period freeze) run under a current owner of the entity, chosen deterministically as the earliest owner membership, with `actor = system:<job>`. Every tenant write therefore has a human principal.
- **Scheduling** needs a cross-tenant list of targets. That list comes from one allowlisted SECURITY DEFINER function, `app.job_targets(job)`, which returns only (`entity_id`, `principal_user_id`, `connection_id`). Only `poly_worker` can execute it.
- **Reference-data writes** (prices, CPI, tax tables) go through role-targeted policies (`TO poly_worker`). `poly_app` gets SELECT only. A role-targeted policy cannot be unlocked by any GUC that `poly_app` sets.
- **Cash-in-Transit aging** is computed on read by the exceptions report, not stored by a job. That removes a job that would need cross-tenant reads.
- **`poly_worker` tenant DML** is limited to the import path:
  - `import_batch`, `raw_transaction`, `reported_balance`
  - `draft`, `transfer_match`
  - status columns of `provider_connection`
  - `journal_entry` / `journal_line`, for `auto_post` only

- `tax_period_freeze`, `audit_event`

4.5 The purge path — ruling on OQ-4

The purge function is owned by a dedicated NOLOGIN `poly_purge` role, with role-targeted DELETE policies. The guard order is ARCH's, with two guards added.

- **Why a function owned by `poly_owner` fails (T-51).**
  - A definer function owned by `poly_owner` runs as the table owner, and under `FORCE ROW LEVEL SECURITY` the owner is subject to policies.
  - Append-only tables deliberately have no DELETE policy. So the function would delete *zero rows and report success*.
  - Adding a `poly_owner` DELETE policy would instead hand the purge capability to the migration owner.
- **Why a separate role needs no credential.** A NOLOGIN role has no credential; it is only the owner identity for the definer function. The entry point is just as narrow: one function, with EXECUTE granted to `poly_app`. The only change is which role's privileges the function runs with.
- **Shape.**
  - `poly_purge` is NOLOGIN and NOBYPASSRLS, with DELETE on the purgeable tables only.
  - Each purgeable table gets `CREATE POLICY purge ON <t> FOR DELETE TO poly_purge USING (entity_id = nullif(current_setting('app.purge_entity', true), '')::uuid)`.
  - The append-only triggers allow DELETE only when `current_user = 'poly_purge'` *and* `OLD.entity_id` matches the GUC. `poly_app` can set the GUC but can never become `poly_purge`.
  - `app.purge_entity(entity_id, export_id, reauth_at)` is SECURITY DEFINER, owned by `poly_purge`, with `search_path = ''`.
- **Guards, in order, in one transaction:**
  1. Re-authentication within 5 minutes. The route checks it, and the function requires the timestamp as an argument.
  2. Sole-owner check against `app.current_user_id()`.
  3. An `export.completed` audit event for this entity, by this user, within 24 hours, carrying a SHA-256.
  4. *(New.)* No tenant write to the entity since the export's snapshot marker, so a manager's later postings cannot slip past the export. The entity row is locked `FOR UPDATE`.

5. An `entity.purged` `audit_event` is written before the first delete. It records the actor, entity id and name, the export event, per-table row counts and timestamps. `audit_event` has no FK to `entity` and no purge policy.
6. Every connection is revoked at the provider. The worker revoke job must have succeeded; a failed revocation blocks the purge and is never skipped.
7. Set `app.purge_entity` transaction-locally and delete in dependency order.
8. *(New.)* Assert zero rows remain for the entity in every purgeable table before commit. This is the fail-loud guard against T-51.

- **Not permitted:**
  - purge by a job;
  - purge through the API without interactive, typed confirmation of the entity name;
  - purge of a partial date range (that is an edit, and edits are reversals);
  - purge while a second owner exists.
- **Backups:** purged data persists in backups for up to 35 days, the store's fixed retention (R-5, INFRA §6.3). The purge confirmation says "up to 35 days in backups" (A-35).
- **Rejected alternatives:**
  - `ALTER TABLE … DISABLE TRIGGER` is DDL. It needs an ACCESS EXCLUSIVE lock and the table owner, and it affects every tenant.
  - Tombstoning is not a purge, so it breaks the PRD §5 promise.
  - BYPASSRLS is broader than needed.
- **No execution delay (OQ-23).**
  - The purge runs as soon as the guards pass.
  - Every member is emailed immediately, before the entity's rows go.
  - Recorded as R-16.
- **Roll-up junction.** The `entity_rollup` purge policy matches either column, so purging an entity removes it from every parent and the parents' owners are notified (ARCH §3.4). The cap-table rows on *other* entities that name the purged entity as a holder are open: see A-19.

4.6 Account deletion in-flow — who invokes the purge (OQ-25)

The account being deleted is the only invoker of every step. This follows ARCH §3.4a.

- **Who.** Only the user who is deleting their account, signed in, with a fresh step-up:
  - password plus their second factor, or a passkey;
  - an OAuth-only user does a forced fresh round trip (`prompt=login` , `max_age=0`), plus their second factor (required for an owner);

    - **a user with no second step**: the password, or the fresh round trip, plus a code sent to the account's email, with the same lifetime, attempt cap, send limit and session binding as the first-enrolment code (§4.1). No factor is set up. A departing user gains nothing from a factor they would use once, and a password thief without the mailbox can neither delete nor enrol (T-84, T-82). The confirmation authorises this deletion run only, including the own Person's export it produces, and no other step-up action.

  An enrolled user's deletion is unchanged. An unenrolled user can hold no manager or owner role outside their unshared own Person, so their run is that Person's purge (with its export), their viewer memberships ended, and any held raise cancelled. The plan, transfer-or-purge choices, exports and typed name are otherwise unchanged. Never a job, never another member, never the operator CLI. The user still holds owner when each purge runs, so the §4.5 sole-owner guard passes on its own terms, and no guard is relaxed for this flow.
- **The plan is bound to the session.** The per-entity choices (transfer to whom, purge with which export) are held server-side, keyed to the session that made them.
  - Another session, or a new sign-in, starts again from the list.
  - If the 5-minute step-up lapses mid-run, the user is prompted again. The run never continues silently.
- **Founder succession.** For each entity where the user is the founding owner and other owners remain, the flow requires a successor named among them (§4.2.2). A sole-owned entity's transfer target becomes founder.
- **Exports.** Every purge choice carries its own `export.completed` event, by this user, within 24 hours (guard 3). The flow produces these exports before the final confirmation.
- **Order.** Transfers first, then purges, then the tombstone.
  - A transfer names a current member who is not mid-deletion themselves. The recipient is notified, and the transfer writes an `audit_event`.
  - A failed provider revocation stops the run at that entity (§4.5 guard 6). Nothing later is applied.
- **Audit.** One `account.deleted` `audit_event` records every choice: the transfer targets, and each purged entity's id with its export SHA-256. It sits alongside the per-entity `entity.purged` events. All of them keep the tombstoned user id.
- **Email.** Every member of a purged entity is emailed as each purge begins (OQ-23). The departing user gets a confirmation at their address before the tombstone removes it. Every remaining member of a shared entity they leave is notified (ARCH §3.4a).
- **Tests** (C-52):
  - another member cannot drive a deletion's purge step;
  - a plan made in session A cannot be confirmed from session B;

- a lapsed step-up blocks the next purge;
- an unenrolled user's deletion needs the password and the emailed code, offers no factor set-up, and is refused with the password alone;
- the unenrolled deletion confirmation does not pass step-up for any other action;
- a failed revoke leaves the account undeleted and the remaining entities untouched;
- the tombstone transaction refuses if a sole-owned entity appeared during the flow.

5. Data Classification & Handling

User data falls into three sensitive tiers (from the Research brief), plus two non-tenant classes. Tiers 2 and 3 together are "sensitive personal financial data". Memo and payee text routinely leaks medical, income and relationship details, so it is not "just transactions".

Everything sits on an encrypted volume (C-72) and travels only over TLS. The table covers handling beyond that baseline. Assigning a tier to a new field is a security-gate question.

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
T1 Authentication secrets — Restricted	Password hashes, TOTP seeds, backup codes, session records, native/MCP bearer tokens, verification and reset tokens, IdP OAuth tokens	Passwords use scrypt. Our own bearer tokens and verification codes are hashed. BetterAuth's `verification` rows are keyed by a SHA-256 hash of their identifier (`verification.storeIdentifier: 'hashed'`), and the magic-link plugin hashes its token before that (`storeToken: 'hashed'`), so the table holds no usable magic-link or reset token (code-read, 1.7.18; tested). A value read from the table and presented as a token is refused. Backup codes and TOTP seeds are encrypted with XChaCha20-Poly1305 under the auth secret. Session tokens are stored in the clear (measured, 1.7.18; R-12). `encryptOAuthTokens: true` encrypts the access and refresh tokens; it is off by default (measured). It does not cover the ID token, which a `databaseHooks.account` before-hook drops (measured on create; update is set the same way). We request no offline OAuth scopes, because we need identity only.	Never logged. Never returned after creation. Never in a URL, except single-use magic links, which expire after 5 minutes (the plugin default), and single-use reset links, which expire after 15 minutes. Password reset is not enabled: no reset sender is configured (POLY-137).	Sessions: 7 days idle, 30 days absolute. Tokens: until used or expired. All deleted on account deletion.
T2 Standing bank	SimpleFIN access URLs, Plaid	HPKE-sealed, with AAD binding (§6.3, C-30). `poly_app` has no column SELECT (C-31).	Never sent to any client in any form, not even masked	Until unlink. Revoked at the

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
- access credentials — Restricted (highest value)	access tokens, broker refresh tokens		or partial. Never in DTOs, logs or the raw payload. Opened only in `worker`.	provider on unlink, account deletion or purge (C-32).
T3 Financial data and PII — Confidential	Balances, transactions, payees, memos, holdings, lots, entries, tax figures, entity names and classification, account last-4, raw payloads (redacted), exports, `audit_event`, user name and email	RLS on the membership join. Full account numbers are never stored (C-34). No field-level encryption: it would break sums, ranges and reports, for a threat that volume encryption plus RLS already covers.	Members only, per §4.2.1. Payloads are manager+ only. Logs carry opaque IDs only (C-33). Exports are streamed and never persisted (C-51). Emails are visible only to co-members who are owner or manager.	Ledger: indefinite (append-only) until entity purge. Raw rows follow the ledger. `audit_event`: indefinite. On account deletion, the user row becomes a tombstone (C-52).

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
Operator secrets	Credential private keys, DB passwords, Plaid secret, Resend key, BLS key, OAuth client secrets, BetterAuth secret, backup key, `CRON_SECRET`	Files under `/run/secrets`, mode 0400. Never in the repo, an image or the database (§6.2).	Never logged. Prod and staging values differ. The backup key and the credential key are never stored together.	§6.4
T4 Public reference	Prices, CPI-U, tax tables, system report templates	—	No tenant column.	Indefinite

No outbound user data (a control). ARCH §1 says nothing about a user's books leaves the box, except through a provider call the user initiated or an owner-initiated export. Treat that sentence as a control. Any feature that would break it needs a security review and a PRD-level decision, not just a library import. Examples: crash reporting with payloads, hosted error tracking, telemetry that includes tenant data, or an LLM call that carries transaction text.

Stored, but never logged. T3 is stored in the clear under RLS, but scrubbed from logs. Logs get tailed over SSH, pasted into bug reports and shipped to aggregators, which makes them the likeliest escape route. That is why scrubbing is enforced by a CI canary (C-33) and not left to convention.

6. Secret Management

Two populations. Operator secrets: one set per instance, held by the person running the box. User credentials: one per connection, held sealed in the database. §6.3 resolves ARCH OQ-1.

6.1 Inventory

Secret	Lives in	Mounted into	Rotation
Credential keyring (X25519 private keys, by `key_id`)	`/etc/polycarpic/secrets/credential_keyring`, plus an offline copy in the operator's password manager	`worker` only. Public keys go to `web` as non-secret config.	§6.4
Postgres role passwords (`poly_app`, `poly_worker`, `poly_migrator`)	Host secrets directory	Only the container that uses that role	On compromise or a change of operator
`db_password_backup` (`poly`	Host secrets directory (Hetzner: root disk; home box: the encrypted partition)	The `backup` container only, which has no internet route. Never `web`, `worker`, `migrate` or the uploader.	On compromise or a change of operator. This credential reads everything, so a suspected leak is RB-2, not a routine rotation.

Secret	Lives in	Mounted into	Rotation
`_backup`, C-79)			
`db_password_operator` (`poly_operator`, C-84)	Host secrets directory, `operator/` (staging: `staging-secrets/operator/`)	The one-shot `operator` service only. It starts only on `docker compose --profile operator run --rm operator …`, never restarts, and logs nothing (logging driver `none`). No long-running service mounts it: not `web`, `worker` or `migrate`, prod or staging. The compose lint's secret matrix and its `operator-one-shot` rule refuse it anywhere else.	On compromise or a change of operator. Its holder can issue no-entity `viewer` invitations (C-01) and nothing else.
Better Auth secret	Host secrets directory	`web` alone. `AUTH_SECRET_FILE` is registered for `web` only (`packages/config/src/registry.ts`), so the config loader refuses it when set in any other service, and the compose lint's secret matrix refuses an `auth_secret` mount on any other service.	On compromise. Rotating it invalidates every session, which is the intended effect in an incident. It also encrypts TOTP seeds, backup codes and OAuth tokens (measured, 1.7.18). So rotate through BetterAuth's versioned `secrets` option: the new version encrypts, and old versions still decrypt until each row is rewritten (code-read). Replacing the secret outright would lock out every enrolled user.
OAuth IdP	Host secrets directory	`web`	At the provider, on suspicion

Secret	Lives in	Mounted into	Rotation
client secrets, Resend key			
Plaid client secret, broker OAuth clients, price-provider key, BLS key	Host secrets directory	`worker`, plus `web` for the Plaid token exchange	At the provider, on suspicion
Backup encryption key (age)	The public half (recipient) at `/etc/polycarpic/age-recipients.txt`. The private half only in the operator's password manager and on the printed recovery sheet, on a different sheet from the keyring. Never on any box, never in a backup (§6.6).	Host backup job (public half only)	On a change of operator. Keep the old private key until its last backup expires (35 days).
Backup store keys	**Put-only** key `backup_store_credential`: host secrets directory, and the operator's laptop for the	Host backup job only; no container	At B2, on suspicion or a change of operator. Neither key can delete or shorten retention (§6.6).

Secret	Lives in	Mounted into	Rotation
(B2, split)	weekly Neon export. **Read** key: password manager, used only on the drill machine, never on a box. The B2 master key: password manager only, never used for backups.		
`github_read_token`	Host secrets directory. Fine-grained, this repository only, `contents:read` and `packages:read`.	Host deploy script only; no container	Expires after 90 days; a monitor check fires 14 days ahead (INFRA §8.4). Deleted if the repository and package become public.
`heartbeat_url`	Host secrets directory. Carries the Healthchecks.io ping key: whoever holds it can fake a heartbeat and mask an outage.	Host scripts only; no container	Regenerate the ping key on suspicion.
`staging_basic_auth`	Staging secrets directory, as a bcrypt hash; the password itself only in the password manager	Caddy (staging route)	On suspicion
Disk keys	Hetzner: the data volume's LUKS key at `/etc/polycarpic/luks.key` (0400) on the root disk; it protects a detached volume only (C-72). Home box: the LUKS recovery passphrase, in the password manager and	None	On a change of operator, or after RB-2

Secret	Lives in	Mounted into	Rotation
	on the recovery sheet, never on the box; the TPM holds the other slot.		
Neon export role (Vercel target only)	The operator's password manager; used from the laptop only. Read-only, and it bypasses RLS, because a dump must see every row: `poly_backup`'s profile without REPLICATION (C-79). Whether Neon lets a role hold BYPASSRLS is unmeasured (OQ-INFRA-6). GitHub never holds it.	None (laptop `neon-export`)	On suspicion, and if the target is retired
Operator console accounts and the Hetzner API token	Password manager, with two-step sign-in on each account. The Hetzner token is deleted after provisioning (C-78).	None	Per C-78
Staging credential keyring	The staging secrets directory, separate from prod's	Staging `worker` only	Freely: losing it means relinking sandbox items

Secret	Lives in	Mounted into	Rotation
(sandbox tokens only)			
DNS-API token (local box, real-domain mode)	Host secrets directory	Caddy only	At the DNS provider, on suspicion. Scoped to one zone.
`CRON_SECRET` (Vercel only)	Vercel env var	Internal cron route	On compromise
CI credentials	None that reaches a box: the deploy is pull-based (C-83). Actions uses its per-job `GITHUB_TOKEN`, widened to package and attestation writes only in the build and release jobs. The Vercel names check uses a read-scoped Vercel token in a GitHub environment with required reviewers; it reads names, never values (INFRA §4.7).	Actions runner	On compromise
Aggregator	Postgres, sealed (§6.3)	Opened only in `worker`	Driven by the provider. Re-sealed on key rotation.

Secret	Lives in	Mounted into	Rotation
tokens and URLs (T2)			

6.2 Operator secret handling

- **Files, not environment variables.**
  - Secrets live on the host in one directory per service: `/etc/polycarpic/secrets/<service>/<name>` for prod and `/etc/polycarpic/staging-secrets/<service>/<name>` for staging. Each directory is `root:<service gid>` 0750 and each file `root:<service gid>` 0440, never world-readable. Every service runs as its own numeric uid and gid, and none uses `group_add`, so a file has exactly one reader. A secret that two services use (for example `auth_secret` in web and worker) is two files, one in each service's directory. Plain `docker compose` (without swarm) bind-mounts each file with its host owner and mode, so the access check is the host file mode, not the compose file (unmeasured; the `image` job's secret-mode test measures it). Running a container as root or making a file 0444 are both refused.
  - They are mounted as compose `secrets:` at `/run/secrets/`, and apps read them through the `*_FILE` convention.
  - Why: environment variables leak through `docker inspect`, `/proc/<pid>/environ`, child processes and error reporters that dump `process.env`.
- **Per-container scoping.** Each container gets only the secrets it needs. `web` never gets the credential private key or the backup key. Staging gets its own values, and its own keyring, which seals only Plaid Sandbox and SimpleFIN demo tokens. It never gets prod's keyring or the production Plaid secret (C-77).
- **Nothing in images.** No secret is ever baked into an image or passed as a build arg. `.env.example` lists names with a comment each, never values.
- **Fail closed at startup.** Outside development, the app refuses to start if a required secret is missing, or if a secret-looking env var is set in production. The secret-looking refusal applies in staging as well, so what passes staging passes production. In every `APP_ENV`, development included, the loader has one source, the file its `*_FILE` variable names. Setting the plain variable for a registered secret (for example `AUTH_SECRET`, or BetterAuth's own

`BETTER_AUTH_SECRET`) refuses start, so no environment fallback can creep in. Every refusal names the variable and never the value or the file's content.

- **Read once.** Secrets are read at startup. There is no runtime secret-fetch path and no admin endpoint that reveals configuration.
- **Vercel.** The tertiary target uses Vercel's encrypted env vars (R-3). The hosting order is Hetzner, then the local box, then Vercel + Neon.
- **Local box.** Same file layout as the Hetzner path. The secrets directory lives on the TPM-unlocked encrypted partition, so a pulled disk yields no secret (INFRA D-1). In real-domain mode the DNS-API token is a file secret mounted into Caddy only, scoped to one zone (C-76).
- **Host-only secrets.** `backup_store_credential`, `github_read_token` and `heartbeat_url` are read by root's host scripts and are never mounted into a container, so a container escape short of root does not reach them.
- **Hetzner.** The secrets directory is on the root disk. Encrypting it there would put the key on the same disk, so it would add nothing; a captured server is RB-2.

6.3 Credential sealing — ruling on OQ-1

App-layer *asymmetric* sealing in `packages/secrets`. `web` holds only public keys; the private keyring is mounted into `worker` only.

The seam is `seal()` / `open()` with a `key_id`. pgcrypto and an external KMS are rejected (§6.5).

- **Why asymmetric.**
  - With a symmetric master key, whoever seals can also open. The sealing process is `web`, because it receives the SimpleFIN setup token or the Plaid public token.
  - `web` is also the internet-facing process, and the most likely place for RCE or SSRF. So one `web` compromise would yield every bank token (T-31).
  - Asymmetric sealing removes that path at about the same implementation cost.
- **Algorithm.** HPKE (RFC 9180) base mode: DHKEM(X25519, HKDF-SHA256), HKDF-SHA256, AES-256-GCM. The library (e.g. `@hpke/core`, or an equivalent over `node:crypto`) is pinned in the `packages/secrets` design file, with SecEng sign-off. No home-rolled construction.
- **Binding.** The HPKE `aad` is the canonical encoding of (`connection_id`, `entity_id`, `provider`, `key_id`, `alg_version`). A blob copied to another row fails to open (T-32).
- **Stored columns.** `enc` (the encapsulated key), `ciphertext`, `key_id`, `alg_version`. These replace ARCH §3.1's "ciphertext + nonce + wrapped DEK + key_id".
- **Flows.**

  - *Link* (in `web`): receive the setup or public token, exchange it with the provider, `seal()` the result under the active public key, insert the row, drop the plaintext.
  - *Poll* (in `worker`): `open()` inside one call frame, for one provider call. The plaintext is never cached and never held in a module-level variable. `packages/import` does not import `open()`: `worker` builds an opener for the one connection the job is for and passes it into `ingest` (ARCH §2.2). `import` holds it only for that call and never stores it.
  - *Unlink*: `web` enqueues a revoke job. `worker` opens the credential, calls `adapter.revoke()`, then deletes (C-32).
- **Database enforcement.** `poly_app` has column INSERT and UPDATE on the sealed columns, but no SELECT. `poly_worker` has SELECT. The columns never appear in DTOs or `openapi.json` (C-31).
- **Errors.** Provider errors are re-thrown with the credential stripped. A raw provider exception never reaches a response or a log without passing through the scrubber.
- **Import boundary** (ARCH §2.1 layer map, `layers.json`; built by POLY-82, run in CI by POLY-96).
  - Both import forms are refused across a package edge. An import by package name is checked against the layer map's allow-list. An import by path is refused wherever it reaches another package's folder, even where the map allows the edge, because a path import skips the target's `exports` map. Lint covers every import form: `import`, re-export, `import()`, `require()`, `import x = require()`, type imports, `import.meta.glob`, `new URL(…, import.meta.url)` and `/// <reference path>`. It runs over every source file type under `apps/*` and `packages/*`, Svelte included. Lint rule: `poly/no-relative-cross-package`, generated from `layers.json`.
  - A non-test source file never leaves its own package by path, and never imports test code. Test files may reach out only to the repo-root `tests/`, which is excluded from the image build context.
  - Nothing in a package builds a require function. `module` / `node:module` are not imported in any form, and `createRequire` and `getBuiltinModule` are refused as names (`poly/no-require-factory`). No `.cjs` or `.cts` file sits under a `src` folder. Residual: a key computed through an aliased global object is not caught.
  - No package-level indirection. Structure tests refuse the following: a `package.json` `imports` field; a `link:`, `file:`, `npm:` or aliased `workspace:` dependency or override; a workspace package declared under any name but its own `@poly/<name>`; a pnpmfile; a tracked symlink; and a path alias in a tsconfig or the Vite/Svelte config.
  - **Sign-up confinement** (`layers.json`: `confined`, `reservedLiterals`, `confinedMembers`). `packages/db/src/new-user.ts` is imported only by `context.ts`,

in every import form and inside the package too (`poly/confined-module`). The definer names are string literals in exactly one file each, matched on the decoded value so an escape does not hide them: `enter_new_user` in `new-user.ts`, `claim_invitations` and `accept_invitations` in `packages/tenancy/src/invitation.ts` (`poly/reserved-literal`). `enterNewUser` as a property name, called, read, destructured or as a computed literal key, appears only in `packages/auth/src/user-hooks.ts` and `packages/db/src/context.ts` (`poly/confined-member`). `no-eval`, `no-implied-eval` and `no-new-func` are on for `apps/*` and `packages/*`, with the runtime globals declared. Residual: a name built at run time or reached through a computed global is not caught; the database refusals in C-84 decide.

- Only `packages/secrets` imports the HPKE library.
- `open()` is the `secrets/open` subpath. Only `apps/worker` may import it. `web` and `import` may import only the index, which seals.
- The index never imports the open module internally. Lint cannot see an import inside a package, so a check on the index's module graph holds this (A-39). The bundle scan of the `web` build (ARCH §7.6) is the second check.
- An inline `eslint-disable` of these rules fails lint (A-39). Resolution does not cover the subpath, because `web` declares `secrets` as a dependency, so lint is the only source-side guard.
- **This is a second line, not the boundary.** TB-4 is held by the keyring: it is mounted into `worker` alone, and `web` connects as `poly_app`, which cannot read the sealed columns (C-31) or run `app.job_targets`. On the box, one image carries both entrypoints, so the open code is on `web`'s disk anyway. Without the key, it opens nothing.

6.4 Key rotation for an operator with no KMS

1. Run `polycarpic secrets rotate` in `worker`. It generates key *n+1*, adds it to the keyring, and publishes its public key as the active sealing key.
2. It re-seals every row with `key_id = n`: open with *n*, seal with *n+1*. The step is idempotent and resumable.
3. It asserts that zero rows remain under key *n*, removes *n*, and writes an `audit_event`.
4. When to rotate: on any suspicion that the worker, the box, the operator's SSH key, or a backup together with the keyring was compromised, and whenever the operator changes. Annual rotation is optional hygiene. If the keyring *and* the database may both be exposed, rotation alone is not enough: revoke every token at the provider and relink (RB-1).

A lost keyring means relinking, not data loss. Every connection shows "relink required". The ledger is not encrypted under this key. The runbook says this plainly, so that nobody stores the key next to the backup "to be safe". The keyring is never part of the backup set.

6.5 Why not pgcrypto, and why not a secrets manager

- **pgcrypto (rejected).** The key and the plaintext would pass through SQL. That puts them in `pg_stat_activity`, potentially in statement logs, and in the same process as the data. A stolen dump must yield inert ciphertext. With the key co-located, it does not.
- **Vault or a cloud KMS (rejected for v1).** On a single VPS, a vault's unseal material sits on the same box. So it adds a service for a novice operator to run without adding real separation. It would also break the near-zero-spend guardrail and add an availability dependency. The `key_id` seam leaves room for a KMS-backed keyring later; revisit if R-6 ever triggers.
- **The credential-free path stays the recommended default.** The CSV/OFX adapter stores no credential. An instance that never links an aggregator holds zero T2 data. The operator runbook presents linking as a convenience trade-off.

6.6 Backup key custody — owned by DevOps (ARCH OQ-9)

Requirements on INFRA (all met by INFRA §6 and D-3; built as below):

- Backups are encrypted on the box before they leave it.
- The key can be recovered without the VPS.
- The key is distinct from the credential keyring, and the two are never stored together.

Recommended shape:

- An asymmetric recipient (age-style), so the box holds only the public half.
- A write-only or append-only store credential, or immutable snapshots.

With that shape, a fully compromised box can neither read nor delete its own historical backups. The acceptance test is a restore onto a clean machine. RPO and RTO numbers are DevOps's call (INFRA §6.4: 15 minutes and 4 hours on Hetzner).

As built:

- **Store.** A Backblaze B2 bucket with Object Lock in *compliance* mode and a 35-day default retention. In compliance mode no key, the master key included, can delete a version or shorten its lock before it expires (as B2 documents it; measured at the first drill, OQ-INFRA-4). Governance mode is refused, because a key holding the bypass permission can lift it. A

lifecycle rule hides objects after 36 days and removes them a day later, so no deletion credential exists anywhere.

- **Box key: put-only.** Write on this bucket only, with none of `deleteFiles`, `bypassGovernance`, `writeFileRetentions` or `writeBucketRetentions`. An upload under an existing name makes a new version, and the locked version stays.
- **Read key: off the box.** List and read only, held by the operator for the drill.
- **Residual: cost, not loss.** A compromised box can fill the bucket, and locked junk is billed for 35 days. A B2 storage cap with an alert bounds it (INFRA §6.1).
- **Residual: the B2 login.** Whoever takes over the B2 account can read nothing (the objects are age-encrypted) and delete nothing locked. What closing the account or a lapsed payment does to locked objects is unmeasured. Two-step sign-in is required (C-78).
- **Vercel target (OQ-INFRA-1).** Backed up by Neon's restore window plus a weekly export from the operator's laptop: a read-only role that bypasses RLS, encrypted to the age recipient, uploaded with the put-only key. GitHub holds no database credential. This is re-ruled when real data first lands on the Vercel target. A GitHub-held export would make GitHub hold a credential that reads every ledger (TB-12); SecEng would then require at least that role and nothing more, in a GitHub environment with required reviewers and a branch restriction, the job pinned by SHA with no third-party action near the secret, and encryption to the age recipient inside the job.
- **Fallbacks.** A Hetzner Storage Box is allowed on the Hetzner target only after its cannot-delete test passes. It cannot serve the home box, whose outbound allowlist has no SFTP port (C-76).

7. Controls Catalog

Each control maps to threats in §3. No application code exists yet, so Status names the milestone where the control must land. A feature that touches a control cannot pass the Validate security gate without it.

ID	Control, location and test	Threats	Status
C-01	Invite-only registration, the first account included: there is no first-sign-up path, and the operator bootstraps with `polycarpic user invite <email> --bootstrap`, which refuses on a non-empty instance. OAuth sign-in for an unknown email is rejected unless an invite is pending. There is no reopen switch. The operator CLI `polycarpic user invite <email>` issues an audited invitation that shares no entity. It runs in the one-shot `operator` service and connects as `poly_operator` (C-84). • **The operator's invitation.** ◦ It shares no entity, has no inviter, carries role `viewer`, and expires 7 days after issue. ◦ `invitation.issued_via` is `'operator'` or `'bootstrap'`. A check constraint holds that shape for every operator row. It runs one way only: a row with no entity need not be an operator row. ◦ `poly_app` cannot write `issued_via`, so an owner's invitation always reads `'owner'`. ◦ Only `app.issue_operator_invitation` writes an operator row (C-15). No caller parameter sets the expiry, the role or the entity. ◦ The CLI checks the address, and the definer checks it again: at most 254 characters, no control or space characters, and exactly one `@` with something on both sides. It is stored lower-cased. • **Instance state.** ◦ The plain invite refuses on an empty instance, and `--bootstrap` refuses once any user exists. Both exit 2 and write nothing. ◦ Issuing is serialised on one advisory lock, bounded by a 2-second `lock_timeout`.	T-01, T-95, T-97, T-98, T-99, T-100	POLY-M1

ID	Control, location and test	Threats	Status

    - A second `--bootstrap` revokes the unclaimed earlier one. A partial unique index allows at most one live bootstrap row.
    - `claim_invitations` and `accept_invitations` refuse a bootstrap invitation once any other user exists.
- **Delivery.**
    - The CLI draws 32 random bytes and sends only their hash to the database.
    - It writes the link, `<PUBLIC_ORIGIN>/sign-up?invitation=<token>`, once to stdout, only after the commit has returned. The origin comes from the validated `PUBLIC_ORIGIN` alone, never from a flag.
    - The token, the link, the hash and the address never reach stderr, the logger or telemetry.
    - The command runs as the one-shot `operator` container's main process (`docker compose --profile operator run --rm operator user invite <email>`). That service's logging driver is `none`, so the link reaches the attached terminal and no container log. The compose lint refuses the operator password on a service without that driver (INFRA §7.5). Measured with a stand-in line on the real service definition, not with a real invite link (`process/cairn/reviews/POLY-146/logging-measure.md`).
- **Audit.**
    - Each issue writes an `invitation.issued` `audit_event`, and each bootstrap revocation an `invitation.revoked` one, in the same transaction.
    - The actor is the literal `operator:cli`, with no actor user and no entity.
    - The payload is the invitation id, plus `issued_via` on issue. It never holds the token, the hash or the

ID	Control, location and test	Threats	Status

address.

Lives at: `packages/auth` hooks; `packages/tenancy` (`operator-invitation.ts`); worker CLI (`apps/worker/src/cli`); `packages/db` SQL
Tested by: Integration:

- a 2nd signup is rejected;
- OAuth for an unknown email is rejected;
- an invite accept works;
- a CLI invite yields a user with only their own Person, and writes an `audit_event` ;
- a config scan finds no self-signup flag;
- On an empty instance, sign-up with no invitation and OAuth sign-in for any email are refused;
- `invite --bootstrap` on an instance with one user exits 2 and writes nothing, the audit included, and the plain invite on an empty instance does the same;
- a second `--bootstrap` revokes the first, whose link is then refused;
- two concurrent `--bootstrap` runs both succeed and leave exactly one live bootstrap row;
- a `poly_app` session holding the advisory key makes `--bootstrap` exit 1 within 3 seconds and write nothing;
- with a user present, a live bootstrap invitation is refused on every user-creating route, token-less ones included, and leaves no user row;
- stdout is exactly one line, the link on `PUBLIC_ORIGIN` whatever `BETTER_AUTH_URL` or `ORIGIN` say, and the link's hash is the row's;
- across every run, the token appears in no stderr or log line, no `invitation` or `audit_event` column, and no query text or parameter;
- compose lint, rendered prod and staging: a service other than `operator` mounting the operator password, and an `operator` service that restarts,

ID	Control, location and test	Threats	Status
	lacks the `operator` profile, or logs with any driver but `none`, each fail; • a failure at commit prints no link and exits 1; • `expires_at − created_at` is exactly 7 days; • each bad address and each bad argument exits 1 with nothing on stdout; • the bootstrap invitation is single-use, accepted only for its email, and refused after expiry; • every refusal on `/sign-up/email` (no, malformed, unknown, expired, revoked, claimed, other-email token; a lock timeout) is a byte-identical 403 and leaves no `user` row; two sign-ups racing on one token give one user; • a token-based sign-up claims only its own invitation while another for the same email stays live; a manager or owner invitation is claimed and not seated; a claimant selects no `invitation` row; • the plaintext token appears in no log line; `/sign-up` sends `no-referrer` and `no-store`; • pgTAP: `poly_app` cannot select `token_hash`; a 31-byte hash fails the check; the four set-once columns refuse a change; exactly one function in schema `app` inserts into `invitation` (`issue_operator_invitation`), and none into `user`; an operator row with an entity, an inviter or a role other than `viewer` fails the check; a `poly_app` insert that supplies `issued_via` is refused.		
C-02	Session cookie is `__Host-`, `HttpOnly`, `Secure`, `SameSite=Lax`. 7 days idle, 30 days absolute. All sessions revoked on a password or MFA change. **Lives at:** `packages/auth` config **Tested by:** Playwright `Set-Cookie` assertions; config unit test.	T-04	POLY-M1
C-03	MFA (TOTP, passkeys) per §4.1: required for owner and manager, except the owner row on a user's own unshared	T-02, T-08, T-81,	POLY-M1 (MFA,

ID	Control, location and test	Threats	Status
	Person (OQ-34); optional for viewers. Step-up re-auth within 5 minutes on the listed actions, always with the second factor. Operator MFA-reset CLI, audited. A role raised to manager or owner is held in `membership.pending_role`, with `role` unchanged, until the member enrols; only the `poly_govern` promotion makes it effective (§4.1; ARCH §3.1c). The first factor is enrolled only after the password and a code sent to the account's verified email are confirmed. The code is single-use, lasts 10 minutes, is voided after 5 wrong entries, is sent at most 3 times per 15 minutes, and is bound to the session; BetterAuth's enrolment endpoints are wrapped by a `hooks.before`, and its `otp` method is off (§4.1). BetterAuth's writes run inside the request's `withTenant` transaction through the routing pool, on its Kysely path, never the Drizzle adapter (A-31; §4.1), so the database guard checks the grant by user and session. `session.freshAge` is 30 days, and the before-hook is an allowlist over `/api/auth/*`: open, enrolment, step-up or blocked, and 404 for anything unlisted (ruling (3) above, A-34). A completed first enrolment sets the step-up stamp on the session that BetterAuth issues to replace the old one (A-32). A backup code counts as step-up only once `second_step_at` is set. **Lives at:** `packages/auth`; the `requireRecentAuth()` guard; worker CLI; `packages/tenancy`; the ARCH §3.1c triggers **Tested by:** Integration: every step-up action returns 403 without a recent re-auth; the CLI writes an `audit_event`. pgTAP: a viewer raised to owner without a second step is refused a manager write and an owner write, is not counted by `app.owner_count`, and is not a required approver; after enrolment all four flip and an `audit_event` is written. pgTAP: `poly_app` cannot set `second_step_at` or promote a held row; at 1 → 2 a direct-path held raise is cancelled and an approved one kept; removing the last method is refused while manager and allowed while viewer. OQ-34, integration: an unenrolled user whose only membership is their own unshared Person can sign in and view; their first gated action (creating a Trust, the first invitation on the own Person, linking it under a parent, an export, a bank link) redirects to Set up a second step and is not	T-82, T-83, T-86	required for owner and manager), then with each gated feature

ID	Control, location and test	Threats	Status
	performed, with no row written; each succeeds after enrolment. Email anchor, integration: a session with only the password cannot complete a first enrolment (no factor row, `second_step_at` null) through the UI or by calling `/two-factor/enable`, `/two-factor/verify-totp`, `/passkey/generate-register-options` or `/passkey/verify-registration` directly; a backup code from an unverified enrolment does not pass step-up; a first TOTP verify that the guard refuses leaves `twoFactorEnabled` false and the old session in place (one transaction); an unenrolled user's gated action (a Trust, an export) is not performed until both the password and the emailed code are confirmed and the factor is set; a code from session A is refused in session B; a used, expired or 5-times-wrong code is refused; the 4th send in 15 minutes is suppressed with an identical response; the code goes to the current address during a pending email change; a user who already has a factor adds a second without any email code. Step-up: after a completed first enrolment the gated action proceeds with no second Confirm it's you, and a 403 follows after 5 minutes. pgTAP (A-29): `poly_app` cannot insert a passkey, set `twoFactorEnabled`, or mark TOTP verified for a user with no factor and no live grant for the session in `app.session_id`. Integration (A-31): the routing pool is tested against the pinned BetterAuth version, built from the app's exported configuration. A write outside `withTenant` fails closed, and a rollback removes BetterAuth's rows; a lint rule refuses `drizzleAdapter` in `packages/auth`. pgTAP (`SET NULL`): a grant consumed and then orphaned by the session delete survives with `session_id` null, and the verified UPDATE in the same transaction passes; a grant whose session is deleted before use is refused at the flip and at the passkey INSERT for a new session of the same user; a grant consumed in an earlier transaction does not let a later one mark TOTP verified; `consumed_txid` matches when the consuming write ran inside a savepoint. Integration (allowlist): every endpoint of the configured instance is classified, or the test fails; each step-up path answers 403 without a step-up in the last 5 minutes, whatever the session's age; each blocked path answers 404; an enrolled user's session with no recent step-up cannot register a passkey, and a 29-day-old session with one can; an OAuth-only user's session cannot read the TOTP URI, regenerate backup codes or disable TOTP; the app's session list returns no token.		

ID	Control, location and test	Threats	Status
C-04	BetterAuth `rateLimit` in memory storage, never on the database: a rolled-back request would erase its own counter. Sign-in, sign-up, magic link, password reset and second-step verification are each limited to 5 a minute per client IP; a per-email limit is our own counter, because BetterAuth keys only on IP and path. **The client IP** is the one `X-Forwarded-For` entry that Caddy writes, overwriting any value the client sent (`header_up X-Forwarded-For {remote_host}` , no `trusted_proxies`). The app reads that header only, sets no `trustedProxies` , and accepts exactly one valid IP; IPv6 is bucketed by /64. Outside development, an `/api/auth/*` request whose IP does not resolve answers 503 and is never put in a shared bucket. A CDN or tunnel in front of Caddy changes this trust and needs a new review. Generic auth error messages. Body-size limits on every route. **Lives at:** `packages/auth` (`options.ts` , the auth handler); `hooks.server.ts` ; the Caddyfile **Tested by:** Integration: the 6th attempt from one IP in a minute returns 429 while another IP still gets 401; a missing, two-entry or invalid `X-Forwarded-For` returns 503 and writes nothing; two addresses in one IPv6 /64 share a bucket; responses for known and unknown emails are identical. Edge: a forged `X-Forwarded-For` reaches `web` as one entry equal to the peer.	T-02, T-07, T-09	POLY-M1
C-05	OAuth uses PKCE and `state` . Accounts link only when the IdP asserts a verified email *and* there is either an existing session or a matching pending invite. Web v1's providers are Google and GitHub. Sign in with Apple arrives with the native iOS milestone (POLY-122, unscheduled). Each provider asserts verification differently: • **Google:** the ID token's `email_verified` is true. • **GitHub:** the profile carries no verification claim. The email is taken only from the `/user/emails` entry that is both `primary` and `verified` , through our own `getUserInfo` for the provider. BetterAuth's default,	T-03, T-96	POLY-M1

ID	Control, location and test	Threats	Status
	which takes the public profile email, is not used. With no such entry the sign-in is refused. The requested scopes are `read:user` and `user:email`, nothing more. • **A new account on these paths** claims invitations by its email only when that email is verified, and the database checks `user.email_verified` (C-01). An account created by password sign-up stays unverified, and BetterAuth's `requireLocalEmailVerified` default (true) keeps an IdP sign-in for the same address from linking to it. • **Apple, when POLY-122 lands (a condition on that issue, not a live control):** the ID token's `email_verified` is true, whether sent as a boolean or the string `"true"`. A Hide My Email relay address is a distinct email that matches no invitation, so it links only through an existing session. Before POLY-122 ships, it settles how an invitee who hides their email accepts an invitation, and adds Apple to this control's tests. **Lives at:** `packages/auth` **Tested by:** Integration: a mocked IdP sending `email_verified=false` does not link, for each of the two; a GitHub account whose matching email is unverified, or verified but not primary, does not link; a Google sign-up with `email_verified=false` and a GitHub sign-up whose invited address is unverified or not primary claim nothing and leave no user; a GitHub sign-up whose primary verified address is invited claims it; after a password sign-up, a Google callback for the same address does not link and adds no `account` row; the GitHub authorize request asks for no scope beyond the two; no Apple provider is configured.		
C-06	Kit's origin check on form submissions stays on (its default), never disabled. Mutating `/api/v1` routes accept JSON only. SameSite cookies. **Lives at:** `apps/web/vite.config.ts` (the `sveltekit({…})` options; Kit 3 has no `svelte.config.js`); `/api/v1` wrapper	T-05	POLY-M1

ID	Control, location and test	Threats	Status
	Tested by: Integration: a cross-origin form POST is rejected; `text/plain` returns 415.		
C-07	CSP through Kit's `csp` option, `mode: 'auto'` (nonce or hash): `default-src 'self'`; `script-src 'self'`, never `unsafe-inline` or `unsafe-eval`; `style-src 'self' 'unsafe-inline'` (component positioning uses style attributes); `img-src 'self' data:`; `font-src` and `connect-src 'self'`; `object-src 'none'`; `base-uri` and `form-action 'self'`; `frame-ancestors 'none'`. A widening of any directive needs SecEng review. HSTS for 1 year at Caddy. `nosniff`. `Referrer-Policy: strict-origin-when-cross-origin`. • **Same-origin script.** `script-src 'self'` makes any same-origin response executable as script. The one static script is `static/theme.js`: it reads `prefers-color-scheme` and the `<html>` class, and sets only the `dark` class. `nosniff`, and the rule that no route serves user-controlled bytes as JavaScript or HTML, keep an upload from becoming script. • **The theme cookie.** `poly-theme` (`light` | `dark` | `auto`; `Path=/`, `SameSite=Lax`, one year) is written by client script, so it is not `HttpOnly`, and it holds nothing secret. The server maps it through an allowlist to the `<html>` class and never echoes its value. A forged value changes only the theme. The sidebar's `sidebar:state` cookie works the same way: exactly `false` gives `data-sidebar="collapsed"`, anything else `expanded`, and only those two literals reach the page. • Icons are static data (tag and attributes) rendered as SVG elements, never markup strings. Icon data never comes from user input or the network. • Avatar and other images stay under `img-src 'self' data:`. A provider avatar is served same-origin, and `img-src` is never widened to remote hosts.	T-04, T-06	POLY-M1

ID	Control, location and test	Threats	Status
	Lives at: `apps/web/vite.config.ts` (Kit 3 has no `svelte.config.js`); `apps/web/src/hooks.server.ts` (theme cookie); Caddyfile **Tested by:** Header assertions in the Playwright smoke run.		
C-10	Role separation and hygiene per §4.3(k). The hygiene set is every NOLOGIN definer-owning role: `poly_authz`, `poly_purge`, `poly_rollup`, `poly_graph`, `poly_stake`, `poly_govern` and `poly_journal`. Each is NOBYPASSRLS, owns no tables, and is a member of no role. Each is granted to no role but `poly_migrator`, and that grant is `WITH INHERIT FALSE, SET TRUE` and no ADMIN option (C-85), so `poly_migrator` reaches them only by an explicit `SET ROLE` inside a migration. No TRUNCATE, REFERENCES or TRIGGER grants to login roles. No CREATE on `public`. The only roles with BYPASSRLS or REPLICATION are the bootstrap superuser and `poly_backup` (C-79). *Where the roles are defined.* Roles are cluster objects, and only a superuser can grant `REPLICATION` and `BYPASSRLS`, so they are not created by the Drizzle chain. `packages/db/sql/roles.sql` is their definition of record, reviewed like a migration. A superuser runs it before every `migrate`: `provision.sh` on the box, the harness in CI. It holds no password. It converges on every run: it creates any missing role, then `ALTER`s every role to its exact attributes and re-issues every grant. Before taking the lock, the runner refuses with exit 2, applying nothing, unless all twelve roles exist and `poly_migrator`'s memberships are exactly `poly_owner` plus the seven roles above, with those options. *Condition (seceng, POLY-83):* the same preflight also refuses when `poly_owner` or any of the seven has a member other than `poly_migrator`, or when `poly_backup` is a member of anything but `pg_read_all_data`. `roles.sql` only adds grants and never revokes a stray one, so this check is the only thing that catches drift on the live box. pgTAP runs only in CI.	T-13, T-21, T-89	POLY-M1

ID	Control, location and test	Threats	Status
	Lives at: `packages/db/sql/roles.sql` (superuser, outside the chain); the runner's role preflight in `packages/db` **Tested by:** pgTAP catalog assertions, including a query over `pg_roles` that fails if any other role has `rolbypassrls` or `rolreplication`.		
C-11	`withTenant()`: `set_config('app.user_id', …, true)` as the first statement; branded `SessionUserId`, constructed only through `db/session` (importable by `packages/auth` alone; no cast to the brand elsewhere); nesting guard; raw pool not exported and the Postgres driver imported only by `packages/db`; session-level `SET` banned by lint; pooler-mode assertion in the deploy checklist. **Lives at:** `packages/db`; `layers.json` and the root ESLint config (ARCH §2.1; POLY-82); INFRA checklist **Tested by:** pgTAP pool-reuse test (tenant A, then tenant B on the same backend, sees 0 of A's rows); Vitest: a nested call throws. POLY-82 lint fixtures: the driver imported outside `packages/db`, `db/session` imported outside `packages/auth`, and a deep import past the index all fail.	T-11	POLY-M1
C-12	Fail-closed context: `app.current_user_id() = nullif(current_setting('app.user_id', true), '')::uuid`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: on a backend that already carried a context, a query with no context returns 0 rows on every tenant table and raises no error.	T-11	POLY-M1
C-13	ENABLE + FORCE RLS and at least one policy on every table with a tenant column. The probe is catalog-derived, with an inversion canary. The global-table allowlist is a security-gate decision. The probe covers every relation outside the schemas `pg_catalog`, `information_schema` and `pg_toast`. Temporary relations (`relpersistence = 't'`) are out of scope. They are private to one session and never shared across tenants, and no role holds TEMP on the database (pgTAP asserts this). That assertion is what makes the exclusion	T-10, T-12	POLY-M1

ID	Control, location and test	Threats	Status
	safe. Canaries are permanent tables. Unlogged tables stay in scope. Tables, partitions and partitioned parents (`pg_class` kinds `r` and `p`) are checked on `relrowsecurity` and `relforcerowsecurity`; partitions count because a policy on the parent does not cover a direct query on a child. A materialized view or foreign table (kinds `m` and `f`) fails the probe unless it is on the exemption list, since RLS never covers either (T-13). The exemption list is named in the test file: BetterAuth's tables (`user`, `session`, `account`, `verification`, plus the `twoFactor` and `passkey` tables when they land; guarded by column grants, §3.1c) and `drizzle.__drizzle_migrations` (`poly_app` and `poly_worker` have SELECT only). Nothing is exempted by name pattern or schema. Each exemption is asserted to exist in the catalog, so a stale or misspelled entry fails the run. Canaries are created and dropped in the test: RLS on but not forced fails, RLS off fails, and an exempt canary passes. Adding an exemption needs SecEng review of the diff. FORCE is written in the same migration that creates the table's policies. The probe's "at least one policy" is satisfied by a permissive `USING (true)`, so it is not proof of isolation. Two more checks close that gap: no policy on an in-scope table applies to PUBLIC (`polroles` containing 0), and no policy that applies to `poly_app` or `poly_worker` has a constant-true qual or check. Role-targeted `USING (true)` for a definer-owning role (`poly_authz`, `poly_govern`) is allowed. Isolation itself is proved per table by a cross-tenant assertion (C-17). **Lives at:** `packages/db` battery **Tested by:** pgTAP in CI; the build fails if any table is uncovered.		
C-14	Per-command policies with `WITH CHECK`; no `FOR ALL`. Tenancy and actor columns are not updatable. No DELETE policies on append-only tables. The entity-type exception is label-only: `poly_app` and `poly_worker` hold column-level SELECT on `entity_type` that omits	T-10, T-14, T-19	POLY-M1

ID	Control, location and test	Threats	Status
	`owner_user_id`. Column INSERT and UPDATE privileges for `poly_app` and `poly_worker` are an allowlist: UPDATE only on `entity.name`, `entity.tax_classification`, `membership.role`, `invitation.revoked_at` and `notification.read_at`; INSERT only on the columns the app supplies (never `id`, `created_at`, `purged_at`, `accepted_at` or `revoked_at`). A table-level grant counts as every column. `invitation.token_hash` is not readable by `poly_app`. **Lives at:** `packages/db` schema **Tested by:** pgTAP: a viewer INSERT is denied; an insert with a forged `entity_id` is denied; an `UPDATE` of `entity_id` gets permission denied.		
C-15	Authz functions are owned by `poly_authz`, with a role-targeted policy on `membership`. Every definer function sets `search_path = ''` and has EXECUTE revoked from PUBLIC. SECURITY DEFINER functions are allowlisted. The allowlist includes `poly_journal`'s numbering and chain trigger functions (C-24). • **Sign-up definers** (`packages/db/sql/definers.json`): `app.enter_new_user(uuid)`, owned by `poly_authz`; `app.claim_invitations(bytea, text)` and `app.accept_invitations(bytea)`, owned by `poly_govern`. Each is EXECUTE to `poly_app` alone. The two invitation functions also pin `lock_timeout = '5s'` and lock rows in id order. • **Operator definer:** `app.issue_operator_invitation(text, bytea, boolean)`, owned by `poly_govern`, EXECUTE to `poly_operator` alone. `poly_worker` calling it gets 42501. ◦ The owner's own EXECUTE is revoked. ◦ It refuses with 42501 whenever `app.user_id` is set, so no job or tenant transaction reaches it.	T-13, T-20, T-97, T-98, T-100	POLY-M1

ID	Control, location and test	Threats	Status

    - It never reads `app.actor`, and it pins `lock_timeout = '2s'`.
    - `poly_operator` holds no privilege on any table, and `poly_worker` no INSERT or UPDATE on `invitation` or `audit_event`.
    - `poly_govern`'s INSERT on `invitation` is column-exact, and one policy admits it: no entity, no inviter, role `viewer`, `issued_via` operator or bootstrap, and no user in context.
    - `poly_govern`'s `UPDATE (revoked_at)` is admitted, with no user in context, only on an unclaimed bootstrap row. Only this definer's body writes `revoked_at`.
    - Its name is a reserved literal, allowed only in `packages/tenancy/src/operator-invitation.ts`. That file can be imported only from the CLI's `user-invite.ts`.
- `claim_invitations` refuses when `app.user_id` is set, returns nothing, and writes nothing, so a signed-in transaction gets neither a count nor a lock. `accept_invitations` takes only the token hash from its caller and reads the email from the `user` row.
- `poly_govern`'s membership INSERT policies are exactly two: `seat_creator`'s (the caller, as owner, on the entity just created) and `accept_invitations`'s, which admits only `role = 'viewer'` for the caller. `poly_app` holds no INSERT on `membership`, and its policies on `invitation` are exactly the owner's select, insert and update.

Lives at: `packages/db`
Tested by: pgTAP: the set of `prosecdef` functions equals the allowlist, with owner and EXECUTE list; `proconfig` pins `search_path`; a manager can list members without recursion; `claim_invitations` returns void and raises 42501 with `app.user_id` set; an owner invitation is claimed and not seated even with `second_step_at` set; only `seat_creator` and `accept_invitations` insert into `membership`; `pg_policies`

ID	Control, location and test	Threats	Status
	for `invitation` and `poly_app` lists exactly three; `issue_operator_invitation` raises 42501 with `app.user_id` set and is refused EXECUTE as `poly_app`, `poly_govern` and `poly_authz`; with `app.actor` forged, its audit row still reads `operator:cli`; with no user in context, `poly_govern` cannot revoke a non-bootstrap row or a claimed bootstrap row; exactly one `app` function writes `revoked_at`. Lint: the literal outside its file, plain or escaped, is refused, and so is an import of the issuer or of `db/operator` from anywhere but `user-invite.ts`.		
C-16	Views are `security_invoker = true`. No materialized views over tenant data. **Lives at:** `packages/db` **Tested by:** pgTAP catalog check on `reloptions` and `relkind`.	T-13	POLY-M1
C-17	The §4.2.1 matrix is stored as data. QA generates an endpoint × role test matrix, and a route with no matrix row fails CI. • **Probes.** Each active row carries probes that run as `poly_app` under `withTenant` in a rolled-back transaction, for owner, manager, viewer and an outsider with no membership. The outsider must be denied on every probe, so every active row is also a cross-tenant check. • **Activation.** A row is active when its tables exist and every function in its `requires` list exists. `requires` holds back only the probes that need the missing function: a deny the current schema can answer runs now. An active row with no probes fails. The set of active rows is pinned in the test, and a row that drops out fails. • **Reverse coverage.** Every in-scope relation is named by a matrix row, by the C-13 exemption list, or by a reviewed `unlisted` entry with a reason. A table on the exemption list may not appear in the matrix. A stale or uncovered entry fails. `public.notification` (recipient-scoped) and `public.entity_type` (type	T-10, T-25	POLY-M1, growing with each feature

ID	Control, location and test	Threats	Status
	metadata, R-9) are unlisted, and each keeps its own cross-tenant assertion. • **The harness can fail.** A canary adds a `TO poly_app USING (true)` SELECT policy to `membership` in a rolled-back transaction, and asserts that the outsider probe then reports allow. **Lives at:** `packages/tenancy`; QA suite **Tested by:** Generated Vitest + pgTAP tests; coverage check.		
C-18	Tenant-visible primary keys are UUIDs. No sequential IDs in URLs or the API. **Lives at:** `packages/db` schema **Tested by:** Catalog check (`pk_not_uuid`, with a canary): every primary-key column in scope has type `uuid`.	T-17	POLY-M1
C-19	Parent links (OQ-19): • **Insert** needs owner on both the child and the parent. This is unchanged, and it is what stops T-16. • **Delete** needs owner on either end. • **Read.** A link is readable with any membership on the parent, or owner on the child. `app.parents_of(entity_id)` returns ancestor ids and display names, with the direct parents marked, to the child's owner only. • **No `poly_app` or `poly_worker` tenant policy reads `entity_rollup`.** Structure grants no row. The only policies that consult it are the role-targeted ones of `poly_rollup` (C-25). **Lives at:** `packages/tenancy`; `entity_rollup` policies **Tested by:** pgTAP: • owner of the child only is denied the insert; • a member of the parent sees no child rows; • a non-member reads no link; • `parents_of` returns names only, and nothing to a non-owner;	T-16, T-71	POLY-M1 (junction); `parents_of` with POLY-1.1

ID	Control, location and test	Threats	Status
	• a catalog check that no policy for `poly_app` or `poly_worker` references `entity_rollup`.		
C-25	Roll-up and graph definers (OQ-19, OQ-25): • **`poly_rollup`** owns `app.rollup_totals`, `app.rollup_economic`, `app.rollup_tree` and `app.parents_of`. ◦ NOLOGIN, NOBYPASSRLS, owns no tables, SELECT only. ◦ Its role-targeted policies are confined to `app.rollup_descendants()` (plus the equity rows of one-link entity holders of those entities, for weighting), and, for `parents_of`, to the ancestors of entities the caller owns. ◦ Results come back through a fixed `RETURNS TABLE` of aggregates by entity, report line and currency: never an account, entry, line, payee, memo, lot or position. ◦ The guard is viewer+ on the parent. ◦ `rollup_economic` takes no Person argument. • **`poly_graph`** (new) owns the BEFORE INSERT trigger functions `app.check_rollup_edge()` and `app.check_equity_holding()`. ◦ It has graph-wide SELECT on `entity_rollup` and `equity_holding`, SELECT on owner `membership` rows and `entity.kind`, and INSERT on `notification`. ◦ The wide read is acceptable because a function returning `trigger` cannot be called to return data. ◦ Refusals name no entity (R-15). ◦ Both triggers take the advisory lock. • **Placement notice.** A notice goes to every owner beneath, in the same transaction, with an	T-70, T-71, T-72, T-73, T-77, T-78	Triggers with the junction (POLY-M1); definers and popover with POLY-1.1

ID	Control, location and test	Threats	Status

`audit_event`. It names the new top-level parent and says what becomes visible.

- **Ancestor-invite notice.** When an invitation to a parent becomes live (after any §4.2.2 approval, before the invitee's first sign-in), every owner of every entity beneath that parent, at any depth, gets a notification. It is written in the same transaction by a `poly_graph` trigger (AFTER INSERT on `invitation`, and on `membership` for a membership created without one, deduplicated per person).
  - It names the parent and the new member, and carries the consent-disclosure sentence and the pull-out link.
  - Any role triggers it, because viewers see totals too.
  - A later role raise on the parent does not.
  - Nothing waits on the recipients.
- **Consent disclosure.** The inclusion step (adding an entity to a parent), the placement notice and the new-member-above notice carry this text verbatim, as three sentences, with `<parent>` the only variable:

  > Members of <parent>, and of any parent above it, see class and category totals for this entity for any period.
  > For a small entity, that effectively reveals individual amounts and their dates.
  > Nothing else is shown: no payee, memo, account or counterparty.

  - The first sentence names every parent above because a member of any ancestor can call the roll-up on that ancestor, and its descendants, at any depth, include this entity (`app.rollup_descendants()`). So members of every parent above see the same totals, now and after any later placement (T-73).

ID	Control, location and test	Threats	Status
	◦ `<parent>` is the parent the entity is being added to, or, in a notice, the parent named in its first sentence. The wording is a security requirement. The Playwright check compares the rendered block to this text exactly. Changing it needs a security-gate review. Any date range is allowed: there is no period guard. • **Holdings popover on a parent.** Positions come only through ordinary `poly_app` RLS, for the parent's own accounts and for entities where the viewer is a member. Any other entity shows its total and the boundary panel. No definer returns a position. • All of these functions are on the definer allowlist (C-15) and in the role-hygiene set (C-10). **Lives at:** `packages/db` SQL; `packages/tenancy` **Tested by:** pgTAP (ARCH §3.3a–c tests, plus): • a parent viewer without child membership gets totals equal to the child owner's figures, and 0 rows from every tenant table and from the popover's position read for that child; • a cycle through an entity invisible to both the inserter and `poly_rollup` is refused; • Playwright: the inclusion step, the placement notice and the ancestor-invite notice render the consent-disclosure text; • pgTAP: an invitation going live on a Holding notifies the owners of a Person two levels down whom the inviter cannot see, and a role raise does not; • calling a `poly_graph` function directly fails; • the result-column set of each `poly_rollup` function equals its pinned shape.		
C-27	Cap table and allocated income, under the new `poly_stake` role (NOLOGIN, NOBYPASSRLS, owns no tables, SELECT only; OQ-25):	T-74, T-75	POLY-1.0 (tax allocation)

ID	Control, location and test	Threats	Status
	• **`app.cap_table_candidates(entity_id)`** is for an owner of `entity_id` only. It returns the id and display name of members' own Persons, and of Person, Trust and Business entities visible to the caller. Nothing else. • **`app.my_stakes()`** is a holder's owners' read path. It returns entity id, display name, units, total units and effective-from, for holder entities the caller owns (or the caller's own Person). Other holders' rows are never returned. The `equity_*` SELECT policies are not widened for non-members. • **`app.allocated_income(taxpayer_entity_id, from, to)`** returns income totals by tax tag × stake, never rows. ◦ Its reads are confined by role-targeted policies to `app.held_entities()`: entities in which the caller's own Person, or an entity the caller owns, has ever held units, plus one link onward. ◦ Guard: the caller has a membership on the taxpayer entity. • **Holder notification** is written by the `poly_graph` trigger (C-25). **Lives at:** `packages/db` SQL; `packages/tax` **Tested by:** pgTAP (ARCH §3.1a tests, plus): • a non-owner calling `cap_table_candidates` gets 0 rows; • `my_stakes` returns only the caller's holder rows; • a holder's owner reads 0 rows from `equity_holding` directly on a held entity where they are not a member; • `allocated_income` returns totals equal to Σ(tagged income × stake), and 0 rows for an entity outside `held_entities()`.		; `my_stakes` with the cap-table editor
C-20	Only owners change membership. A trigger keeps at least one owner (except during purge). Every change	T-15	POLY-M1

ID	Control, location and test	Threats	Status

writes an `audit_event`. With more than one owner, the actions in §4.2.2 go through C-28.

- **The first membership.** An entity's first owner row is written only by `app.seat_creator()`, an AFTER INSERT trigger on `entity`, SECURITY DEFINER, owned by `poly_govern`. It seats `app.current_user_id()` as owner and raises `42501` when no user is set. Its INSERT policy on `membership` is `TO poly_govern` and checks `user_id = app.current_user_id() AND role = 'owner'`. No `poly_app` policy lets a user seat themselves on an existing entity, so an entity left without members cannot be claimed. Policies added later for other `poly_govern` definers are OR'd with this one, so each must be role-targeted and narrow on its own.
- **Every other membership write.** `poly_app` holds UPDATE (`role`) and DELETE on `membership`, owner only, and no INSERT. A new member is seated only by the invitation-accept definer, against a live invitation addressed to them, so a direct insert can neither skip the invitee's consent nor the second-step gate (C-03).
- **The owner count.** `app.owner_count(entity_id)` returns null to a caller with no membership on the entity, so it is neither a size nor an existence oracle. Every consumer treats null as a refusal: `owner_count(…) = 1` is not true for null, and the last-owner trigger refuses on 1 or null.
- **The last-owner trigger.** `app.keep_an_owner()` locks the entity row (`FOR UPDATE`) before it counts owners, so two owners demoting each other at once serialize and one is refused (`23514`). If the lock finds no row, it refuses instead of continuing unlocked. It is VOLATILE, and it reads the count in a statement after the lock.

ID	Control, location and test	Threats	Status
	Lives at: `packages/tenancy`; `packages/db` SQL (triggers) **Tested by:** pgTAP: a manager's grant is denied; removing the last owner raises; an entity insert with no user set raises `42501` and leaves no row; the creator holds exactly one owner row; a non-member reads `owner_count` as null; two connections demoting the two owners of one entity: one succeeds, one is refused.		
C-28	Co-owner approval (§4.2.2): • `change_request` and `change_approval` tables. • Direct `poly_app` writes for (a) to (c) pass only while `app.owner_count = 1`. • `app.apply_change` is owned by `poly_govern`. It recomputes the approvers at apply time and re-validates the payload. • Requests expire after 14 days. Step-up on request, approve and decline. An audit event at every step. • The invite email is sent only on apply. • At the 1 → 2 owner transition, every outstanding direct-path invitation is revoked in the same transaction, under a lock on the entity row (OQ-31, §4.2.2). A direct-path invitation is one with `issued_via = 'owner'` and no `change_request_id` (ARCH A-24). Operator and bootstrap rows carry no entity, so the transition never touches them. • The founding owner, with `app.force_change`, can force only co-owner removal or demotion, after a decline or expiry. Transfer is founder-only, and the founder must hold owner. **Lives at:** `packages/tenancy`; `packages/db` SQL **Tested by:** pgTAP: • with 2 owners, a direct membership INSERT, a role raise or an `equity_holding` INSERT by one owner is denied; • `apply_change` with one approval missing is denied;	T-26, T-15, T-81	The gate, the founder trigger and the 1 → 2 revocation in POLY-M1, with tenancy (ARCH §3.1b); the request flow in POLY-1.0 (sharing, POLY-70); the cap-table edit with the editor

ID	Control, location and test	Threats	Status
	• an approval by an ex-owner does not count; • a decline or an expiry blocks; • a stale cap-table payload is declined; • a sole owner's invite applies at once; • removing a viewer stays single-owner; • `force_change` by a non-founder, on a pending request, on an invite or a cap-table edit, or targeting the founder, is denied; • a forced removal notifies the target and writes `change.forced`; • the founder cannot drop their owner role without first transferring the founder role; • the tombstone is refused while the user is still the founder of a multi-owner entity; • when a second owner joins, every other outstanding direct invitation (any role) is revoked in that transaction and its inviter notified; an invitation applied through `apply_change` is not; • a revoked invitation's token is refused at acceptance; • two direct owner invitations accepted concurrently end with exactly two owners and the other invitation revoked; • after dropping back to one owner, a new direct invitation is revoked again at the next 1 → 2 transition; • a raised owner still waiting for enrolment does not trigger the transition; enrolment does. Integration: no invite email before apply; each step without a recent re-auth returns 403.		
C-21	Append-only: row-level BEFORE UPDATE/DELETE triggers plus a statement-level BEFORE TRUNCATE trigger. The only exception is `current_user = 'poly_purge'` together with a matching `app.purge_entity`.	T-21, T-50	POLY-M2

ID	Control, location and test	Threats	Status
	Lives at: `packages/db` SQL **Tested by:** pgTAP: UPDATE, DELETE and TRUNCATE all raise for `poly_app`, `poly_worker` and `poly_owner`; `poly_app` with the GUC set still raises.		
C-22	Deferred balance constraint trigger (ARCH §3.2). **Lives at:** `packages/ledger` SQL **Tested by:** pgTAP + property-based tests.	T-22	POLY-M2
C-23	A BEFORE INSERT trigger *forces* the actor from the session context. `actor_kind = user` requires `actor_id = app.current_user_id()`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: a forged `actor_id` is overwritten or rejected.	T-23, T-25	POLY-M2
C-24	Per-entity ledger hash chain: • **Hash.** `entry_hash = sha256(prev_hash ‖ canonical(entry, its lines))`, in `journal_number` order per entity. The canonical encoding is pinned and versioned: field order, numeric text form and UTF-8 NFC. • **When it is written.** The hash covers the lines, so it is written at commit by a deferred constraint trigger, after the balance check. It goes into an append-only `journal_chain` row (entity, `journal_number`, entry, `prev_hash`, `entry_hash`, `alg_version`). The posted entry is never UPDATEd. The chain head sits on `entity_journal_counter`, whose row lock already serializes an entity's postings. • **Who writes it (OQ-30, confirmed with changes).** `poly_app` and `poly_worker` cannot write `journal_chain` or `entity_journal_counter`. The numbering and chain triggers are `SECURITY DEFINER` functions owned by `poly_journal`, NOLOGIN and NOBYPASSRLS. Its grants, each through a role-targeted policy (`TO poly_journal`), are:	T-24, T-21, T-80	POLY-M2 (with the ledger); the numbering trigger with POLY-64

ID	Control, location and test	Threats	Status

    - SELECT, plus UPDATE of `last_number`, `head_number` and `head_hash` only, on `entity_journal_counter`;
    - INSERT on the counter, used only by an AFTER INSERT trigger on `entity` that creates the row with number 0 and the fixed seed. No other role inserts a counter row, so no caller can pre-seed a head;
    - INSERT on `journal_chain`, never UPDATE or DELETE. `journal_chain` is append-only under C-21, and only the purge deletes it;
    - SELECT on `journal_entry`, `journal_line` and `journal_chain`. The read is graph-wide, which is acceptable for the reason given for `poly_graph`: a function returning `trigger` cannot be called to return data.
- **No line after the chain row.** `SET CONSTRAINTS … IMMEDIATE` is open to any role, so `poly_app` could fire the deferred chain trigger before an entry's lines exist. A BEFORE INSERT trigger on `journal_line`, also owned by `poly_journal`, refuses a line whose entry already has a `journal_chain` row. The chain therefore always covers every line, and an early fire fails the posting instead of leaving a false SEV1 for the verifier.
- **Readers.** `poly_app` and `poly_worker` read `journal_chain` and the head through ordinary tenant RLS, viewer and up, for the export head and the verifier. The verifier writes only `audit_event` and `notification`.
- **Verifier.** A weekly job recomputes each entity's chain. A break writes an `audit_event` and emails the entity's owners and the operator, as SEV1 ledger tampering (§10).
- **Exports.** Every export carries the chain head (`journal_number`, `entry_hash`), and the `export.completed` audit event records it.

ID	Control, location and test	Threats	Status
	• **Purge.** A purge deletes the entity's chain, and the `entity.purged` event records the final head. • **Limits.** A superuser can recompute the whole chain. Detection therefore rests on a head held off the box: an export the owner keeps (R-1). **Lives at:** `packages/ledger` SQL; worker verifier job; export module **Tested by:** pgTAP and integration: • an edit made as superuser (triggers disabled) to an entry, a line or the order is detected by the verifier; • a rolled-back posting leaves no chain row and no gap; • `poly_app` cannot insert or update `journal_chain`, or insert or update an `entity_journal_counter` row; • a new entity gets its counter row at number 0 with the seed, and a caller-supplied `journal_number` is overwritten; • with `SET CONSTRAINTS ALL IMMEDIATE`, inserting an entry then its lines is refused, and no chain row commits; • catalog: each `poly_journal` function is attached as a trigger only to its intended table (`pg_trigger.tgfoid` to `tgrelid`), and a direct call fails; • an export's head matches the verifier's; • a correction (reversal plus replacement) takes two chain links.		
C-30	HPKE sealing with AAD binding. `web` seals, `worker` opens; keys are held in a keyring by `key_id` (§6.3). **Lives at:** `packages/secrets`; the import rules in `layers.json` (ARCH §2.1; POLY-82) **Tested by:** Unit: round trip works; a swapped AAD, tampered ciphertext or unknown `key_id` all fail closed. Integration: the `web` container has no keyring file. Lint (POLY-82): `secrets/open` outside `apps/worker` and the HPKE library	T-30, T-31, T-32	Seam in POLY-M2 (POLY-130); first real use in POLY-1.0 (POLY-77)

ID	Control, location and test	Threats	Status
	outside `packages/secrets` both fail. Build: the `web` bundle scan (ARCH §7.6). The opener and index-graph tests are in A-39.		
C-31	`poly_app` has no SELECT on the sealed columns. DTOs and the OpenAPI spec omit them. **Lives at:** `packages/db` ; DTOs **Tested by:** pgTAP: permission denied. OpenAPI freshness diff.	T-31, T-33	POLY-M2
C-32	Unlink enqueues a worker revoke job. The row stays `revoke_pending` until the provider confirms, with retries and backoff; a failure is surfaced to the user. Every step writes an `audit_event` . **Lives at:** `packages/import` ; worker **Tested by:** Integration: when the fake provider's revoke returns 500, the row is not deleted.	T-34	POLY-M2 (fake provider), POLY-1.0
C-33	Logs are safe by construction. Pino serializers log only declared fields, and `redact` covers known paths. Never logged: bodies, `Cookie` , `Authorization` , SQL parameters, memo, payee. The declared fields are a committed JSON Schema, `packages/config/log.schema.json` , with `additionalProperties: false` . The logger keeps a field only if its key is declared *and* its value passes that field's type, pattern and length. Anything else is dropped and counted, never named. Every field is a bounded primitive (enum, pattern or format, with a maximum length). The only object is `err` , which is `{type, code}` : no message, stack or driver detail. There is no free-text message field, so a raw provider payload has nowhere to go. Child-logger bindings and the process crash handlers pass the same filter. `redact` is a backstop, not the control. **Lives at:** `@poly/config/log` (the only module that imports `pino`) **Tested by: Canary test:** the integration run seeds canary token, URL, account-number and memo values. CI captures every log line and fails if any canary appears.	T-33	POLY-M1 (harness), extended with each adapter

ID	Control, location and test	Threats	Status
C-34	Adapters redact full account numbers, PANs and credential-bearing fields from stored payloads, keeping last-4 only. Payloads are readable by manager+ only. **Lives at:** Each `packages/import-*` **Tested by:** Per-adapter fixture containing canary numbers; pgTAP: a viewer cannot read the payload.	T-33, T-36	POLY-M2, POLY-1.0
C-35	`polycarpic secrets rotate` (§6.4). **Lives at:** `packages/secrets` CLI **Tested by:** Integration: after rotation every row opens under the new key and none under the old.	T-30, T-31	POLY-1.0
C-40	Upload caps (e.g. 10 MB, 50k rows). OFX 2.x parsed with DTD and external entities disabled; OFX 1.x parsed with a non-XML parser. No eval. **Lives at:** File adapter **Tested by:** Unit: XXE and billion-laughs fixtures are rejected; an oversize file returns 413.	T-35, T-09	With the file adapter (POLY-77)
C-41	Svelte escaping. `{@html}` is lint-banned everywhere except one sanitizing component. CSV export neutralizes any cell starting with `= + - @`, tab or CR. **Lives at:** the root ESLint config (`scripts/layers/generate.mjs`, block `poly/svelte`), over every `.svelte` file; inline directives are inert in `.svelte` files (`noInlineConfig`), so no comment lifts the rule (A-39); export module **Tested by:** Lint fixtures (`tests/layers/fixtures/24-svelte-html`): a bare `{@html}`, the line, block and script-side disable forms, and a `packages/` path all fail. Unit: a `=HYPERLINK(…)` cell exports inert. Playwright: a `<script>` payee renders as text.	T-06, T-55	POLY-M1 (lint), then with the export feature
C-42	The report formula engine parses formulas to an AST over an account allowlist. `eval` and `new Function` are banned. Node-count, depth and time caps apply. **Lives at:** `packages/reports` **Tested by:** Unit: pathological and malicious formulas are rejected.	T-09	POLY-M5

ID	Control, location and test	Threats	Status
C-43	Zod validates every route and form action. Unknown keys are stripped; a mismatch returns 400. **Lives at:** `apps/web` wrapper **Tested by:** Unit tests on the wrapper.	T-10, T-14	POLY-M1
C-44	Parameterized queries only. Lint bans `sql.raw()` and `sql.unsafe` with non-literal arguments. Raw SQL is allowed only in reviewed migrations. CodeQL `js/sql-injection` runs in CI. **Lives at:** ESLint; CI **Tested by:** Lint + CodeQL.	T-18	POLY-M1
C-45	Reference-data anomaly flag: a price move over 50% day-over-day, or an out-of-bounds CPI revision, is stored but flagged. **Lives at:** `packages/refdata` **Tested by:** Unit.	T-43	POLY-M4
C-46	Attachments (ARCH §3.5 @ fcffe6c): • Stored as `bytea` under RLS. The bytes live in `attachment_blob`, separate from the metadata. • Size limit of 10 MB. Type is sniffed server-side against an allowlist; the client-declared type is never trusted. • Served only as a download: `Content-Disposition: attachment` with an RFC 6266 sanitized `filename*`, plus `nosniff` and a CSP of `default-src 'none'; sandbox`. • Composite entity FKs on attachments and their links. • Insert-only, and included in the purge. • Per-entity storage quota and upload rate limit. • File contents are never parsed. **Lives at:** `packages/attachments` (per ARCH) **Tested by:** Integration: a spoofed type is rejected; a file name containing CR/LF is sanitized; response header assertions; a cross-entity link is denied; purge leaves 0 attachment rows.	T-37	With the attachments feature

ID	Control, location and test	Threats	Status
C-50	Purge per §4.5: the `poly_purge` role, 8 guards checked in order, and a typed confirmation. **Lives at:** `app.purge_entity`; tenancy route **Tested by:** pgTAP: a non-sole owner is denied; a stale export is denied; a write after the export is denied; `poly_app` with the GUC set is denied outside the function; a successful purge leaves 0 rows in every purgeable table plus 1 `audit_event`.	T-50, T-51, T-08	The purge feature (POLY-1.0 scope)
C-51	Export is streamed, never persisted. If it has to run async, the file is encrypted in temp storage and expires after 24 h. Owner only, with step-up; audited with a SHA-256; rate-limited. **Lives at:** Export module **Tested by:** Integration: no file remains after download.	T-53	The export feature
C-52	Account deletion: the user row becomes a tombstone; ledger rows are never UPDATEd. BetterAuth `deleteUser` is overridden. Sessions, accounts, MFA and passkeys are deleted. Linked credentials are revoked, and the user's connections go `principal_invalid`. Per ARCH §3.4a, each solely-owned entity is resolved in the flow by transfer to a named current member, or by a purge through C-50, or the user aborts. The deleting user is the only invoker (§4.6). The tombstone transaction re-checks that no sole-owned entity remains. **Lives at:** `packages/auth` override; tenancy **Tested by:** Integration: • entries remain and resolve to "deleted user"; • the §4.6 negatives; • abort before the final confirmation changes nothing.	T-41, T-54, T-84	POLY-1.0
C-53	`audit_event` is append-only and has no purge exception. Its `entity_id` FK has no `ON DELETE` action: entity rows are never deleted (the purge leaves a tombstone), so no audit row is removed with its entity. A BEFORE UPDATE OR DELETE trigger refuses every	T-52, T-25	POLY-M1 (the table), then events

ID	Control, location and test	Threats	Status
	change, whoever runs it. Readable by owners and by the acting user. **Lives at:** `packages/db` **Tested by:** pgTAP: the row survives a purge and cannot be updated.		with each feature
C-60	Job principal per §4.4: `linked_by_user_id`, `principal_invalid`, no context-free fallback, `app.job_targets()`, role-targeted reference-data policies, restricted `poly_worker` grants. **Lives at:** `packages/db`; `apps/worker` **Tested by:** pgTAP: a job whose linker was revoked writes 0 rows and flips the connection's state; `poly_app` cannot write reference data.	T-40, T-41	POLY-M2 (POLY-64)
C-61	Vercel topology (OQ-13): two Vercel projects from the same repo. • `web` exposes only user routes. It holds only the public keys and the `poly_app` DB credentials. • `jobs` exposes only cron job routes. Each route checks `CRON_SECRET` with `timingSafeEqual`. It holds the keyring and the `poly_worker` credentials. • Job routes are absent from the node build and from the `web` project. **Lives at:** `apps/web`, `apps/worker` (Vercel entry) **Tested by:** Build tests: • the route manifest of each project equals its allowlist; • the `web` build contains no keyring reference; • the node build returns 404 for job routes. Env check in CI: `web` has no keyring variable.	T-42, T-31	POLY-M1 (split); keyring before POLY-77
C-70	Only Caddy publishes ports. Prod and staging use separate compose networks. No Docker socket mounts. Containers run non-root, `read_only`, with `cap_drop: [ALL]` and `no-new-privileges`.	T-60, T-65	POLY-M1 (INFRA)

ID	Control, location and test	Threats	Status
	Lives at: `compose.yml` (DevOps) **Tested by:** Compose policy lint in CI; a post-deploy external port scan shows only 22, 80 and 443.		
C-71	Hetzner target. SSH is key-only (one `ed25519` key per laptop, user `polyops`), with no root login and no password or keyboard-interactive login. Unattended security upgrades, with an alert when a reboot has been pending for 3 days. The firewall is the Hetzner Cloud Firewall, which sits outside the host, so Docker cannot bypass it; it admits only 22, 80 and 443, on IPv4 and IPv6. The local box uses C-76 instead. • **SSH is open to the internet by design (D-2).** The operator's address changes, and with key-only login, password guessing gets nowhere. Restricting 22 by address is optional; the Hetzner console recovers a lockout (INFRA §8.9). • **Residual.** A pre-authentication flaw in sshd is reachable from the internet. The answer is the patch path above, not the firewall. **Lives at:** INFRA §3.3, §7.1 (provisioning) **Tested by:** `provision-check`, which asserts from `sshd -T` that `passwordauthentication`, `kbdinteractiveauthentication` and `permitrootlogin` are `no`, and that unattended upgrades are on; the outside-in scan after every provisioning shows only 22, 80 and 443 on both address families, and a password login ends `Permission denied (publickey)` (INFRA §7.1 step 15).	T-60	POLY-M1 (INFRA)
C-72	The Postgres volume and the local backup spool sit on an encrypted (LUKS) volume. • **Hetzner.** The volume's key is a file on the root disk (INFRA §3.2). It protects a detached or snapshotted volume only, not a captured server or a snapshot of both disks. A captured server is RB-2. • **Local box: unlocked automatically by the TPM** (built in INFRA §3.5, §7.2):	T-61, T-69, T-85	INFRA v1; the local-box variant before the first home-box deploy

ID	Control, location and test	Threats	Status
	◦ LUKS2, with a TPM2 key slot sealed to PCR 7, the Secure Boot state, only. PCR 11 is not used by default: without signed PCR policies every kernel update changes it, and the box stays locked after the automatic reboot. It is added only if Ubuntu 24.04 is measured to ship signed policies (OQ-INFRA-2). ◦ A second, recovery passphrase key slot, kept offline in the operator's password manager and on the recovery sheet, never on the box. ◦ The secrets directory lives on the encrypted partition. ◦ Swap is off, so no secret or decrypted page reaches an unencrypted disk. ◦ Firmware hardening: current firmware; a setup password; Secure Boot on, with the factory keys, in user mode; TPM on; boot from USB or network disabled. ◦ Console hardening: no console autologin; the emergency and single-user shells require the root password (long, random, in the password manager); the bootloader has a password, and its editable and recovery entries are removed. ◦ **The root partition is encrypted too.** PCR 7 records the Secure Boot policy, not the files that boot. With an unencrypted root partition, an offline edit there would run with the disk unlocked (T-85). So root is encrypted as well, in one LUKS2 container with the data, under the same TPM path (PCR 7) and recovery-passphrase slot (layout below). Unattended restart is kept. Devops proves the mechanism on Ubuntu 24.04 server at the first home-box deploy (OQ-INFRA-10). No real data goes on the box until it is proven. **Accepted residual:** the EFI partition and the boot files (kernel, initramfs) stay unencrypted, and Ubuntu's initramfs is unsigned. An attacker with the box for		

ID	Control, location and test	Threats	Status

a while who rebuilds the initramfs can still capture the unlock. Measuring the boot files without locking the box after updates is OQ-INFRA-2 (R-21).

- **Layout: confirmed.** One LUKS2 container holds an LVM group with `/` and `/srv/polycarpic` as logical volumes, under one PCR-7 TPM binding and one recovery-passphrase slot. EFI and `/boot` stay unencrypted (R-21).
  - The root filesystem is encrypted, and a single unlock keeps the unattended restart.
  - The secrets directory is encrypted wherever it sits, since both volumes are inside the container. Swap stays off, and is not added as a third volume.
  - Automatic reboot stays off until the kernel-update reboot check passes (INFRA §7.2 step 6). Accepted, on one condition: while it is off, a pending reboot (`/var/run/reboot-required`) older than 7 days raises the monitor alert, so a kernel fix does not wait silently for the monthly window.
  - If OQ-INFRA-10 fails with both methods, devops stops and returns to SecEng and the Principal. No fallback is pre-approved.

Lives at: INFRA §3.2, §3.5, §7.2
Tested by: `provision-check` : `systemd-cryptenroll` lists exactly a password slot and a `tpm2` slot bound to PCR 7; `swapon --show` is empty; the secrets directory is on the LUKS mount; no autologin; root and bootloader passwords set. Hands-on: with Secure Boot turned off in the firmware, boot asks for the recovery passphrase, and with it back on, the disk unlocks by itself (INFRA §7.2 step 7); `e` at the boot menu asks for the bootloader password (step 8). `lsblk` shows `/` and `/srv/polycarpic` both on logical volumes of the one LUKS mapping, whose slot listing is exactly a password slot and a `tpm2` slot on PCR 7; after a plain reboot and after a kernel-update reboot the box unlocks with no prompt (INFRA §7.2 step

ID	Control, location and test	Threats	Status
	6). The Secure-Boot-off test is unchanged; its passphrase prompt now appears at the console before the network starts. Monitor: a pending reboot older than 7 days alerts.		
C-73	Backups per §6.6, as built in INFRA §6 (D-3): encrypted on the box to an age recipient whose private key is on no box; a B2 bucket with Object Lock in compliance mode and a 35-day default retention; a put-only box key and a separate read key off the box; a monthly off-box restore drill, whose first pass gates the v1.0 release. **Local box:** nightly, to the same bucket, over 443. A Hetzner Storage Box is unreachable under C-76's outbound allowlist. A rotated removable drive stays optional, never required, and is never kept beside the box. **Lives at:** INFRA §6.1 to §6.3, §6.7 **Tested by:** the drill's checks, run off the box, including `delete attempt: DENIED` with the put-only key and a check that the globals carry the RLS roles. With the put-only key, overwriting an object's version, shortening its retention and changing the bucket's default retention are each refused (OQ-INFRA-4).	T-61, T-62	Before the first backup that holds real data; the local-box variant before the first home-box deploy
C-74	Secrets are files, scoped per container (§6.2). Staging has its own distinct values and its own keyring (C-77). The app refuses to start if a secret is missing. No secrets in images. **Lives at:** INFRA; config loader; `infra/lint/policy.json` **Tested by:** Trivy secret scan; the compose-policy lint (secret → service matrix, default deny; each service's `file:` sources lie under its own directory, prod and staging apart); the `image` job's secret-mode test (a service reads its own 0440 files and is refused another service's); `packages/config` loader unit tests; `check-config` run against `migrate`, `web` and `worker`, with the canary file content absent from the output.	T-66, T-31	POLY-M1
C-75	Postgres logging: `log_parameter_max_length = 0`, `log_parameter_max_length_on_error = 0`, `log_statement = 'ddl'`. **Lives at:** Postgres config **Tested by:** Config assertion in the battery.	T-33	POLY-M1

ID	Control, location and test	Threats	Status
C-76	Local-network box posture (OQ-24, ARCH §7.8). This replaces C-71's cloud firewall on that target: • **Router.** No port forwarded, and UPnP/NAT-PMP disabled on the router. • **IPv6.** The box's IPv6 is disabled, or the host firewall drops all inbound IPv6. A home router may pass inbound IPv6 to a global address. • **Host firewall.** nftables, default-deny inbound. Allowed in: 443 (and an 80 redirect) and SSH from the LAN CIDR only. • **Ports.** Caddy is the only published port, bound to the LAN address (`"<lan-ip>:443:443"`). A `DOCKER-USER` chain rule drops non-LAN sources, because Docker-published ports bypass ufw. • **Outbound.** Limited to DNS, NTP and HTTPS (443). • **SSH.** Key-only, no root login. • **Updates.** Unattended security upgrades. • **Monitoring.** An outbound heartbeat, and its absence alerts. • **Remote access.** None from outside the LAN in v1. A VPN later is a new security review. • **TLS: two modes**. Plain-http sign-in is refused in both: Caddy redirects 80 to 443, and the app rejects any auth route reached over plain http, trusting the forwarded protocol only from Caddy. `__Host-` cookies need `Secure` anyway. ◦ **Default, real-domain mode.** ▪ A subdomain of a domain the operator owns, resolved only on the LAN (split-horizon). There is no public A or AAAA record. ▪ A DNS-01 wildcard certificate (`*.home.<domain>`), so the box's own name stays out of public CT logs. ▪ The registrar or DNS-API token is limited to that one zone, a file secret mounted into	T-60, T-67, T-68, T-79	Before the first local-box deploy

Caddy only (R-18).

- Google and GitHub sign-in work. Apple arrives with native iOS (POLY-122).

- **Supported fallback, `.local` with a private CA.**
  - The operator runs a private CA and installs its root on each device. The root carries X.509 `nameConstraints` permitting only the box's name. Browser enforcement of constraints on a locally installed root is unmeasured. Measuring it, for each supported browser, is a **gate item for the first home-box deploy** (R-20).
  - The root key is generated off the box and kept offline (the operator's password manager). The box holds only a name-constrained intermediate that issues short-lived leaf certificates (T-79, R-20).
  - In private-CA mode the Google and GitHub buttons are hidden *and* every OAuth route returns 404 server-side. Sign-in is email + password + the second step.
  - Whether passkeys work on a `.local` RP ID is unmeasured; TOTP is always offered.
- **The mode is explicit, and checked at startup (OQ-32).** `TLS_MODE` must be set; nothing infers it. The check tests the zone, never the host, because in real-domain mode the host deliberately has no public record:
  - **Offline, decisive.** A public origin ending in `.local`, or whose registrable domain is not under a public suffix (the bundled Public Suffix List), requires `private-ca`.
  - **Online.** Any other name requires `real-domain`, and its registrable domain's zone must resolve publicly (an SOA from a public resolver). An NXDOMAIN for the zone refuses start. A DNS timeout does not: the app starts,

ID	Control, location and test	Threats	Status

logs the unverified check, and retries it. A missed check costs only sign-in and certificate issuance, never exposure. A public A or AAAA record for the box's own name is logged as a warning, because it breaks the split-horizon promise.

- A mismatch either way refuses start, with the reason logged.
- **Certificate issuer.** The provisioning check and the heartbeat health check confirm the served leaf matches the mode: in private-CA mode it chains to the box's name-constrained intermediate, and in real-domain mode to a public ACME issuer. A mismatch alerts the operator.

- **Moving a `.local` box to a real domain later.**
  - Every session ends, because cookies are host-bound.
  - Passkeys do not survive, because they are bound to the old RP ID. Before the move, the runbook requires each owner and manager to hold TOTP or backup codes, with the operator's `reset-mfa` CLI as the fallback.
  - Google or GitHub sign-in is then added by linking from Settings while signed in (C-05: an existing session and a verified email). It is never auto-linked by email.
  - Every device should remove the old private root.

Lives at: INFRA local-box provisioning (DevOps)
Tested by:

- a provisioning check script (nftables rules, the `DOCKER-USER` rule, the publish binding, the sshd config);
- an outside-in scan, from a host off the LAN, of the home's public IPv4 and the box's IPv6 address,

ID	Control, location and test	Threats	Status
	showing nothing open; • a test that an outbound connection to a non-allowed port fails; • an auth route over plain http is refused in both modes; • in private-CA mode every OAuth route (all three providers) returns 404; • startup: a `.local` name, or a name under no public suffix, outside private-CA mode refuses; a public-suffix name in private-CA mode refuses; real-domain mode on a host with no public record but a resolvable zone starts; a zone NXDOMAIN refuses; a DNS timeout starts and logs; • the issuer check alerts when the served leaf does not match the mode; • a certificate for an out-of-constraint name issued under the private root is rejected by each supported browser.		
C-77	Staging credential isolation (ARCH §7.5): • **Its own keyring.** Staging's keyring has distinct `key_id`s, lives in its own secrets directory, and is mounted into staging `worker` only. • **Sandbox secret only.** Staging holds Plaid's sandbox secret, never the production secret. The `client_id` is shared across Plaid environments, and the secret is not. • **Startup checks.** Staging refuses to start unless `PLAID_ENV=sandbox`. Prod refuses to start with sandbox. • **SimpleFIN on staging** accepts only access URLs whose host is the demo bridge (exact host unmeasured; pinned at POLY-77). • **No real data.** No real credential may ever be linked on staging.	T-76, T-65	POLY-77 (the first real adapter on staging)

ID	Control, location and test	Threats	Status
	Lives at: config loader; `packages/import-plaid`, `packages/import-simplefin`; INFRA **Tested by:** Config-loader unit tests for both refusals; an adapter unit test rejecting a non-demo SimpleFIN host under `APP_ENV=staging`; a provisioning check that the staging and prod secret directories share no file.		
C-78	Operator accounts and the provisioning token. • Every console account that can reach a target has two-step sign-in, with its codes in the password manager, its own password, and the operator's own address for recovery: Hetzner, Cloudflare, Backblaze, GitHub, Healthchecks.io, Resend, the registrar, and Vercel and Neon if used. • The domain carries the registrar's transfer lock. • The Hetzner API token (read and write: it can rebuild the server, move the volume or open the rescue console) is deleted once `create.sh` has run, and generated again for the next run. It is not left in `~/.config/hcloud`. **Lives at:** INFRA §7.0, §7.1 steps 4, 5 and 8 **Tested by:** the runbook checklist; `provision-check` cannot see these, so the monthly window re-reads the list (INFRA §8.2).	T-88	Before the first deploy
C-79	**The backup login (ruling on ARCH §3.3, §7.2).** `poly_backup` is confirmed as the one login with BYPASSRLS, and only under every condition below. Why it is needed: `pg_basebackup` needs REPLICATION, and `pg_dump` stops on a FORCE-RLS table it cannot read whole. Why it is acceptable: a REPLICATION login already copies every byte of the cluster, so BYPASSRLS widens how the role reads, not what it can read. What matters is where the credential can be used and where its output can go. • **Attributes.** LOGIN, REPLICATION, BYPASSRLS; NOSUPERUSER, NOCREATEDB, NOCREATEROLE; a member of `pg_read_all_data` and of nothing else;	T-89, T-61, T-62	POLY-M1 (INFRA), with the first backup

ID	Control, location and test	Threats	Status

no other grant; owns nothing; `CONNECTION LIMIT 3`; `ALTER ROLE poly_backup SET default_transaction_read_only = on`.

- **Where it connects from.** The `backup` container sits on its own compose network, shared with `postgres` only. The network is `internal: true` (no internet route) and has a fixed subnet. `pg_hba.conf` lists these lines first, so they match first:

```
host replication poly_backup <backup-net-cidr>
scram-sha-256 host polycarpic poly_backup
<backup-net-cidr> scram-sha-256 host all
poly_backup all reject host replication all
all reject
```

- **The credential and the internet never share a process.** The `backup` container holds `db_password_backup` and the age *public* key. It pipes each dump and base backup through `age` as it is taken, so plaintext never lands in the spool. It does not hold `backup_store_credential`. The upload is a separate step (host script or `uploader` container) that holds only the put-only key and the spool, and is not on the backup network. This replaces INFRA §2.2's single `backup` service, which held the put-only key with internet egress.
- **Output.** The globals use `pg_dumpall --globals-only --no-role-passwords`, so no password hash leaves in the globals file; restored logins get new passwords. The data dump does carry BetterAuth's tables (password hashes, session tokens): that is the reason the age private key is on no box (§6.6).
- **Slots and WAL.** `max_replication_slots = 2`, `max_wal_senders = 3`, and `max_slot_wal_keep_size` set (2 GB to start; unmeasured, tuned with OQ-INFRA-5), so a slot cannot fill the disk. The 5-minute health ping is

ID	Control, location and test	Threats	Status

skipped, and an alert raised, if `pg_replication_slots` shows any non-temporary slot. The base backup runs `pg_basebackup –X none` as tar to stdout, so it takes no slot. WAL comes from `archive_command`, which encrypts each segment with `age` itself.

- **Detection.** `log_connections = on`. `polycarpic-ops status` lists the last 24 hours' `poly_backup` connections against the backup and pre-migration job runs, and flags any unmatched one.
- **Same path for every dump.** The pre-migration backup (INFRA §4.5) and the nightly set both run through this container. No other dump path, and no superuser dump, exists in the scripts.
- **Rejected alternatives.** (i) *Physical backup only*: REPLICATION alone still reads everything, and it loses the portable dump that restores across majors and serves the pre-migration backup. (ii) *Dump as the table owner*: `poly_owner` would need LOGIN and would carry DDL and ownership, which is worse than a read-only BYPASSRLS role. (iii) *Dump as the superuser* through `docker compose exec`: puts the superuser in a nightly job. (iv) *A per-table `T0 poly_backup USING (true)` policy with `--enable-row-security`*: the same reach, plus a silent failure, because a table that misses its policy dumps zero rows instead of erroring.

Lives at: `packages/db` role migration; the Postgres config fragment and `pg_hba.conf` (INFRA §2.2); the backup and upload scripts (INFRA §6.1)
Tested by: pgTAP: the attributes and membership above, and no grant beyond `pg_read_all_data`; an INSERT, UPDATE or DDL as `poly_backup` fails. `provision-check`: from `web` and from the uploader, a `poly_backup` login (SQL and replication) is rejected; from `backup` it succeeds; `backup` cannot reach the internet; the uploader has no `db_password_backup`; the spool holds only `.age` files; the slot and WAL settings. Compose-policy lint: only `backup` and `postgres` are on the backup

ID	Control, location and test	Threats	Status
	network, it is `internal`, and only `backup` mounts `db_password_backup`; `uploader` is on `upload-net` alone, which `postgres` is not on, so it shares no network with the database. The drill restores the base backup with the archived WAL. The drill: a restore of the dump has every row of a FORCE-RLS table (counted against the base-backup restore).		
C-80–83	Supply chain (§8). **Lives at:** CI; INFRA §4.4 **Tested by:** §8	T-63, T-64, T-66, T-92, T-93, T-94	POLY-M1
C-84	**The context-free paths (ARCH §2.1, §3.1).** There are two: `db/auth` for sign-up and `db/operator` for the operator's CLI. Only `db/auth` can set `app.user_id`, and only through the new-user step. • `db/auth` exports BetterAuth's plain pool (login `poly_app`, no other role), its Kysely dialect, `withContextFreeTx` and the derived `fields` map. Only `packages/auth` may import it, by the layer map. It is on the A-39 no-inline-disable list. • `withContextFreeTx` takes no user id. It refuses to nest inside `withTenant`, `withSystem` or `withOperatorTx`, and they refuse to nest inside it (the same guard, over all four). Its first statement asserts `app.user_id` is empty. • `db/operator` exports `withOperatorTx` and the pool shutdown, nothing else. ◦ Only `apps/worker/src/cli/user-invite.ts` may import it, by the layer map's confinement. No other `@poly/db` subpath re-exports it. That file ships in the app image but runs in the `operator` service, never in `worker`. ◦ `withOperatorTx` runs on its own pool, built from the operator's `DATABASE_*` settings, and takes no argument besides its callback. On every call, before any connection, the pool refuses with `OperatorPoolRefused` unless the configured	T-90, T-11, T-01, T-98	POLY-M1

ID	Control, location and test	Threats	Status

user is `poly_operator`, so in a `worker` process it fails closed. That check is a seatbelt. The boundary is that no process but the `operator` service holds `poly_operator`'s password, and that `poly_worker` cannot execute the issuer (C-15).

  - `poly_operator` is a LOGIN role with the same attribute hygiene as the others. It holds CONNECT, USAGE on schema `app` and EXECUTE on `app.issue_operator_invitation`, and nothing else, by catalog and by effective privilege. It is admitted by `pg_hba` from the app subnet with scram only.
  - It asserts `app.user_id` is empty and sets only `app.actor = 'operator:cli'`, locally.
  - Its handle has no new-user step.
  - The actor setting is a label for the database's logs. The definer it serves does not trust it (C-15).

- The new-user step is reachable only through the transaction handle that `withContextFreeTx` passes in (a non-enumerable method, `enterNewUser`), never from any `@poly/db` index. It uses `set_config('app.user_id', id, true)`, never a session `SET`. It refuses a second call in one transaction, and a call on any other kind of transaction, where the method does not exist.
- *The proof of a new user.* A `BEFORE INSERT` trigger stamps `user.signup_txid` with the top-level transaction id (`pg_current_xact_id()`, the same inside BetterAuth's savepoint) on an insert by `poly_app`, and forces it to null for any other role. One user per transaction: a second stamped insert is refused. `poly_app` cannot write the column. The step's definer, `app.enter_new_user`, refuses unless the user's stamp equals the current transaction id, `app.user_id` is empty, and the user has no Person.

ID	Control, location and test	Threats	Status

An UPDATE does not change the stamp, and a setting cannot forge it.

- *Acceptance trusts the same proof.* `app.accept_invitations` refuses unless `app.current_user_id()` is a user stamped by this transaction, so a forged `app.user_id` (R-2) cannot seat an existing user. While the stamp is set, the user's `email` and `email_verified` cannot change, so a sign-up cannot re-address itself to another invitee before accepting. It then clears the stamp. A deferred constraint trigger refuses any commit that leaves a stamp on a `user` row, for every role, so no committed row carries one and a restored database carries only nulls.
- *Residual.* Code that runs SQL as `poly_app` can still create a brand-new account and act as it within that transaction (R-2). It cannot reach an existing user through either step. Such code can also insert a fresh user carrying an invited address with `email_verified` true, and accept that address's invitations; the account carries the invitee's own address, so the invitee can recover it by magic link and password reset, and must also unlink any IdP account they do not recognise (R-2).
- Each user-creating route runs whole in one transaction: the invitation check and its single-use consumption, the `user` row, the Person, the owner membership and the seeded accounts. Invitation-only sign-up (C-01) therefore cannot leave a user row behind when an invitation is refused or consumed twice. The session-loader re-run stays as a backstop, under `withTenant`.
- *ARCH §3.1c:* the sign-up hook refuses to run when no `withContextFreeTx` is open. It throws and never opens a transaction of its own. This catches a route that skips the wrapper in testing; it is not a production guarantee, because by then BetterAuth has already

ID	Control, location and test	Threats	Status
	committed the user row. The guarantee is one test per user-creating route (POLY-86), and the backstop is the session-loader re-run. **Lives at:** `packages/db` (`db/auth`); `packages/auth` (hooks, routing pool); `layers.json` **Tested by:** pgTAP and integration: the step refuses an existing user's id (also after this transaction updated that user's row), a second call, a call after the transaction ended, and an id whose insert was rolled back to a savepoint; `poly_app` cannot write `signup_txid`; a second user insert in one transaction is refused; acceptance forged to an existing user, or to a user inserted earlier, raises and writes no membership; changing the new user's `email` or `email_verified` before acceptance raises, and a name change does not; a sign-up that skips acceptance is refused at commit and leaves no row; a superuser insert commits with a null stamp; with the step removed, the sign-up fails under RLS and leaves no user row; a throw after the step leaves no `user`, `entity` or `membership` row; after commit and after rollback, the next checkout of that backend reads `app.user_id` empty (C-12), and after `withOperatorTx` it reads both `app.user_id` and `app.actor` empty; `withOperatorTx` with `DATABASE_USER=poly_worker` throws `OperatorPoolRefused` and opens no connection, and inside it `current_user` is `poly_operator`; `poly_operator`'s attributes, memberships and every ACL are catalog-exact, and its effective privileges are EXECUTE on the issuer alone, USAGE on `app` alone, and no CREATE or TEMP; all sixteen nesting pairs of the four wrappers throw; no export surface but `db/operator` carries `withOperatorTx`; two sign-ups racing on one invitation give one user; with no wrapper open, the hook throws and writes nothing (POLY-86, per route). Lint fixtures: `db/auth` imported from outside `packages/auth` fails, with or without a disable comment; the confinement fixtures under Import boundary (§6.3).		
C-85	**The migrate client and the chain's integrity (ruling POLY-76 §4.2–§4.4).** • The session-`SET` lint exemption names one file, the migrate module in `packages/db`, not a glob. Only the one-off migrate entry point imports that module, and	T-91, T-11, T-13	POLY-M1

ID	Control, location and test	Threats	Status

the `web` and `worker` builds do not contain it (layer map, bundle scan).

- That module opens a single client as `poly_migrator`, never a pool, and closes it in `finally`. It takes `pg_advisory_lock` before `migrate()`.
- It refuses to run, and `migrate --plan` reports, when the tracking rows are not a hash-for-hash prefix of the journal. CI refuses a journal whose `when` values do not strictly increase.
- A custom migration may `SET ROLE` only to `poly_owner` or a role in the C-10 hygiene set, and it ends on `SET ROLE poly_owner` (file check). `SET SESSION AUTHORIZATION`, `RESET ROLE` and `SET LOCAL ROLE` to any other role fail the same check.
- *The end-of-file role, by privilege (ARCH).* Drizzle's `migrate()` commits inside itself, with no hook before the commit, so a check on `current_user` can only run after it. Instead, only `poly_owner` may insert into `drizzle.__drizzle_migrations`. A file that ends on another role then fails its tracking insert, and the whole run rolls back. No other role holds INSERT, UPDATE, DELETE or TRUNCATE on that table. The runner still asserts `current_user = 'poly_owner'` when `migrate()` returns, as a second check.
- *For that to hold:* `poly_migrator`'s memberships in `poly_owner` and in the definer-owner roles are granted `WITH INHERIT FALSE, SET TRUE`, and with no ADMIN option (Postgres 16 and later; the spike used INHERIT, so this is unmeasured until POLY-83). Without a `SET ROLE`, `poly_migrator` then holds none of those roles' privileges.
- *Generated migrations are current.* The `lint` job's "Generated-output freshness" step runs `scripts/cairn/check_dist_freshness.py --ci` on every PR that sets `node`, `image` or `ci`. For each `ci: true` pair in `infra/ci/generated.json` (today

ID	Control, location and test	Threats	Status

`db-migrations`), it copies the package to a temp dir outside the repo and runs `drizzle-kit generate` there, with stdin `/dev/null` and no token or `DATABASE_*` variable. It fails if the copy's `migrations/` differs from the committed one. Git history is not read, so a shallow clone or a commit order cannot make it pass. It proves that the schema matches the latest snapshot: no schema change is left without its migration. It does not prove that committed SQL is byte-for-byte what drizzle-kit would emit, and a stray committed file outside the journal is not seen. Such a file never runs, because the migrator applies journal entries only. The check fails closed: a missing or invalid registry, a missing root, a generator that is absent, fails, prompts or times out, or zero selected pairs exits 2 and turns the job red.

- Roles: `poly_owner` is NOLOGIN and NOBYPASSRLS, and `poly_migrator` is its only member. `poly_migrator` owns no object and holds no database CREATE (POLY-83 measures this). The `drizzle` schema and its table are owned by `poly_owner`, and `poly_app` and `poly_worker` get USAGE and SELECT only.

Lives at: `packages/db` migrate module and `packages/db/sql/roles.sql` (C-10); CI journal check; CI generated-output freshness step and `infra/ci/generated.json`; `layers.json`
Tested by: lint fixtures (a `SET` in any other file fails; the migrate module imported from `apps/web` fails); pgTAP over `pg_auth_members` (`inherit_option` false, `set_option` true, `admin_option` false on every grant to `poly_migrator`; no other member of `poly_owner`), the grants on the tracking table, and the owner columns of `pg_class`, `pg_proc`, `pg_namespace` and `pg_type`; POLY-100's failing-file and concurrent-run tests; a fixture where an older-timestamp migration and an edited applied file are both refused; a fixture file ending on `SET ROLE poly_authz`, with the file check bypassed, rolls the whole run back;

ID	Control, location and test	Threats	Status
	`scripts/cairn/tests/test_generated_freshness.py` (stale, fresh and every fail-closed exit) and `tests/ci/generated-freshness.test.ts` (real drizzle-kit: a schema edit without `generate` fails, and the registry pins `db-migrations` as `regenerate` and `ci: true`).		

8. CI/CD Security

- **C-80 Dependencies.**
  - Installs use `pnpm install --frozen-lockfile`.
  - pnpm lifecycle scripts are blocked: every install runs in pnpm's strict dependency-build mode, so a dependency with a build script that no entry covers fails the install. `allowBuilds` in `pnpm-workspace.yaml` holds only boolean `false` entries, which decline a package's build by name. Allowing a build (a `true` entry, `onlyBuiltDependencies` or `dangerouslyAllowAllBuilds`, in the workspace file, `package.json` or `.npmrc`) is a C-80 change and needs SecEng review.
  - `minimumReleaseAge` is at least 3 days in both pnpm and Renovate, so a freshly compromised release has time to be yanked before we pick it up.
  - Renovate never auto-merges major versions.
  - The image's `apt-get` step uses only the base image's own signed Debian sources. It adds no source or key, and uses none of `--allow-unauthenticated`, `--allow-insecure-repositories`, `--allow-downgrades` or `[trusted=yes]`, nor their `Acquire::` or `APT::Get::` equivalents. It may upgrade only. `dist-upgrade`, `full-upgrade` or a new package needs its own ruling. A static test in `tests/image` checks this.
  - The runtime image carries no package manager. `npm`, `npx`, `corepack`, `pnpm` and `yarn` do not resolve on `PATH`, `/usr/local/lib/node_modules` is absent, and the image's `User` is non-root (`node`). apt and dpkg stay only because the process is not root. This is checked on the built image, in the PR `image` job and in `build.yml` before the scan, not on the Dockerfile text.
  - Strict mode is never relaxed. `strict-dep-builds` (any spelling, including the `pnpm_config_strict_dep_builds` environment variable) is set nowhere: not in a workflow, a Dockerfile, `.npmrc` or `pnpm-workspace.yaml`. The local install, every `ci.yml` job, `build.yml` and the image all run the same strict `pnpm install --frozen-lockfile`, so a new dependency with a build script fails the pull request. `tests/ci/strict-builds.test.ts` checks both rules.
  - `pnpm audit --audit-level high` gates the build.
- **C-81 Workflow hardening.**
  - Actions are pinned by commit SHA.
  - Workflows default to `permissions: contents: read`, and widen per job only. In `ci.yml` only the `cairn` and `changes` jobs hold `actions: read`, and `GH_TOKEN` is set only on the step in each that runs `infra/ci/anchor.mjs`.
  - No `pull_request_target` that runs untrusted code.

- No secrets are exposed to fork PRs.
- Pull-request event data (`github.event.*`, `github.head_ref`) never appears as a `${{ }}` expression inside a `run:` body. It reaches scripts only through `env:` (T-93).
- `actions/checkout` sets `persist-credentials: false`. No CI job commits or pushes to the repository; CI timings leave as an artifact or a step summary.
- GitHub-hosted runners only, never `self-hosted`. Tools fetched in a `run:` step are pinned by version and checked against a sha256 before they run. No third-party path-filter action; the change filter is in-house `git diff`. A job runs on every path it executes or imports, transitively. The patterns live in `infra/ci/changes.mjs` and nowhere else. Every diff that feeds a filter runs `git diff --no-renames -z`, so a rename lists both of its paths, and a name with a non-ASCII byte, a tab or a quote is read literally, never C-quoted. `changes.mjs` exits 1 on input that is not NUL-terminated. The anchor skip (`infra/ci/anchor.mjs`, the one implementation): a pull-request push skips the heavy suites only when the head differs from an anchor in tracker and ledger files alone (`process/cairn/{issues,milestones,majors,archive,reviews}/` other than the pinned rulings, `process/STATE.md`, `process/DECISIONS.md`). The anchor is the head of a successful `pull_request` run of `ci.yml` from this repository on the same branch. It must be present in the clone, be an ancestor of the head, and contain the current base. No symlink or gitlink may appear in that diff. The tests that read ledger content and `cairn check` still run. Any lookup, ancestry, diff or mode failure falls back to the full base-to-head filter. The aggregate is red when a filter output is missing or is not exactly `true` or `false`. A static test in `tests/ci` walks the battery's imports and fails on a file the `db` filter does not match. The pin check fails on a tag, a branch, a short sha, or a `uses:` it cannot parse.
- The attested build restores no cache: buildx `no-cache` and `pull`, no `type=gha`, no `actions/cache` restore. PR jobs may cache the pnpm store, keyed by the lockfile hash only (T-94).

- **C-82 Scanning.**
  - gitleaks runs in CI and as a pre-commit hook. In CI it runs on every PR, outside the change filter, so a docs-only PR is scanned too.
  - CodeQL (JS/TS) runs on PRs. Not yet built: POLY-102 builds it, and until then this line is a target, not a control in force.
  - Trivy scans images and fails the build on fixable critical or high findings and on embedded secrets.
  - Each release carries a CycloneDX SBOM.
  - Each release image gets an `actions/attest-build-provenance` attestation.

- In `build.yml` the order is build, push by sha tag, then Trivy on the pushed digest, then SBOM (from the image, attached to the same digest), then attest. Nothing is attested unless Trivy passed, so a failing image may sit in GHCR, but no box will run it. A `.trivyignore` entry carries a CVE, an expiry date and an issue id, and seceng reviews it. An expired entry fails the scan.
- No DAST in v1 (R-13).

- **C-83 Deploy path. Pull-based (ARCH §8.1; built in INFRA §4.4, D-5).** GitHub holds no credential that can reach a box. A systemd timer on the box polls every 5 minutes, pulls by digest, and runs nothing until each check passes:

  1. `gh attestation verify --repo richmosko/polycarpic --signer-workflow .github/workflows/build.yml --source-ref refs/heads/main --deny-self-hosted-runners` on the digest. The signer workflow alone does not show that the image was built from `main`: a branch that edits `build.yml` gets an attestation with the same name (T-92);
  2. for prod, `git verify-tag` against `/etc/polycarpic/allowed_signers`;
  3. for prod, the tag's commit equals the commit in the image's label. The label is inside the attested image, so a validly signed tag cannot be paired with a different attested image.

  Any failure logs, alerts and stops, and the failed digest is quarantined.

  - **The signer list comes from the operator.** `allowed_signers` is installed at bootstrap from the operator's own machine and changed only by an operator step. It is never fetched from the repository, where anyone who can push could add a key. The deploy script and its timer change the same way, through `polycarpic-ops self-update`, which runs the same checks.
  - **The box token.** A fine-grained token, this repository only, `contents:read` and `packages:read`, used by the host deploy script and mounted into no container. It expires after 90 days, and a monitor check fires 14 days ahead (INFRA §6.5, §8.4). It is deleted if the repository and the package become public.
  - **Residual.** The attestation proves the image was built by `build.yml` in this repository from that commit, not that the commit is safe. The signed tag is the human gate, so whoever holds the signing key holds prod.
  - **Tested by:** on staging before the first prod deploy, an image attested by another workflow or repository is refused; an image attested by `build.yml` run from a branch other than `main` is refused; a tag signed by a key not in `allowed_signers` is refused; a correctly signed tag whose commit differs from the image label is refused; the token reminder fires at 14 days.

- **Who builds.** `build.yml` runs on a push to `main` and on its weekly schedule only, and a manual dispatch is refused off `main`. The weekly run is a full rebuild in `build.yml`: push, scan, SBOM, attest, with no cache. It exists because the runtime stage runs `apt-get upgrade`: Debian security fixes land between base-image digests (10 fixable HIGH were measured on the pinned digest). So a commit can carry several attested digests over time. The image is a product of the commit and the signed Debian archive at build time. The attestation and the SBOM record what shipped; byte-for-byte reproduction from the commit alone is not claimed.
  - Each run pushes a tag unique to that run (`sha-<commit>-<run_id>`). No shared tag (`sha-<commit>`, `main`, `latest`) is created or moved before that run's attest step passes. `release.yml` and the staging poll take the digest from the newest successful `build.yml` run's `digest.txt` on `main`, and never resolve a tag. A failed weekly build leaves every box on the last attested digest.
  - Prod's digest is frozen at its tag, so prod is patched only by a new tag. The weekly run therefore also runs Trivy, scan only, on the digest of the newest published `v1.*` Release. A red result is an alert under INFRA §5's windows and deploys nothing.
  - Within one run, the scanned digest, the SBOM subject, the provenance subject and `digest.txt` are the one build-step output.
- **release.yml is a fail-fast mirror, not the authority.** On a `v1.*` tag it checks the tag's signature against a signer list held in a repository variable the operator sets. It confirms the tag's commit is on `main`, and finds the newest attested digest for it. It verifies that digest with the same flags as the box, and checks that the image label equals the tag's commit. It refuses to overwrite an existing Release. It never builds, pushes or installs, and it holds no `packages: write`. A manual run takes a commit and runs the checks only; the publishing job cannot be reached from it. The box's `allowed_signers` stays the authority.

- **Signing.** `/merge-pr` signs release tags (SSH signing) before the first prod deploy, because a tag triggers deployment. Per-commit signing is not required (R-10).

9. Logging, Monitoring & Detection

- **Logged:** request ID, user UUID, entity UUID, route, action, outcome, latency, job name and run ID.
- **Never logged:** T1 or T2 values, request or response bodies, memo or payee text, amounts, emails, cookies, `Authorization`, SQL parameters. The C-33 canary test enforces this.
- **Storage and retention.** JSON logs stay on the box, rotated, for 30 days. Postgres logging is set per C-75. An optional OTel export must pass through the same serializers.
- **Operator alerts**, sent by email through Resend:
  - more than 20 sign-in failures in 10 minutes;
  - MFA disabled, or a new passkey added;
  - a membership grant;
  - a provider link or unlink;
  - an export or a purge;
  - 3 failures of a revoke job;
  - a connection going `principal_invalid`;
  - a backup failure, or no successful backup within the RPO;
  - an RLS battery failure on deploy (this also blocks the deploy);
  - more than 50 authz denials or empty scopes per user in 10 minutes.

  The thresholds are provisional and unmeasured; tune them after POLY-M1. Email on these events costs almost nothing.
- **User-facing notices.** Users get an email for a sign-in from a new device, a password or MFA change (including a first factor enrolled), and being granted or removed from an entity.
- **Audit.** `audit_event` (C-53) survives purge. The journal actor columns (C-23) are the ledger's own audit trail. C-24, the per-entity hash chain, makes ledger edits detectable. A weekly verifier checks it, and every export carries the chain head.

10. Incident Response

- **On call:** each instance's operator. For the Principal's instance, that is the Principal. Alerts arrive by email (§9).
- **Severity:**
  - **SEV1**: suspected T1 or T2 exposure, or keyring exposure; a confirmed cross-tenant read or write; code execution on the box; ledger tampering detected.
  - **SEV2**: T3 exposure limited to one user; no backup within the RPO; a revoke that cannot complete; a critical CVE with a known exploit in a deployed dependency.
  - **SEV3**: a vulnerability with no evidence of exploitation; a threshold breach that turns out benign.
- **Runbooks.** These cover the security content only; exact commands live in the INFRA runbook.
  - **RB-1: keyring or worker compromise (SEV1).**
    1. Stop `worker`.
    2. Revoke every token at its provider: Plaid `/item/remove`, SimpleFIN bridge revocation, broker OAuth revoke. If the database and the keyring may both have been exposed, treat every token as burned.
    3. Generate a new keyring.
    4. Relink every connection.
    5. Rotate the provider API secrets.
    6. Review `audit_event` and the logs for credential opens that happened outside import runs.
  - **RB-2: web or box compromise (SEV1).**
    1. Put Caddy into maintenance mode.
    2. Snapshot the volume for forensics.
    3. Rotate every operator secret, including the BetterAuth secret. This invalidates all sessions.
    4. If the worker or the keyring was reachable, run RB-1.
    5. Rebuild from a clean, attested image on a fresh host.
    6. Restore the last backup taken before the compromise, and verify it against the C-24 chain and against the head of the last export the owner holds off the box.
  - **RB-3: cross-tenant leak bug (SEV1).**
    1. Disable the affected route.
    2. Add the failing case to the RLS battery or the authz matrix *before* fixing it.

3. Identify affected users from request logs, by route and user ID.
4. Notify the affected members.

- **RB-4: lost keyring (SEV3).** Generate a new keyring. Every connection then shows "relink required". The ledger is unaffected.
- **RB-5: backup failure or restore (SEV2).** Follow INFRA. Restore to staging first, and run the balance and hash verifier before promoting.
- **RB-6: compromised dependency (SEV1 or SEV2).**
  1. Pin to the last known-good version.
  2. Rebuild and redeploy.
  3. Diff the SBOMs to check whether the bad version shipped in a release image.
  4. If it did, run RB-2.

- **Post-incident:** a blameless postmortem within 5 business days as a cairn issue comment or review file (location per `process/WORKFLOW.md`). Record decisions in process/DECISIONS.md. The regression test lands with the fix.

11. Open Risks (Accepted)

Each risk has a rationale, a compensating control, a re-evaluation date and an approver. ARCH cites R-1 and R-4.

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-1	The operator's superuser, and root on the box, bypass RLS. They can also disable the append-only triggers (`ALTER TABLE … DISABLE TRIGGER`, `session_replication_role`) and read the keyring.	The operator owns the box, the volume and the keys. No database control can bind them (PRD §6).	C-24 hash chain (POLY-M2): silent edits become detectable, and the chain head goes into every export. Superuser access only through `docker compose exec` or an SSH tunnel. C-71.	On any decision to host for others	Richmosko
R-2	RLS does not stop SQL injection or code execution in `poly_app` or `poly_worker`. Each process asserts its own tenant GUC, and Postgres verifies nothing (`app.current_user_id()` reads `app.user_id`, which any role can set).	Nothing is deployed, and the target design removes most of this reach. • **Target.** Sign-in runs as a separate service. It is the only holder of the auth secret and the only role that reads sessions, verification tokens and second-factor secrets. Postgres sets a	• C-44 (parameterized SQL, lint, CodeQL); C-30 (`web` cannot open credentials); C-10 (no DDL or TRUNCATE).	As the first stage (POLY-147) and the target	Richmosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	• SQL as `poly_app`, through an injection bug or the stolen password, acts as any user. It can also read every live session token (useless without the auth secret, R-12). It cannot redeem a token it reads from `verification`, which holds only hashes (T1). It can still write there: `poly_app` holds INSERT on `verification`, so it can plant a magic link for any address and redeem it for a real session, which outlives the injection until it is revoked. • Code execution in `web` adds the auth secret. It can turn any stored session token into a valid cookie and decrypt	transaction's user only from a live session, through one definer. The definer records the user in a transaction marker keyed by the transaction id, which `poly_app` and `poly_worker` cannot write. `app.current_user_id()` reads the marker, never a GUC. A session reaches the database through a handle stored only as a hash. The worker acts only through grants scoped to one connection's import path. • **Hardening in place.** BetterAuth's verification tokens are stored hashed (T1). The auth secret is mounted into `web` alone (§6.1). The operator CLI runs in a one-shot `operator` service as its own login role, `poly_operator`, with `EXECUTE` on the	• On the sign-up path, such code cannot act as an existing user, but it can create a new account for an invited address and take that address's invitations (C-84). • On the operator path, only `poly_operator` can execute the issuing definer, and it refuses any transaction with a user set (C-15). An operator invitation	(POLY-149) land, and no later than 2027-03-31	

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	every TOTP seed, backup code and OAuth token. Its reach lasts until the secret is rotated, every session is revoked and every user re-enrols. • SQL as `poly_worker`, or code execution in `worker`, acts as any user within the worker's grants: every tenant row the worker can read, and the import-path writes (§4.4). It cannot issue invitations: only `poly_operator` can execute the issuing definer, and `worker` never holds its password (C-15, C-84). `worker` holds no auth secret (§6.1). Apart from identity, it opens	issuing definer only (C-84). • **Wording rule while only the first stage is in place.** The first stage is the marker, a scoped worker entry, and an interim definer through which `web` still names the user. While it alone is in place, this row says the following, and neither "closed" nor "unchanged". The worker can no longer name an arbitrary user. A single injected statement inside one of `web`'s marked request transactions reaches only the requesting user. That holds on three conditions, each with its own test: every `web` query is sent in the driver's extended mode, so a two-statement string is refused; the marker is the first statement of the real	shares no entity, carries role `viewer`, and expires in 7 days (C-01). Each issue and revocation writes an `audit_event` whose actor is fixed by the definer, not by the caller (C-01).		

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	every sealed bank credential (C-30).	`BEGIN`, so rolling back to a later savepoint leaves it in place; and a second marker in a transaction is refused, with `poly_app` holding no `UPDATE` or `DELETE` on the marker table. Any user is still reachable through an injection on a path `web` runs outside a marked transaction (BetterAuth on the plain pool, sign-up), through a stolen `poly_app` password, and through code execution in `web`. • **Residual once the target is in place.** The sign-in service can create a session for anyone, so its compromise acts as any user. It is exposed only through the `/api/auth/*` routes, with no pages of the app, no uploads and no provider data, but			

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
		BetterAuth is still a large library facing the internet. Code execution in `web` still acts as every user whose requests pass through it while the attacker is there. It cannot mint an identity after eviction. If `web` serves the sign-in pages, it can alter them and capture passwords as they are typed.			
R-3	On Vercel, both projects live in one Vercel account. Anyone with access to that account or team, or a leaked Vercel token with env-read scope, can read the `jobs` keyring. Vercel functions are also public HTTPS endpoints, so the `jobs` project is reachable from the internet. It is guarded only by `CRON_SECRET`, with no user input. *Narrowed by*	Vercel is the tertiary, last-resort target. The residual risk is administrative (account-level), not a web-process compromise.	Separate projects and env scopes (C-61). Vercel account MFA and least-privilege team membership. Rotate `CRON_SECRET` on suspicion. If the two-project split proves infeasible, Vercel may run only the fake or	Before POLY-77 ships on Vercel	Richmosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	OQ-13: the web/worker split is kept.		CSV adapters, never a real aggregator credential. Hobby-plan cron runs at most once a day (measured, ARCH §7.6 @ e3b4297). On Vercel, provider-side revocation after an unlink can therefore lag up to about 24 h, and the token stays live at the provider until then. The unlink UI must say so.		
R-4	Staging shares the prod kernel and auto-deploys unreleased code from `main` (TB-10). Staging holds its own keyring and Plaid Sandbox / SimpleFIN demo tokens.	It saves a second box. Staging's data and tokens reach no real bank, so what is at risk is a pivot into prod, not staging's own data.	C-70 (separate compose networks, non-root, read-only containers, no Docker socket); C-74 (distinct secrets); C-77	ARCH §7.2 @ 6b2f11b keeps	Richmosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
			(staging's own keyring, sandbox secret only, refuses production endpoints); staging sits behind Caddy basic auth or an IP allowlist (ARCH §7.1).	staging co-located. Re-evaluate if staging ever touches non-fake data, or the app is offered at larger scale.	
R-5	Purged entity data persists in backups for	Rewriting encrypted backups is	The purge confirmation	If rete	Ric

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	up to 35 days (D-4).	disproportionate, and the lock that makes backups trustworthy also forbids it.	says "up to 35 days in backups" (A-35). The store enforces 35 days; there is no long-term archive tier in v1.	ntion changes	hMosko
R-6	The multi-tenant design leaves a hosted offering possible. Reversing the non-goal would bring in GLBA, state privacy law and possibly PCI, and would make R-1, R-2 and R-13 unacceptable.	Hosting for others is a standing non-goal (PRD §3).	This row makes any reversal a deliberate, costed decision that re-opens §1.	Only on a deliberate decision to host	RichMosko (PRD §3)
R-7	Viewers are not required to use MFA, nor is the owner of a user's own Person while it is unshared (OQ-34). Owners and managers elsewhere are.	Viewers cannot mutate data, link credentials or grant access. The unenrolled own-Person owner can edit only their own unshared books; exporting, purging, sharing,	C-04 rate limiting; onboarding leads with OAuth; new-device email (§9).	GA	RichMosko (

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
		granting and linking a bank all force enrolment first.			OQ-34)
R-8	Losing the keyring makes sealed credentials unrecoverable, and rotation is a manual operator step.	Automating rotation would mean storing the new key somewhere automation can reach. Unrecoverable is the right failure mode here, because recovery is simply relinking.	The rotate command is idempotent and resumable (C-35); the runbook states the failure mode plainly; the ledger is not encrypted under this key.	Annually	Rich Mosko
R-9	User-defined entity types are readable across tenants, by anyone with a membership on an entity of that type (kickoff §2.4).	A viewer needs the type label to render the entity.	A narrow policy plus a column-level SELECT grant expose the label and kind only, never the creator ID (C-14). An entity may use only a system type or one its creator owns, so a type id	If types ever carry more than a label	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
			seen once does not grant a lasting read. Users are told that custom type names are shared.		
R-10	Commits are unsigned.	With per-agent worktree identities, commit signatures prove little. Branch protection already prevents unreviewed merges.	Branch protection (admins included, no force-push, required `cairn` check). Release tags are signed (§8).	First prod deploy	Rich Mosko
R-11	Revoking a grant does not recall data a client already fetched: an open browser tab now, the native cache later.	This is inherent to any client. The next request is denied.	The native milestone must purge its cache on 401, 403 or an empty scope. SecEng reviews that milestone.	Native milestone	Rich Mosko
R-12	*Confirmed (measured, better-auth 1.7.18).* BetterAuth stores the session token in the	A DB read already exposes T3. The auth secret lives in the host secrets directory, not in the database (§6.1).	Short sessions (C-02); step-up for sensitive actions (C-03);	POLY-62	Rich M

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	clear in `session.token`; the stored row matched the cookie token. It has no hashing option. A hook that rewrites the token breaks the lookup, because `findSession` matches on the raw token. The browser cookie is `token.HMAC(auth secret)`. A DB read alone cannot forge one. Nor can it redeem a magic-link or reset token, because `verification` holds only a SHA-256 hash of each identifier (T1). On a miss BetterAuth also tries the identifier unhashed, but a stored value presented as a token is still refused: the route adds its prefix, and a stored hash has none (code-read, 1.7.18; tested). A DB write can still plant a magic link		the `bearer` plugin off, or on only with `requireSignature: true` (§4.1).		osko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	(R-2). A DB read plus the auth secret can forge a cookie, and so can a DB read alone if the `bearer` plugin runs without `requireSignature`.				
R-13	No DAST, WAF or penetration test.	The surface is small and sits behind Caddy, and no regime requires one. pgTAP asserts the high-value properties more precisely than a scanner would.	The RLS battery; SAST, dependency, secret and image scanning (§8); this threat model is reviewed at every milestone.	GA	Rich Mosko
R-14	Cross-root read access (PT-3, tooling). `cairn serve` reads directories named in a committed `roots:` config (or `--repos`), which can be outside the repo. A malicious `roots:` entry in a PR could point a reviewer's local board at a sibling directory, e.g. `../../.ssh`.	The server is read-only and localhost-only, with no write path to a secondary root. The `/api/board` payload carries only parsed cairn frontmatter, so there is no exfiltration channel. A directory without a `config.yml` is skipped before anything is read. Anyone who can commit a `roots:` entry already has a stronger vector, editing	When more than one root loads, a startup stderr banner logs each resolved absolute root path. It is a human tripwire against a stale or mistyped `roots:` entry silently serving	2027-02-21	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
		`cairn.py` or `board.js` directly, so a runtime path-jail would be theater.	the wrong tracker. Re-evaluate if cairn ever gains a write path to non-primary roots.		
R-15	A parent-link or cap-table refusal ("would make an entity its own ancestor", "too deep") reveals that some link exists, possibly through entities the user cannot see (T-78).	The inserter must own both ends, so the bit is about paths between their own entities. The error names no entity, and naming nothing would make the refusal unexplainable.	Errors name no entity (C-25). Refusals are audited.	If links ever become insertable by a non-owner	Richmosko
R-16	A purge, including one inside an account deletion, runs immediately with no cancellable delay. A hijacked session that passes step-up can destroy an entity's books at once.	A delay needs a background purge the design forbids, and would make the freshness guards stale.	Step-up within 5 minutes (C-03, with a second factor required for owners). Typed entity name. A fresh export by the same user. Every member emailed	If a purge is ever reported as unwanted	Richmosko (OQ

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
			immediately. Operator backups (C-73). R-5 bounds how long recovery is possible.		-23)
R-19	Hostile or captured founder. Other owners cannot remove the founding owner in-app. A founder, or someone in control of the founder's session, can force out co-owners one by one, each after a decline or the 14-day expiry.	Without a tie-breaker, a two-owner dispute blocks forever. The founder's extra power is limited to co-owner removal or demotion, and is never immediate.	Step-up with the second factor (C-03). The decline or expiry wait. A notice to every owner, the target included, plus `change.forced` audit events (C-28). No extra founder power over invites, roles or the cap table. The operator CLI as the backstop.	On the first reported forced removal	Richmosko
R-18	Real-domain mode on the local box keeps a zone-limited DNS-API token on the box (T-68). Whoever steals it can	It is what keeps Google and GitHub sign-in on a box nothing can reach. The box is unreachable from the	The token is scoped to one zone and mounted into Caddy only (C-	On any change to	Richmo

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	issue certificates for that zone, or publish records in it.	internet, so a stolen token gets no path in.	76). It is rotated at the provider on suspicion. CT-log monitoring of the zone is optional.	the local-box TLS mode	sko
R-20	Private-CA mode installs an operator root on each device (T-79). If browsers do not enforce its name constraint, a stolen root key could impersonate any site to those devices.	It is the supported fallback for an operator without a domain.	A name-constrained root, its key offline, and only an intermediate on the box (C-76). The runbook says to remove the root from every device on decommission or on a move to a real domain.	At the first private-CA deploy, after measuring constraint enforcement. That measurement	RichMosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
				is a gate item for the first home-box deploy.	
R-21	With TPM auto-unlock, a thief who takes the *whole* home box gets a machine that boots and unlocks its disk. The data is then protected only by OS login, SSH keys and app authentication, and remains exposed to an OS or service exploit, or to cold-boot and DMA attacks on the running machine. It is also exposed to anyone who opens the case and alters what boots (T-85). PCR 7 does not	A passphrase at boot would leave the box down after every power cut. Theft of a whole box from a home is rare, and a pulled disk stays unreadable.	C-72: sealed to Secure Boot, firmware password, no USB boot, no unauthenticated console. Unattended updates (C-76). Backups encrypted separately (C-73). On theft, run RB-2: rotate every secret and revoke every bank token.	If the box moves out of the home, or holds data for others	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	measure the boot files. The root partition is encrypted too, so the simple edit there does not work. The unencrypted kernel and the unsigned initramfs remain: someone who rebuilds the initramfs can capture the unlock. That takes more skill than a file edit, but it is published.				
R-22	Whoever holds an invitation link can sign up as the invitee by password, so a forwarded or leaked link (T-95) is as strong as a forwarded magic link.	The link went to the invitee's inbox, which is already the recovery root (magic link, password reset).	Single use, expiry, revocation (C-01); the account stays unverified and an IdP sign-in for the address does not link into it (C-05); the real invitee recovers by magic link and a password reset, which revokes	POLY-90	RichMosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
			sessions (C-02).		
R-23	An IdP vouches for an email it controls: a Google Workspace admin can mint a verified address in their domain, and a GitHub account can keep a verified address after leaving the employer that issued it. Either can claim invitations addressed there (C-05).	Email is the recovery root anyway; an invitation is a deliberate act by an owner who chose the address.	Owners revoke unclaimed invitations; a manager or owner invitation seats only after a second step is enrolled (C-03); the C-05 provider rules.	POLY-90	Rich Mosko
R-24	A sign-in on `/callback/:id` holds a `poly_app` pool connection across BetterAuth's outbound calls to the IdP, and neither `/callback/:id` nor `/magic-link/verify` is in C-04's limited list. Many concurrent callbacks can exhaust the pool.	A household instance has few users; each callback needs a valid `state` from a started flow.	The 5-second `lock_timeout` on invitation rows (C-15). A rate limit on those two routes and `idle_in_transaction_session_timeout` for `poly_app` are open (POLY-137).	First prod deploy	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-17	Aggregate inference. Roll-up figures shown to a parent's members, the economic view, and allocated income shown to a holder's owners can be combined to infer things. Dividing economic by gross gives the viewer's own look-through stake. Allocated income divided by stake gives a pass-through entity's tagged income totals, as a K-1 tells a partner. Observing "today" figures every day accumulates detail over time. A parent member may request roll-up totals for any date range. For a small entity, one-day periods and day-to-day balance differences reveal individual amounts and their dates, though never payee, memo,	Each figure is one the viewer is entitled to. Inclusion in a parent is an explicit grant by the entity's owner, so the exposure is consented, provided the consent is honest.	Consent disclosure at inclusion and in the upward-consent notice (C-25). The fixed aggregate shapes (C-25, C-27). Own and member entities are unaffected.	With POLY-1.1, and on any cross-parent "my net worth" proposal (OQ-29)	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	account or counterparty.				

12. Rulings on ARCH Open Questions

12.1 Final rulings

Each question SecEng ruled on is answered in the body; this table points at where.

Question	Answered in
OQ-1	§6.3–§6.5; C-30, C-31, C-35
OQ-2	§4.2.1, §4.3; C-10 to C-20
OQ-3	§4.4; C-60
OQ-4	§4.5; C-21, C-50
OQ-19	§3.8; C-19, C-25
OQ-23	§4.5; R-16
OQ-24	C-76; T-67 to T-69
OQ-25	§3.8, §4.6; C-25, C-27, C-52, C-77
OQ-30	C-10, C-15, C-24; T-80; A-23
OQ-31	§4.2.2; C-28; A-24
OQ-32	C-76; A-26
OQ-34	§4.1; C-03; R-7; T-81; A-25, A-28
Email-anchored first enrolment	§4.1, §4.6; C-03, C-52; T-82 to T-84; A-29, A-30
Bootstrap	§4.1, C-01, T-01
BetterAuth resolutions	§4.1, C-03, A-33, A-34
`poly_backup`	C-79, C-10, §6.1, T-89
Home-box layout	C-72, T-85, R-21
Route table	§4.1, C-03, T-86, A-38
Layer map	§3.3, §6.3; C-11, C-30, C-41; A-39

Question	Answered in
POLY-76 ruling	C-84, C-85; T-90, T-91; A-40
OQ-INFRA-1	§6.1, §6.6, INFRA §6.6

12.2 Where the A-items live in ARCH

The A-numbers cited in this document are SecEng requirements on ARCH. Each is part of the ARCH body.

Items	ARCH section
A-1 to A-5, A-10, A-12	ARCH §3.2, §3.3
A-6 to A-9, A-11, A-14	ARCH §3.1, §2.3 (A-8's deletion wording also in PRD §5)
A-13	ARCH §6.2 (e)
A-15	ARCH §7 (TB-11, TB-12)
A-16 to A-22	ARCH §3.1a, §3.1b, §3.3b, §3.3c, §3.4b
A-23	ARCH §3.2.9
A-24, A-25, A-28 to A-34, A-38	ARCH §3.1c
A-26, A-27, A-37	ARCH §7.8, §2.2
A-30, A-35	ARCH §3.4, §3.4a
A-36	ARCH §3.3 roles, §7.2
A-39, A-40	ARCH §2.1, §3.1, §3.3, §6.2 (A-39's per-connection opener test is POLY-130's)

12.3 Asks on other owners

- **devops-engineer:**
  - **The `provision-check` review (INFRA §9.3).** The check lists in INFRA §7.1 step 11 and §7.2 step 5 are accepted as the evidence for C-70, C-71, C-72, C-76 and C-77, with the additions in C-71 and C-72. SecEng reviews the script itself when it exists (POLY-M1).

- **qa-engineer:**
  - Every row of §4.2.1 becomes a pgTAP assertion.
  - The battery adds: pool reuse on a dirty backend (C-12); WITH CHECK forgery; catalog coverage plus canary; the definer allowlist; `security_invoker`; TRUNCATE; purge negatives and the zero-rows-remain check (C-50).
  - The log canary (C-33) must land before the first real adapter.
  - The C-19, C-25 and C-27 tests (graph triggers, roll-up shapes, holder reads), the §4.6 deletion-flow negatives (C-52), and the C-77 config refusals.
  - The C-24 `poly_journal` tests, the C-28 1 → 2 revocation tests, the C-03 held-role tests, the C-05 per-provider verification tests, the C-76 startup matrix, and the C-25 consent-text exact match.
  - The C-03 OQ-34 exemption tests, and the A-28 roll-up predicate test.
  - The C-03 email-anchor tests (including the direct endpoint calls), the A-29 pgTAP guard, and the §4.6 unenrolled-deletion tests.
- **product-manager and ux-designer:** the purge confirmation, and PRD §5's purge text, say "up to 35 days in backups" (R-5).
